

長庚大學電機系 115 年度專題成果報告書

結合 LiDAR 與 Camera 之 AMR 智慧導航系統

全程執行期限

自民國 114 年 9 月 1 日起至民國 115 年 6 月 7 日

研究專長屬性

☐通訊 ☐醫工 ☐電路 ☒控制 ☐電力 ☐其它

組員：

姓名：陳品頤 學號：B1221110

姓名：黃蘊錚 學號：B1221122

姓名：李欣霓 學號：B1221136

指導教師： 張永華 教授

中華民國 115 年 6 月 8 日

摘要

本專題以「基於 Sensor Fusion 之 AMR (Autonomous Mobile Robot) 智慧導航系統」為核心，整合 LiDAR、camera、ROS (Robot Operating System) 與 TurtleBot2 移動平台，建置具備室內建圖、定位、路徑規劃、避障與影像感知能力之自主移動機器人。系統以 RPLiDAR A1 取得環境距離資料，完成 SLAM (Simultaneous Localization and Mapping) 建圖、AMCL (Adaptive Monte Carlo Localization) 定位、costmap 與 LiDAR-based 避障流程，使 AMR 具備基本自主導航能力。考量 2D LiDAR 掃描高度固定，對低矮障礙物或近地面小物可能產生感測盲區，因此加入 camera 作為輔助視覺感知來源，以補足 LiDAR 在特定場景下的限制。

系統採功能互補式 Sensor Fusion 架構，LiDAR 負責距離量測、地圖建立、定位與主要避障，camera 則負責低矮障礙物輔助判斷與語意影像辨識。LiDAR 可補足 camera 在暗處或光線不穩時的不足，camera 則補充 LiDAR 無法取得之視覺與近地面資訊。受限於 RGB Camera 無深度距離與 Raspberry Pi 4 運算延遲，目前 camera 輔助避障以出發前的判斷為主，並以固定視角、固定距離與固定大小方式建立輔助障礙區域，提供 planner 進行路徑規劃參考。測試結果顯示，本系統可完成地圖構建、定位、路徑規劃、即時避障與物件辨識等功能，並驗證 LiDAR 與 camera 互補感知應用於 AMR 智慧導航之可行性。

關鍵詞：ROS、SLAM、Sensor fusion、深度學習、TurtleBot

目錄

第一章、緒論	3
1.1 研究背景與動機	3
1.2 研究目的	4
1.3 系統架構圖	4
第二章、理論基礎	5
2.1 樹莓派	5
2.2 ROS	6
2.3 光達	8
2.4 SLAM 簡介	10
2.5 AMCL	12
2.6 Dijkstra 演算法	14
2.7 A*演算法	15
2.8 Dynamic Window Approach	16
2.9 點雲資料與 scan 資料流	18
2.10 Costmap 與路徑規劃概念	21
2.11 RGB+HSV 六通道 CNN 顏色辨識	23
2.12 TF Tree (Transform tree)與座標轉換原理	25
2.13 TurtleBot	27
第三章、實作或模擬過程	29
3.1 系統開發流程與功能配置	29
3.1.1 硬體平台建置	29
3.1.2 感測器資料擷取	32
3.2 ROS通訊與座標整合	32
3.2.1 Topic 資料流設計	33
3.2.2 TF Tree 與座標轉換設定	35
3.3 模擬環境與導航建置	37
3.3.1 Gazebo 模擬環境建置	38
3.3.2 SLAM 建圖流程	40

3.3.3 AMCL 定位與 RViz 視覺化.....	42
3.3.4 Costmap 與路徑規劃設定	45
3.3.5 自主探索與避障演算法實作	47
3.4 Sensor Fusion 感知架構設計	52
3.4.1 Camera 輔助低矮障礙物避障流程.....	53
3.4.2 有無障礙物辨識模型訓練	55
3.4.3 RGB+HSV 六通道 CNN 顏色辨識流程實作	58
3.5 醫療病房巡檢任務流程	61
3.5.1 巡檢點位與移動順序規劃	63
3.5.2 異常辨識與任務切換設計	65
第四章、測試或模擬分析結果與討論	67
4.1 系統整合測試結果	67
4.2 SLAM 建圖與定位結果分析.....	68
4.3 路徑規劃與 LiDAR 避障測試結果.....	69
4.4 Camera 輔助低矮障礙物避障測試結果	72
4.5 有無障礙物辨識模型訓練	76
4.6 口罩辨識結果分析	79
4.7 顏色辨識結果分析	81
4.8 醫療巡檢應用流程測試結果	83
第五章、結論與自評	89
5.1 研究收穫與結論	89
5.2 未來改善建議與發展展望	91
第六章、參考資料	94
第七章、指導老師簽意見（簽名）	97

第一章、緒論

1.1 研究背景與動機

隨著智慧製造、智慧醫療與自動化巡檢需求逐漸提升，自主移動機器人 AMR 在室內場域中的應用也受到重視。AMR 不同於傳統固定路線或人工遙控載具，其核心價值在於能夠自主感知周遭環境，成為本專題的重要研究方向。

自主移動機器人需具備穩定的環境感知能力，才能在室內場域中完成導航、避障與巡檢任務。LiDAR 因具備穩定距離量測能力，常被用於 SLAM 建圖、定位與路徑規劃。然而，2D LiDAR 主要掃描固定高度的水平平面，對於低於掃描高度的電線、地毯邊緣、門檻或地面小物可能無法完整偵測。此情況在日常生活中的掃地機器人也相當常見，例如：機器人可能捲入電線、卡住地毯或被低矮障礙物阻擋，顯示單一感測器在複雜室內環境中仍有其限制。

為改善此問題，本專題導入 camera 作為輔助感知來源，透過影像補足 LiDAR 在低矮區域感測不足的問題。camera 可觀察車體前方近地面範圍，也能進一步支援顏色辨識、物件辨識與口罩辨識等語意感知任務。系統透過 ROI (Region of Interest) 切分，使下方 ROI 用於低矮障礙物輔助判斷，中間 ROI 用於顏色標籤或狀態辨識，上方 ROI 則可用於口罩或物件辨識。透過 LiDAR 與 camera 的 Sensor Fusion，AMR 可同時取得距離資訊與影像資訊，提升室內導航、避障與巡檢應用的完整性。

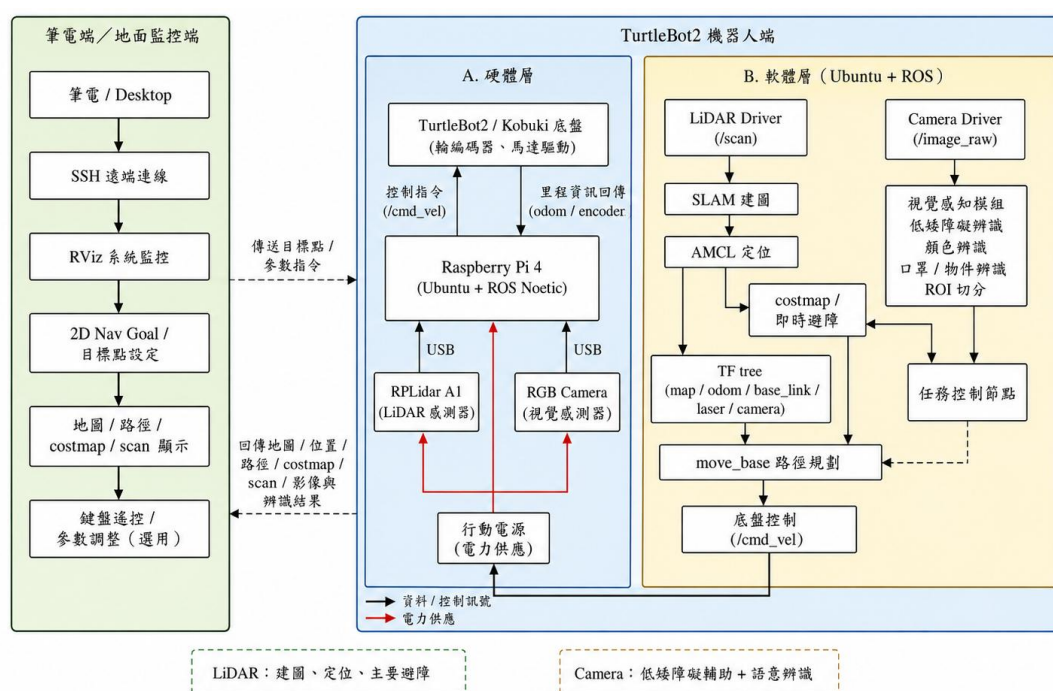
除了輔助避障外，Camera 也能擴展為 Camera-based 視覺感知模組，使機器人具備更多元的巡檢功能。本專題將視覺感知模組分為低矮障礙辨識、顏色辨識與口罩辨識三個子模組。低矮障礙辨識模組主要用於判斷機器人前方近地面是否存在 LiDAR 難以掃描到的障礙物，以輔助導航避障；顏色辨識模組可用於辨識指定顏色標籤，作為任務觸發、區域判斷或流程切換的依據；口罩辨識模組則可用於判斷人員是否配戴口罩，支援智慧病房巡檢與遠端監控應用。透過 Camera 的語意辨識能力，AMR 不再只是單純移動平台，而能進一步理解巡檢場域中的目標、標籤與人員狀態。隨著影像辨識與感測器融合技術的發展，AMR 可結合 LiDAR 與 Camera，同時取得距離資訊與視覺資訊，機器人不僅能自主移動，更能進一步判斷環境狀態並依任務需求做出反應。這些可應用於智慧病房巡檢、工廠巡檢、室內服務與遠端監控等場域，使自主移動機器人在實際環境中發揮更高的安全性、彈性與應用價值。

1.2 研究目的

本專題之主要目的包括以下幾點，第一：建立以 LiDAR 為基礎的室內導航流程，使 AMR 能透過雷射掃描資料完成環境建圖、定位與路徑規劃，形成自主移動所需的基本架構。第二：加入 camera 視覺感知模組，補足 2D LiDAR 對低矮障礙物或近地面小物感測不足的問題，使系統能取得車體前方影像資訊，提升導航安全性。第三：透過 ROI 切分方式，使同一 camera 可依任務需求支援不同影像功能。此外，系統亦希望將 camera 影像辨識結果轉換為可供任務判斷使用的資訊，使 AMR 不僅能移動至指定位置，也能依據感知結果進行後續任務切換。顏色辨識、口罩辨識與物件辨識皆可視為視覺感知功能的應用方向，其中顏色辨識可用於工廠、醫院或巡檢場域中的標籤判斷與事件驅動控制；口罩或物件辨識則可延伸至遠端監控與安全巡檢需求。

醫療病房巡檢在本專題中作為 Sensor Fusion AMR 系統的應用驗證情境，用以展示導航、避障與影像感知功能如何整合為連續任務流程。透過巡檢點位規劃與任務切換設計，可驗證 AMR 是否能依序前往指定位置，並依據感知結果進行狀態判斷或目標切換。此應用並非限制系統只能用於醫療場域，而是作為智慧導航與感測融合能力的具體展示。

1.3 系統架構圖



圖一、TurtleBot 智慧導航系統架構圖

第二章、理論基礎

2.1 樹莓派

樹莓派（Raspberry Pi 4），是一款由英國Raspberry Pi基金會開發的微型單板電腦（Single Board Computer, SBC）。主要設計理念是提供一個具備完整運算能力的微型平台，讓開發者能夠以低門檻學習電腦科學、程式設計與嵌入式系統開發，同時推廣創客文化與硬體創新應用[1]。

硬體架構方面如圖二所示，樹莓派包含ARM架構處理器、RAM記憶體、GPIO（General Purpose Input/Output）介面、USB連接埠、HDMI顯示輸出、網路接口及MicroSD卡插槽。其處理器性能隨代數提升，可運行完整的Linux作業系統[1]，如：Raspberry Pi OS。GPIO 介面提供40根可程控的輸入輸出腳位，可直接控制馬達、感測器、LED或其他電子元件，使其能作為嵌入式控制平台，實現感測器資料擷取、訊號處理與硬體控制等功能。使用者可以透過Python、C/C++、Java等多種程式語言進行開發，並利用豐富的開源函式庫和驅動程式來控制感測器或與外部設備通訊，也支援I2C、SPI、UART、PWM等通訊協定，方便與各類電子元件或模組整合，例如LiDAR、IMU、攝影機、馬達控制器等，廣泛應用於智慧家居、物聯網、機器人、自動化控制及人工智慧系統的快速驗證與實作。



圖二、樹莓派（Raspberry Pi 4）

在本專題中，使用的是Raspberry Pi 4扮演了核心運算與控制平台的角色，負責整合感測器資訊、控制自主移動機器人的導航與避障行為，以及處理顏色辨識模組輸出的事件資訊。將LiDAR掃描資料、RGB攝影機影像及使用者指令整合，並透過ROS節點進行資料交換與任務控制，實現自主巡檢、多點任務切換與事件觸發流程。透過高整合性與穩定性，整個系統能在室內病房環境中持續運行，支援巡檢任務、避障控制及即時決策，使其成為智慧病房巡檢自主移動機器人的核心運算平台。

此外，樹莓派的低功耗與便攜性也是本研究選擇它的重要原因。配合行動電源供應，系統可以在無需外部電源的情況下進行長時間巡檢任務，提升實際部署的靈活性與操作便利性，其開放式硬體設計也方便研究人員依需求擴充功能模組。不僅提供了足夠的運算能力與豐富的通訊接口，還具備良好的擴充性與開發友善性，使其成為自主移動機器人平台中不可或缺的核心運算與控制單元。本研究透過樹莓派進行感測器融合、導航控制、事件判斷及任務排程，驗證了其在智慧病房巡檢場景下的實用性，並為未來智慧醫療平台功能擴展提供可靠的硬體基礎。

2.2 ROS

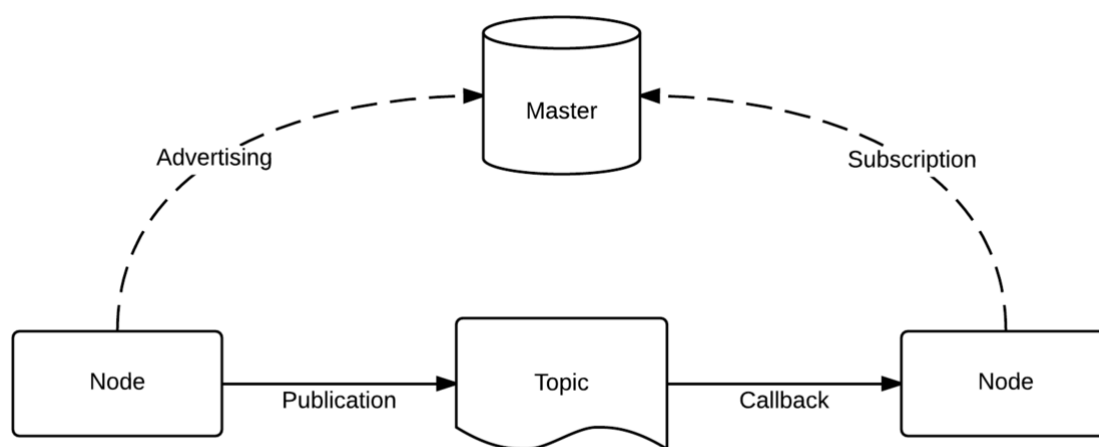
機器人作業系統是一套廣泛應用於機器人開發的開源軟體框架。ROS 是建立在Linux 等作業系統之上的機器人軟體中介層。提供硬體抽象、節點通訊、套件管理、資料記錄、視覺化工具與多種機器人演算法，使開發者能以模組化方式整合感測器、控制器與導航功能。Quigley 等人指出，ROS 的核心價值在於提供一個結構化的通訊層，使分散式機器人程式能在不同運算節點之間交換資料，進而降低大型機器人系統整合的複雜度[2]。

ROS 的基本單位為 Node，通常可將其視為執行特定功能的獨立程式。例如，在AMR 系統中，LiDAR driver node 負責讀取雷射掃描資料，SLAM node 負責根據感測資料建立地圖，navigation node 負責路徑規劃與速度命令輸出，camera node 則可負責取得影像資料並進行視覺辨識。將不同功能拆分為多個 Node 的優點，是能讓系統具有較高的模組化與可維護性；當其中一個感測器或演算法需要更換時，通常不必重寫整個系統，而是針對特定 Node 或資料介面進行調整。

ROS 中常見的資料交換方式為 Topic。Topic 可視為不同 Node 之間傳遞資料的通道，而資料傳送模式則採用 Publisher 與 Subscriber 架構。Publisher 負責將資料發布到特定

Topic，Subscriber 則訂閱該 Topic 並接收資料。以本專題為例，LiDAR driver node 會將掃描資料發布到 /scan Topic，SLAM node 或避障模組則訂閱 /scan 以取得環境距離資訊。此種通訊方式可使資料產生端與資料使用端保持低耦合，讓同一份感測資料能同時提供給建圖、定位、避障與路徑規劃等模組使用。Quigley、Gerkey 與 Smart 亦說明，ROS 透過 message-passing 機制讓機器人系統中的不同程式能有效交換感測與控制資料 [3]。

在 ROS1 架構中，Master 負責協助 Node 彼此找到對方。當某個 Node 啟動後，會向 Master 註冊自己所發布或訂閱的 Topic；另一個 Node 若需要取得特定資料，也會透過 Master 查詢對應的 Publisher。需要注意的是，Master 並不直接傳送所有資料，而是協助建立通訊關係。實際資料傳輸通常是在 Publisher 與 Subscriber 之間進行。如圖三所示，LiDAR node 先向 Master 宣告其發布資料的 Topic，SLAM node 也向 Master 註冊其訂閱需求；當 Master 完成媒合後，LiDAR node 便可透過 Topic 將掃描資料傳送給 SLAM node，使 SLAM 模組能取得環境資訊並進行建圖或定位。



圖三、ROS1 Node、Topic 與 Master 通訊架構圖

本專題採用 ROS 作為 AMR 系統整合核心，主要原因在於他能有效連接感測、建圖、定位、規劃與控制等不同功能層。LiDAR 提供 /scan 資料，Camera 提供影像資訊，Kobuki 底盤與里程計提供移動與姿態資訊，這些資料再透過 ROS Topic 與 TF 座標轉換整合至導航架構中。透過 ROS，系統可將硬體層、感知層、定位建圖層、規劃層與控制層分別模組化，使 AMR 能完成環境掃描、SLAM 建圖、自主定位、路徑規劃、避障控制與病房巡檢任務。另一項重要特性是具備豐富的工具與套件生態系。例如 RViz 可

用於顯示機器人模型、LiDAR 掃描資料、地圖、路徑與 TF 座標關係；rosviz 可記錄與回放 Topic 資料，協助開發者重現實驗過程並分析問題；navigation stack 則提供 move_base、costmap、global planner 與 local planner 等導航相關模組。Pyo 等人指出，ROS 之所以適合教育、研究與原型開發，是因為其套件化架構與開源資源能降低機器人系統開發門檻[4]。

而 ROS 系統在實作時也需要注意 Topic 命名、資料格式、座標系與時間同步等問題。若不同 Node 使用的 Topic 名稱不一致，Subscriber 便無法取得正確資料；若 TF 座標轉換錯誤，LiDAR 掃描點、機器人本體與地圖之間的位置關係也會發生偏差，進而影響建圖與避障效果。Koubaa 所編輯的 ROS 參考書中亦強調，ROS 應用開發需理解節點、訊息、Topic、Service 與工具鏈之間的關係，才能建立可擴充且穩定的機器人系統[5]。因此，本專題在系統整合時，需確認 /scan、/odom、/cmd_vel、/tf 等資料流是否正確，並透過 RViz 觀察地圖、座標系與機器人運動狀態，以確保各模組能穩定協同運作。

ROS 在本專題中扮演系統整合與資料交換的核心角色。透過 Node、Topic、Publisher、Subscriber 與 Master 的架構，AMR 的感測器、演算法與底盤控制能以模組化方式連結。LiDAR、Camera、SLAM、costmap、path planning 與 /cmd_vel 控制命令皆可在 ROS 架構下形成完整資料流，使機器人能由環境感知逐步完成定位、建圖、路徑規劃與任務執行。ROS 是本專題醫療巡檢 AMR 能夠整合多種硬體與軟體模組的關鍵基礎。

2.3 光達

光達(Light Detection and Ranging, LiDAR)是一種關鍵的光學遙感技術，廣泛應用於自動駕駛車輛與機器人領域，以實現精確的環境感知。這項技術透過發射雷射光束來測量與物體之間的距離，進而建立周圍環境的二維及三維點雲地圖，使自動化系統能夠在複雜的場景中進行導航與障礙物偵測[6]。在自動駕駛的應用中，LiDAR 經常與數據處理演算法結合，用於執行同步定位與地圖建構 (SLAM) 等高階任務，透過即時處理大量的感測數據來修正車輛的運動估計，是實現車輛智慧化不可或缺的核心感測器[7]。

LiDAR 的基本運作原理主要基於飛行時間 (Time-of-Flight)，用數學表示即為式

$$D = c \frac{\Delta T}{2} \quad (1)$$

D 為與目標之間的距離， c 為光速，而 ΔT 為光從發射端抵達接收端的時間差。由於光會來回地走，因此若要只算去程的距離，則要將來回的距離除以二。透過計算雷射從發射到接收到的時間差來推估距離，進而建立出周遭環境的點雲資料（point Cloud）[7]。

系統包含發射器與接收器等核心部件。運作時，發射器會向環境發射脈衝雷射，當光束擊中物體並反射回接收器時，系統便會根據光波往返的時間差來計算目標的精確距離。相較於其他感測技術，LiDAR 能提供高頻率且誤差相對穩定的距離測量，不會因測量距離的遠近而產生顯著的誤差變化。透過持續掃描與收集這些測距數據，系統能夠即時重建出環境的幾何結構，即使在車輛高速移動或進行六自由度運動時，也能透過演算法校正點雲的失真，確保地圖的準確性。

在硬體分類上，LiDAR 系統根據其光束轉向機制主要可分為三大類：機械旋轉式、固態轉向式以及全固態閃光（Flash）式。機械式 LiDAR 利用旋轉部件來掃描整個環境，雖然視野廣闊但包含笨重的移動零件；為了克服這些缺點，不需要旋轉部件的固態 LiDAR 應運而生，這類感測器在耐用性與體積上更具優勢。不同類型的感測器在自動化系統中扮演著重要角色，結合里程計與地圖建構演算法，LiDAR 能夠在無需依賴高精度 GPS 的情況下，實現低漂移的即時定位與導航[7]。

相較於傳統的紅外線或超音波感測器，具有高解析度、高精度與高更新率的優勢，可用於即時掃描周圍環境，生成 2D 或 3D 地圖，廣泛應用於自動駕駛車、無人機、掃地機器人等領域[7]。在本專題中，我們選用 2D 雷射掃描器作為機器人感知環境的主要感測器，並結合 ROS 套件進行 SLAM 與障礙物偵測。其所提供的環境資訊是機器人定位與導航的基礎。



圖四、RPLiDAR A1

2.4 SLAM 簡介

同時定位與地圖構建 (The Simultaneous Localization and Mapping, SLAM)。考慮一個在未知環境中建立地圖，在一邊構建地圖的情況下，定位出自身的位置以及姿態，還同時實施導航等操作。此技術常用於自動駕駛、機器人導航等領域[8,9]。其實 SLAM 問題的本質是求解以下的聯合後驗(joint posterior)分佈，式 2:

$$P(x_k, m | Z_{0:k}, U_{0:k}, x_0) \quad (2)$$

其中， x_k 為機器人當前的狀態， m 為地圖， $Z_{0:k}$ 跟 $U_{0:k}$ 分別為觀測與控制輸入的歷史數據[9]。而 SLAM 又可以分成兩種更新方式，時間更新(Time-Update)跟測量更新(Measurement Update)。前者是根據運動模型更新機器人與地圖的先驗分佈，其遞迴關係式可以表達成式 3

$$P(x_k, m | Z_{0:k}, U_{0:k}, x_0) = \int P(x_k | x_{k-1}, u_k) P(x_{k-1}, m | Z_{0:k-1}, U_{0:k-1}, x_0) dx_{k-1} \quad (3)$$

而後者是根據觀測數據修正機器人位置與地圖，計算新的後驗分佈，其遞迴關係式可以表達成式 4

$$P(x_k, m | Z_{0:k}, U_{0:k}, x_0) = \frac{P(x_k | x_{k-1}, m) P(x_{k-1}, m | Z_{0:k-1}, U_{0:k-1}, x_0)}{P(Z_k | Z_{0:k-1}, U_{0:k}, x_0)} \quad (4)$$

SLAM 的實現過程主要包括以下步驟：

(1) 特徵點提取：

特徵點是從環境中識別出的、可重複觀測的物件，例如角落或邊緣。這些地標應具備唯一性與穩定性，使機器人能從不同位置重新觀測到它們。常用方法包括極值點檢測、RANSAC(隨機採樣一致性)等。

(2) 數據關聯：

在機器人移動後，SLAM 系統需要將當前觀測到的特徵點與地圖中已有地標進行匹配，以確保重觀測到的地標能正確更新地圖。錯誤的數據關聯可能導致整個地圖的不一致性。

(3) 狀態估計：

SLAM 的狀態估計通常基於貝葉斯理論，使用濾波器(如擴展卡爾曼濾波器，EKF)或粒子濾波器(FastSLAM)來更新機器人的位置與地標位置。狀態估計考慮了機器人的運動模型與觀測模型，並動態更新不確定性。

(4) 狀態更新：

根據新獲得的觀測數據，修正機器人與地圖的狀態。這一過程基於卡爾曼增益(Kalman Gain)平衡運動模型和觀測模型的誤差，使狀態估計更加準確。

(5) 特徵點更新：

持續向地圖中添加新地標，並根據最新數據更新已有地標的位置與不確定性，使地圖逐步完善。

ROS 中提供多種開源 SLAM 實作工具，如 gmapping、Hector SLAM、Cartographer 等，支援不同硬體架構與應用情境。在本專題中，SLAM 負責協助機器人自動建圖與定位，為後續的導航與任務規劃提供基礎。

2.5 AMCL

AMCL(Adaptive Monte Carlo Location)是改良後的 MCL(Monte Carlo Location)演算法，而 MCL 是一種粒子濾波演算法，用來讓機器人定位的[10-12]。粒子濾波的核心理論基礎是式 5 [10]:

$$bel(x_t) = P(x_t | z_{1:t}, u_{1:t}) \quad (5)$$

此公式即在給定了傳感器觀測數據 $Z_{1:t}$ 和控制輸入 $u_{1:t}$ 的情況下，機器人位姿 x_t 的可能性分佈，也說明了 MCL 如何處理機器人位置以及姿態的不確定性。此模型會得到對於第 n 粒子的測量概率(measurement probability) $P(x_t | Z_{1:t}, u_{1:t})$ ，可用來隨著時間更新每個粒子的權重，也就是利用傳感器的數據來校正每個粒子的可信度[10]。

$$W_t^{[n]} = P(z_t | x_t^{[n]}) W_{t-1}^{[n]} \quad (6)$$

在式 6 中 $P(z_t | x_t^{[n]})$ 為根據觀測模型計算粒子 $x_t^{[n]}$ 與當前傳感器 z_1 測量值的匹配程度。而 $W_{t-1}^{[n]}$ 則是該粒子在前一刻的權重。將此刻所得的權重標準化之後，也就是使其變成 $\sum_{i=1}^n W_t^{[i]} = 1$ 。最後在將各個粒子的位置加權上權重[11]。

$$bel(x_t) \approx \sum_{i=1}^N W_t^{[i]} \delta(x_t - x_t^{[i]}) \quad (7)$$

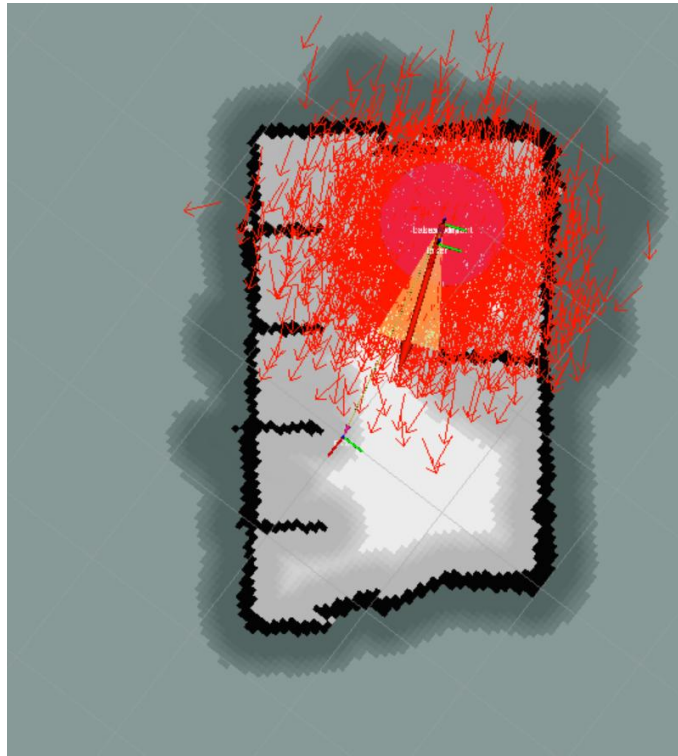
在式 7 中 δ 代表取樣評估概率(sample estimated probability)。上式對於粒子分佈的整體描述，通過權重加權的方式來逼近真實的位姿分佈[11]。以上是 MCL 基本的概念。由於計算資源的有限，因此實務上還會增加重採樣的方式(Resampling)，將某些權重極小、對於系統沒有貢獻的粒子移除，轉而看其他處的權重[11]。

而 AMCL 與 MCL 的差別，是加入了自適應調整功能。自適應調整功能可以依據下式來判定機器人是否被綁架(robot abducted/kidnapped robot)或者是否在簡單的環境，來調整取樣粒子的數量，這樣可以節省運算資源以及在發生機器人綁架的時候，添加隨機粒子到環境中，來快速更新，找到自己的新位置[11,12]。

$$\begin{cases} \omega_{\text{slow}} = \omega_{\text{slow}} + \alpha_{\text{slow}}(\omega_{\text{avg}} - \omega_{\text{slow}}) \\ \omega_{\text{fast}} = \omega_{\text{fast}} + \alpha_{\text{fast}}(\omega_{\text{avg}} - \omega_{\text{fast}}) \end{cases} \quad (8)$$

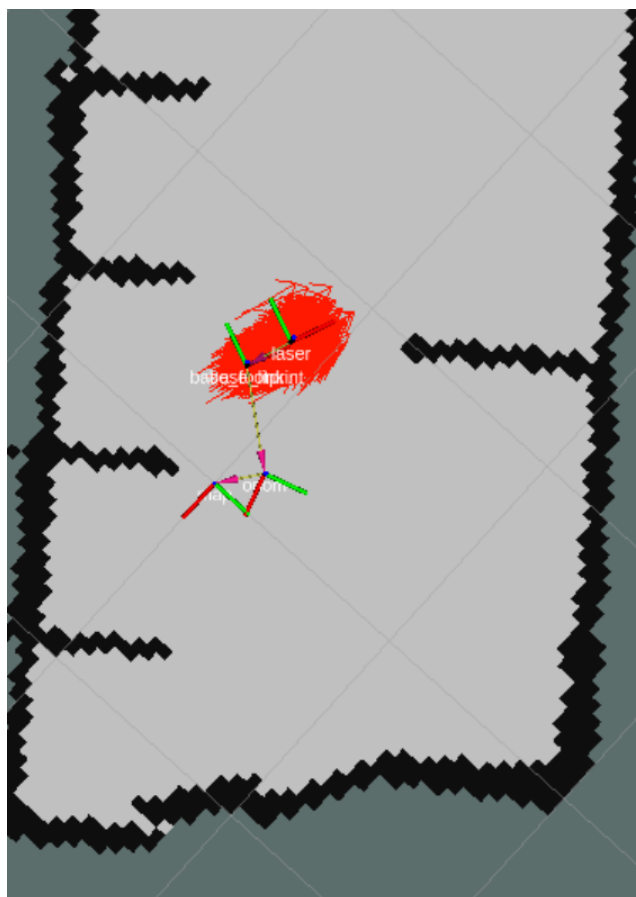
在實作上，會再加入自適應調整粒子數目的機制，如式(8)所示， ω_{avg} 表示全部粒子的平均權重，而 α_{slow} 與 α_{fast} 分別為指數濾波器的長期及短期平均衰減率。在一般情況下，會符合 $0 \leq \alpha_{slow} \leq \alpha_{fast}$ [11]。若不符合，則有可能發生機器人綁架或是其他事件。

粒子可視為機器人可能所在位置與朝向的假設，每一個粒子皆包含位置與姿態資訊。於定位初始階段，系統尚未能完全確認機器人於地圖中的精確位置，因此粒子會分布於較大的範圍內，用以表示目前定位仍存在不確定性。如圖五所示，紅色箭頭為 AMCL 粒子，箭頭位置代表機器人可能所在的位置，方向則代表可能的朝向。當粒子分布較為分散時，表示系統仍在根據 LiDAR 掃描資料、里程計資訊與既有地圖進行比對，以逐步修正機器人位姿估測。



圖五、AMCL 粒子收斂前之分布示意圖

隨著機器人持續接收 LiDAR 掃描資料並更新里程計資訊，AMCL 會根據粒子與地圖環境的匹配程度調整權重。若某些粒子的假設位置與實際感測結果較一致，該粒子會獲得較高權重；反之，與環境觀測不符的粒子權重則會降低，並可能在重採樣過程中被淘汰。經過多次權重更新與重採樣後，粒子會逐漸集中於較可能的機器人位姿附近，如圖六所示。此時紅色粒子分布範圍明顯縮小，代表系統對機器人位置的估測較為穩定，可作為後續導航、costmap 更新與路徑規劃之依據。



圖六、AMCL 粒子收斂後之分布示意圖

2.6 Dijkstra 演算法

Dijkstra 演算法用於解決單源最短路徑問題，其目標是從給定的起始節點(源點)找到到所有其他節點的最短路徑。他使用貪婪策略，在每一步中選擇當前最短的未處理路徑，直至所有節點的最短路徑均被確定[13]。

運作流程如下：

(1) 初始化：

- 將所有節點加入未處理集合(unvisited set)。
- 起始節點的距離設為 0，其餘節點的距離設為無窮大。

(2) 選擇當前最短距離的節點：

- 從未處理集合中選擇距離最小的節點作為當前節點。
- 將該節點標記為已處理。

(3) 更新與鄰居的距離：

- 若發現新連上的路徑距離小於原本標註的距離，則將原本的路徑起始點更新成新路徑的起始點，數學式顯示如式 9：

$$D(v) = \min(D(v), D(u) + c(u, v)) \quad (9)$$

$D(v)$ 為從起點到達的距離，而 $c(u, v)$ 則是從 u 到 v 得距離，這裡的等號採用程式語言中賦值的含意，代表說 $D(v)$ 的值為原本 $D(v)$ 的值與從 $D(v)$ 加上從 u 到 v 的距離取最小值。

(4) 重複步驟：

- 如此重複步驟 2 跟步驟 3，直到所有節點均被處理。

(5) 獲得結果：

- 結果便會得到從起點到達所有路徑的最短距離，也包含我們一開始的目標。

2.7 A*演算法

A*(A-star)演算法是一種廣泛應用於路徑規劃的啟發式搜尋演算法，旨在圖形結構中尋找最短路徑。該演算法結合了 Dijkstra 演算法的全面搜索能力與最佳優先搜尋法的方向性，利用啟發式函數 $h(n)$ 和評估函數 $f(n)$ 來高效地尋找目標節點。而其使用式 10 來決定每個節點的搜索優先順序[14]。

$$f(n) = g(n) + h(n) \quad (10)$$

$g(n)$ 是從起點到當前節點的實際距離；而 $h(n)$ 是從當前節點到目標節點的預估距離（啟發式函數）[14]

其運作流程如下[14]:

(1) 初始化：

- 將起點加入開放列表(open list)。
- 設置閉合列表(close list)為空。

(2) 重複以下步驟，直至找到目標或無法前進：

- 從開放列表中選擇 $f(n)$ 最小的節點作為當前節點。
- 將當前節點移至閉合列表。
- 檢查當前節點的鄰居節點：
 - i. 若鄰居是障礙物或已在閉合列表中，則忽略。
 - ii. 若鄰居不在開放列表中，加入開放列表並計算其 $f(n)$ 值。
 - iii. 若鄰居已在開放列表中，更新其 $f(n)$ 值(若從當前節點的路徑更短)。

(3) 終止條件：

- 若目標節點被加入閉合列表，則找到最短路徑。
- 若開放列表為空，則無法找到目標。

(4) 回溯路徑：

- 從目標節點回溯其父節點直至起點，生成最短路徑。

2.8 Dynamic Window Approach

Dynamic Window Approach (DWA)是一種用於移動機器人動態避障和本地路徑規劃的算法，專門針對帶有速度和加速度限制的機器人而設計。該方法基於機器人的運動動力學，通過直接在速度空間中搜索可行的控制指令，來計算最優的轉速和線速度，使機器人能夠在動態環境中安全移動[15]。

由於計算資源的有限，DWA 通過以下方法減少搜索空間。

- 圓形軌跡：假設機器人以恆定速度運動，軌跡為圓弧或直線。
- 可行速度：只考慮能夠在障礙物前停下來的速度。
- 動態窗口：進一步限制搜索空間，只考慮當前時間段內能夠達到的速度。動態窗口定義了機器人在當前時間段內能夠達到的速度範圍，而該範圍由機器人的當前速度、加速度限制和安全約束(例如：障礙物距離)決定[15]。

運作流程[15]:

(1) 初始化搜索空間：

- 在速度空間中生成所有可能的速度組合(v, ω) (線速度和角速度)。

(2) 限制搜索空間：

- 排除無法安全停下的速度(例如：可能導致碰撞的速度)。
- 限制在動態窗口內的速度。

(3) 評估目標函數：

- 為每個速度組合計算以下目標函數：

$$G(v, \omega) = \alpha \cdot heading(v, \omega) + \beta \cdot clearance(v, \omega) + \gamma \cdot velocity(v, \omega) \quad (11)$$

在式 11 中 $heading(v, \omega)$ 為機器人朝向目標的對齊程度，而 $clearance(v, \omega)$ 為機器人與障物的距離， $velocity(v, \omega)$ 機器人的前進速度。其中， α, β, γ 是權重參數，用於平衡各項指標。

(4) 選擇最佳速度：

- 動態窗口內的速度中選擇最大化目標函數 $G(v, \omega)$ 的速度組合。

(5) 執行速度控制：

- 將選定的速度指令傳遞給機器人執行。

(6) 重複計算：

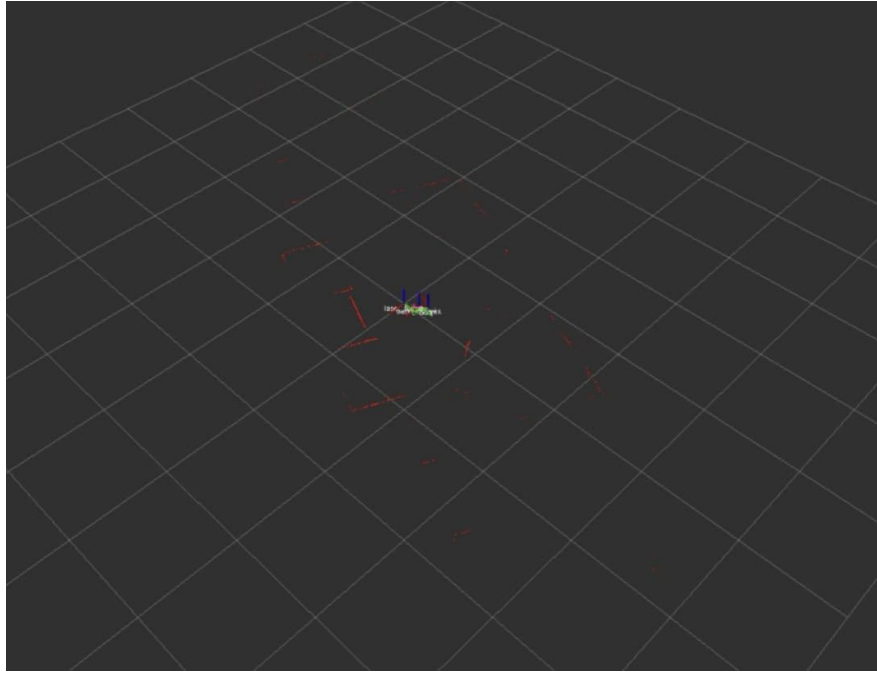
- 在下一個時間步重複以上流程，以適應環境的動態變化[15]。

2.9 點雲資料與 scan 資料流

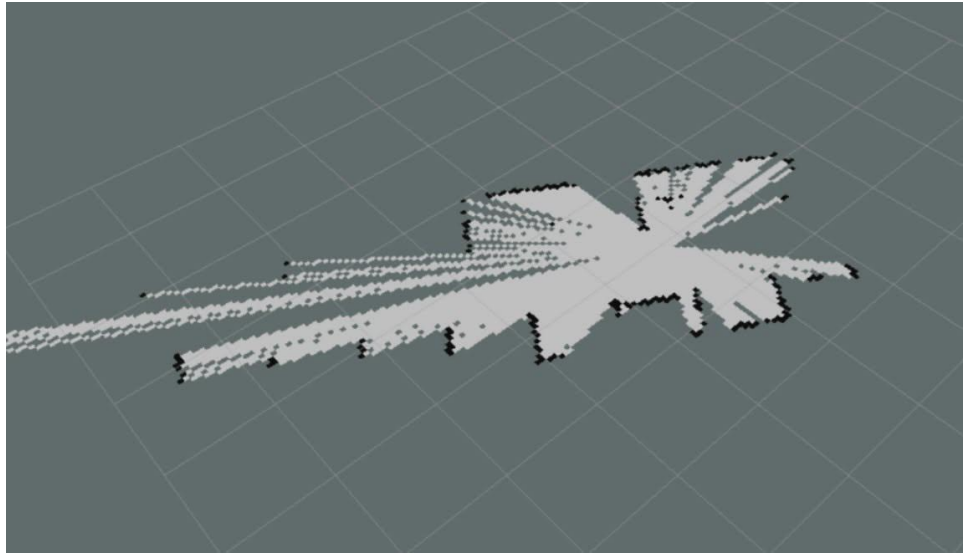
在自主移動機器人系統中，感測器資料是機器人理解環境的基礎。AMR 若要在室內空間中完成建圖、定位、導航與避障，必須先取得周圍環境的距離資訊，再將這些資訊轉換為可供演算法使用的空間資料。LiDAR 作為主要環境感測來源，其掃描結果可描述機器人周圍牆面、障礙物、走廊邊界與物體位置，並進一步提供給 SLAM、costmap 與路徑規劃模組使用。與一般影像感測器不同，LiDAR 不需要先判斷影像中的顏色或紋理，而是直接取得物體與感測器之間的距離，因此特別適合用於室內導航與障礙物偵測。

LiDAR 的基本工作方式是以前固定角度範圍進行連續掃描，並在每一個角度方向上回傳距離量測值。若將每一筆距離資料依照對應角度排列，即可得到一圈或一段範圍內的雷射掃描結果。LaserScan 為平面雷射測距儀的一次掃描資料，內容包含掃描起始角、結束角、角度間距、掃描時間、有效量測範圍、距離陣列與反射強度陣列等資訊[16]。因此，/scan 並不是單一距離值，而是一組依角度排列的距離資料，可用來描述機器人周圍一個平面上的障礙物分布。

為了確認 LiDAR 感測資料是否能正確進入 ROS 系統，本研究先於 RViz 中觀察 /scan 雷射掃描資料的顯示結果。透過 RViz 可直接檢視 LiDAR 掃描點在空間中的分布情形，並進一步觀察掃描資料經由 SLAM 處理後所建立出的環境地圖。圖七顯示本系統於測試環境中取得之 LiDAR 點雲掃描畫面與 RViz 地圖建構結果。由圖可觀察，LiDAR 掃描資料會以離散點的形式分布於機器人周圍，點對應到環境中的牆面、物體或障礙物邊界。圖八則顯示系統根據連續掃描資料逐步建立出的地圖，其中白色區域代表可通行空間，黑色區域代表牆面或障礙物邊界。此結果表示 LiDAR 資料可作為後續 SLAM、定位與導航避障模組的基礎輸入。



圖七、LiDAR 點雲掃描畫面



圖八、RViz 地圖建構畫面

從資料結構來看，`/scan` 中最重要的欄位是 `ranges`、`angle_min`、`angle_max` 與 `angle_increment`。其中，`ranges` 儲存每一個方向上的距離量測值，`angle_min` 表示掃描開始角度，`angle_max` 表示掃描結束角度，而 `angle_increment` 表示相鄰兩筆距離資料之間的角度差。透過這些資料，可以將每一筆雷射距離轉換為感測器座標系下的空間點。若第 i 筆距離資料為 r_i ，其對應角度可表示為式 12：

$$\theta_i = angle_{min} + i * angle_{increment} \quad (12)$$

則該點在 LiDAR 座標系下的平面位置可表示為式 13：

$$x_i = r_i \cos \theta_i, y_i = r_i \sin \theta_i \quad (13)$$

藉由上述轉換，原本以極座標形式儲存的雷射掃描資料，便可轉換為二維平面上的點集合。這些點集合在概念上可視為點雲資料，也就是由大量空間點所組成的環境描述。其特色在於，不需要先建立完整模型，而是直接用許多離散點呈現物體外形、牆面輪廓與障礙物位置。點雲處理可支援濾波、特徵估測、表面重建、配準、模型擬合與分割等常見三維感知任務，因此點雲已成為機器人感知與環境理解的重要資料形式[17]。

在本專題中，雖然使用的是 2D LiDAR，而非完整 3D LiDAR，但其資料概念仍與點雲相近。每一次掃描都會形成一組位於同一水平平面上的距離點，這些點可表示機器人周圍牆面、桌腳、箱子或行人等障礙物的相對位置。若機器人持續移動，系統便可在不同時間與不同位置取得多次掃描結果，並配合里程計與座標轉換資訊，逐步累積出環境結構。這也是 LiDAR 能夠支援 SLAM 的原因之一：機器人不只是在單一時間點看到障礙物，而是能將連續掃描資料與自身位置估測結合，逐漸建立出可供導航使用的地圖。/scan 資料流在 ROS 架構中扮演感測層與演算法層之間的橋樑。LiDAR driver node 會負責讀取實體 LiDAR 回傳的原始資料，並將其轉換成 ROS 可辨識的 LaserScan 訊息後發布到 /scan Topic。接著，SLAM node、避障 node 或 costmap 相關模組會訂閱 /scan，取得周圍距離資訊。由於 ROS 採用 Publisher 與 Subscriber 的通訊方式，不同功能模組不需要直接彼此控制，而是透過 Topic 交換資料[18]。

在導航與避障過程中，/scan 也會被用於建立 local costmap。Costmap 會根據 LiDAR 掃描結果，將機器人周圍的障礙物標記到代價地圖中。若某一方向的距離值小於安全距離，系統便可判斷該方向存在障礙物，並提高該區域的通行代價。對 AMR 而言，這項機制可以避免路徑規劃器選擇過於靠近牆面或障礙物的路線。當環境中出現臨時物品、移動中的人員或推車時，LiDAR 會即時更新掃描資料，使 local costmap 能反映目前環境狀態，進而讓機器人修正移動方向或重新規劃局部路徑。

以本次醫療巡檢為例，/scan 資料流可對應到病房與走廊環境中的安全移動需求。AMR 從起始點出發後，LiDAR 會持續掃描前方與側邊環境，取得走廊寬度、牆面位置與障礙物距離。若系統判斷前方空間可通行，導航模組便可依照全域路徑繼續前進；

若 `/scan` 顯示前方距離低於設定安全閾值，避障邏輯則會降低線速度、調整角速度，或切換至旋轉搜尋模式。這種資料流設計使 AMR 不只是依靠預先建立的地圖移動，也能根據即時感測結果應對臨時環境變化。

此外，`/scan` 資料的品質會直接影響系統穩定性。若 LiDAR 安裝位置過低，可能掃描到底盤本身或地面雜訊；若安裝位置過高，則可能無法偵測較低矮的障礙物。若量測值中出現超出有效範圍的資料，也可能造成 `costmap` 誤判。因此，在實作時需注意 LiDAR 的安裝高度、水平角度、掃描範圍、資料更新頻率與座標轉換是否正確。透過這樣的資料流設計，AMR 可將原始距離量測轉換為可用於建圖、定位與路徑修正的環境資訊。對醫療巡檢任務而言，這不僅提升機器人的自主移動能力，也使系統具備在病房、走廊與護理站等室內環境中安全運行的基礎。

2.10 Costmap 與路徑規劃概念

在自主移動機器人導航系統中，路徑規劃的目標並不只是找出起點到終點的幾何最短路線，而是要在環境限制、安全距離與障礙物分布之下，規劃出可實際通行且碰撞風險較低的路徑。對於醫療巡檢 AMR 而言，機器人可能需要在走廊、病房門口、護理站與充電區之間移動，環境中也可能出現病床、推車、輪椅或行人等臨時障礙物。因此，導航系統必須將地圖與感測器資訊轉換為可供演算法判斷的形式，而 `costmap` 便是 ROS 導航架構中常用的環境表示方法。

`Costmap` 可翻譯為代價地圖，其概念是將機器人周圍或整體環境劃分為許多小格子，每一格依據是否可通行、是否有障礙物、距離障礙物多近等條件，被設定為不同的代價值。代價越高，代表該區域越不適合通行；代價越低，代表該區域相對安全。Lu、Hershberger 與 Smart 提出的 `layered costmaps` 概念中，將不同來源的環境資訊分成多個圖層處理，例如靜態地圖層、障礙物層與膨脹層，再整合成主 `costmap` 供路徑規劃器使用，使導航系統能更清楚地區分不同類型的環境限制[19]。

在 ROS 導航系統中，通常可分為 `global costmap` 與 `local costmap`。`global costmap` 主要根據已建立的地圖，提供全域路徑規劃使用，負責決定機器人從目前位置到目標點的大方向；`local costmap` 則依據 LiDAR 或其他感測器的即時資料，反映機器人附近短距離範圍內的障礙物狀態，供局部避障與速度控制使用。ROS navigation stack 的設計中，導航模組會整合里程計、感測器資料與目標位置，最後輸出可送至移動底盤的安

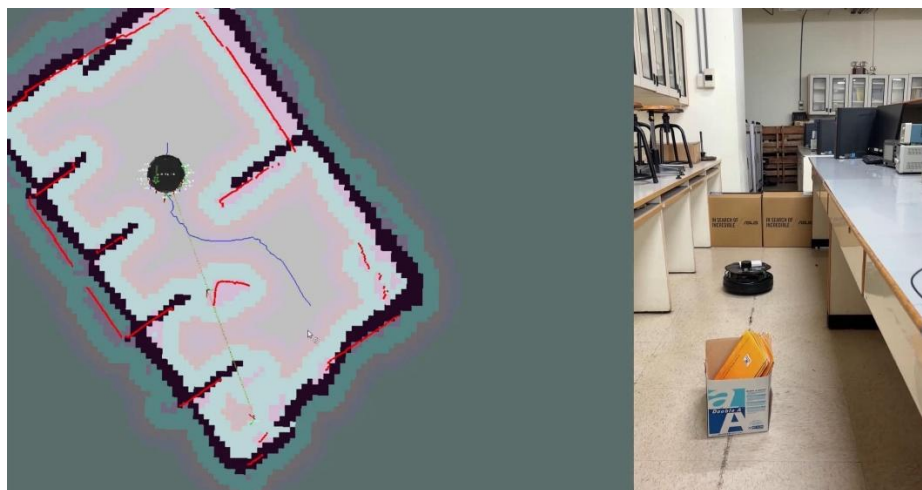
全速度命令。因此，global costmap 與 local costmap 的功能並不相同，前者偏向「規劃整體路線」，後者偏向「處理當下環境」。

Costmap 的常見圖層包含 static layer、obstacle layer 與 inflation layer。static layer 主要來自 SLAM 建立完成的靜態地圖，用來表示牆面、固定隔間或長期存在的不可通行區域。obstacle layer 則會接收 LiDAR 的/scan 資料，將即時偵測到的障礙物標記在地圖中。inflation layer 則會在障礙物周圍產生安全緩衝區，也就是將障礙物附近一定範圍內的格子提高代價，使機器人不會規劃出過於貼近牆面或物體的路線。Zheng 的 ROS Navigation Tuning Guide 指出，costmap 參數對局部規劃器的成功率相當重要，且 ROS 中的 costmap 通常由 static map layer、obstacle map layer 與 inflation layer 組成[20]。

路徑規劃部分可分為全域規劃與局部規劃。全域規劃器會根據 global costmap 計算從起點到目標點的路徑，例如從起始點前往病房 A、再前往護理站或返回充電區。這一層通常處理的是整體方向與長距離路徑選擇。局部規劃器則依據 local costmap、機器人當前速度、障礙物距離與底盤運動限制，產生短時間內可執行的速度命令，例如前進、減速、轉向或暫停。ROS move_base 架構中，會結合 global planner、local planner 與 costmap，以完成移動機器人的導航任務。

在醫療巡檢 AMR 中，Costmap 可用於連接感測資料與路徑決策。當 LiDAR 偵測到前方或側邊存在障礙物時，obstacle layer 會更新障礙物位置；inflation layer 進一步建立安全距離，避免機器人貼近病床、牆面或推車。接著，local planner 會依據更新後的 local costmap 修正移動方向，使 AMR 在維持目標路徑的同時，能對臨時障礙物做出避讓。若原本的路線長時間被阻擋，系統則可重新呼叫全域規劃，從目前位置重新計算通往目標病房或護理站的路徑。

由圖九可觀察到，AMR 在執行導航任務時，RViz 會即時顯示機器人於地圖中的位置、周圍 costmap 變化與規劃路徑。當前方環境出現障礙物時，系統可依據 LiDAR 感測資料更新 local costmap，並由局部規劃器修正移動方向，使機器人能在維持任務目標的同時完成避障。



圖九、實際運行 RViz 圖

整體而言，Costmap 與路徑規劃在本專題中扮演環境判斷與導航決策的核心角色。SLAM 提供靜態地圖，LiDAR 提供即時障礙物資訊，Costmap 則將這些資料轉換成不同代價的可通行區域。透過 global planner 與 local planner 的配合，AMR 不僅能依照任務目標前往指定病房，也能在遇到走廊障礙物或人員移動時進行局部避障。此架構使醫療巡檢機器人具備較完整的自主導航能力，也提高其在室內場域中運行的安全性與穩定性。

2.11 RGB+HSV 六通道 CNN 顏色辨識

顏色辨識其核心目的在於根據影像中的色彩資訊，判斷目標物所屬的顏色類別。數位影像通常由不同通道組成，每一個像素會以數值方式記錄顏色資訊。常見的 RGB 色彩空間由紅色、綠色與藍色三個通道組成，透過三種基礎光強度的組合形成不同顏色。由於 RGB 色彩空間與相機感測器及螢幕顯示方式相近，因此是影像辨識中最常使用的影像表示方式之一。然而，RGB 色彩空間的數值會同時受到顏色本身與環境亮度影響。當光線強弱、陰影或拍攝角度改變時，即使是相同顏色的物體，也可能產生不同的 RGB 數值。因此，若僅以 RGB 作為顏色辨識依據，模型可能會受到光照條件影響，造成辨識穩定性下降。

HSV 色彩空間則是另一種常用的顏色表示方式，其名稱分別代表色相 Hue、飽和度 Saturation 與明度 Value。色相用來描述顏色種類，例如紅色、黃色、綠色與藍色；飽和度代表顏色的純度或鮮豔程度；明度則表示顏色的亮暗程度。相較於 RGB 將色彩與亮度混合表示，HSV 較接近人眼對顏色的感知方式，能將顏色種類與亮度資訊分開

描述，因此常被應用於顏色篩選與影像分析中[21]。OpenCV 文件中也提供 RGB 與 HSV 之間的色彩空間轉換方式，顯示 HSV 在電腦視覺領域中具有常見的應用基礎[22]。

在顏色辨識任務中，若將 RGB 與 HSV 兩種色彩空間結合，可使影像特徵更加完整。RGB 可保留原始影像中的三原色強度資訊，而 HSV 則補充色相、飽和度與明度等色彩特徵。當兩者結合後，影像不再只有原本的 R、G、B 三個通道，而是形成 R、G、B、H、S、V 六個通道。六通道影像的意義並不是增加影像張數，而是讓同一張影像在不同色彩空間下提供更多特徵，使後續模型能從不同角度理解顏色差異。

卷積神經網路 CNN 是一種常用於影像辨識的深度學習模型。CNN 主要透過卷積層擷取影像中的局部特徵，例如邊緣、紋理、形狀與顏色分布；再透過池化層降低資料維度，保留較重要的特徵；最後由分類層輸出辨識結果。深度學習模型能夠透過多層結構自動學習資料特徵，因此在影像辨識、物件辨識與分類任務中具有良好的應用效果[23]。

RGB+HSV 六通道 CNN 的理論基礎，是將兩種色彩空間的特徵共同作為模型學習的依據。相較於單純使用 RGB 三通道影像，六通道輸入可提供更豐富的顏色描述，使模型同時學習原始色彩強度與接近人眼感知的色彩資訊。對顏色辨識而言，此方法有助於改善單一色彩空間資訊不足的問題，並提升模型對不同顏色類別的區分能力。因此，可視為一種結合傳統色彩空間轉換與深度學習分類能力的方法。其理論重點不在於單純增加資料量，而是透過多通道特徵融合，使模型在進行顏色辨識時能取得更完整的色彩資訊，進而提高辨識穩定性與分類準確度。

本系統之影像感知資料主要由 RGB Camera 擷取，如圖十所示。用於取得機器人行進方向之影像資訊，後續可依任務需求進行 ROI 擷取、影像前處理與模型辨識。與 LiDAR 主要提供距離資訊不同，Camera 可取得畫面中的顏色、紋理與物件外觀特徵，因此適合用於顏色辨識、口罩辨識、物件辨識，以及低矮障礙物輔助判斷等視覺感知任務。



圖十、RGB Camera

在本專題中，Camera 並非僅作為一般影像顯示裝置，而是 Sensor Fusion 架構中的視覺感知來源。LiDAR 負責提供環境距離、建圖與主要導航資訊；Camera 則補充 LiDAR 無法取得的影像語意資訊。於顏色辨識任務中，Camera 擷取之 RGB 影像會先依目標區域進行 ROI 擷取，再將影像轉換至 HSV 色彩空間，最後與原始 RGB 通道結合形成六通道資料，供 CNN 模型進行分類。

2.12 TF Tree (Transform tree)與座標轉換原理

在自主移動機器人系統中，不同感測器與運動模組通常各自具有獨立座標系。例如：LiDAR 量測到的障礙物位置是以雷射感測器本身為基準，機器人底盤移動資訊通常以 `base_link` 或 `base_footprint` 為基準，里程計資料則常對應到 `odom` 座標系，而地圖與定位結果則會使用 `map` 座標系。若沒有統一的座標轉換機制，系統即使取得 LiDAR 掃描資料、里程計資訊與地圖資料，也無法正確判斷障礙物相對於機器人本體或地圖的位置。因此，TF Tree 是 ROS 系統中整合感測資料、定位資訊與導航模組的重要基礎。

TF 是 ROS 中用於管理座標轉換關係的工具，其主要功能是記錄不同座標系之間的位置與姿態關係，並讓系統能在指定時間查詢任兩個座標系之間的轉換。Foote 指出，tf library 的設計目的，是提供一種標準方式追蹤整個機器人系統中的 coordinate frames 與 transform data，使不同模組能以一致方式理解空間資訊[24]。ROS 官方文件也說明，tf2 可追蹤多個座標系隨時間變化的關係，並以樹狀結構維護座標框架之間的轉換，使 points、vectors 等資料能在不同座標系之間轉換[25]。因此，TF Tree 不只是 RViz 中用來觀看座標關係的圖，而是讓機器人能正確整合空間資訊的核心機制。它是由多個座

標系與轉換關係組成的樹狀結構。每一個座標系稱為 frame，每一條連線代表父座標系到子座標系的轉換關係。移動機器人常見的座標架構包含 map、odom、base_link 與感測器座標系。REP 105 對移動平台座標系提出慣例，指出 map frame 是 odom 的父座標系，而 odom 是 base_link 的父座標系；同時，每一個 frame 在樹狀架構中只能有一個父節點[26]。這樣的設計可避免座標關係混亂，使定位、里程計與機器人本體之間維持明確的轉換關係。

座標轉換主要包含平移與旋轉兩部分。平移表示兩個座標系原點之間的位置差，旋轉則表示兩個座標系方向之間的姿態差。在二維 AMR 導航中，常見轉換包含 x、y 方向位移與 yaw 角；若考慮完整三維空間，則需包含 x、y、z 位移以及 roll、pitch、yaw 旋轉。當 LiDAR 偵測到某一障礙物時，該障礙物最初的位置是以 laser_frame 表示；若要讓機器人判斷障礙物在本體前方、側邊或地圖中的位置，就必須透過 TF 將資料轉換至 base_link、odom 或 map 座標系。TF Tree 負責將不同感測器所看到的局部資訊轉換成導航系統可共同使用的空間資訊。

在 AMR 系統中，TF Tree 對 SLAM、AMCL、Costmap 與路徑規劃皆具有直接影響。SLAM 建圖時，系統需要知道 LiDAR 相對於機器人底盤的位置，才能將每一次雷射掃描資料放入正確地圖位置。AMCL 定位時，系統需要結合地圖、里程計與感測資料估測機器人在 map 中的位置。Costmap 建立時，系統也必須將障礙物資料轉換到全域或局部代價地圖中，才能判斷哪些區域不可通行。若 TF Tree 中的座標關係錯誤，例如：laser_frame 與 base_link 的安裝位置設定不正確，機器人可能會把障礙物標記在錯誤方向，導致路徑規劃偏移或避障判斷錯誤。此外，圖！！中各轉換關係旁標示的 broadcaster、average rate 與 buffer length，也可用來檢查 TF 是否穩定發布。Broadcaster 代表發布該座標轉換的節點，例如 /slam_gmapping 可發布 map → odom 的轉換，底盤相關節點可發布 odom → base_footprint，而 LiDAR 安裝關係則可由固定轉換發布節點提供。Average rate 表示轉換資料的平均發布頻率，buffer length 則表示 TF 資料在系統中保留的時間長度。這些資訊可協助判斷 TF 是否即時、是否中斷，以及不同模組之間的座標資料是否能被正確查詢。

TF Tree 與座標轉換也是 AMR 能夠穩定整合 LiDAR、底盤、地圖與導航模組的關鍵。ROS Topic 負責傳遞資料，而 TF Tree 則負責說明這些資料在空間中的相對位置。透過正確建立 map、odom、base_footprint、base_link 與 laser_frame 之間的轉換關係，

AMR 才能將 LiDAR 掃描、里程計移動、SLAM 地圖與避障資訊整合為一致的環境判斷結果。對醫療巡檢任務而言，穩定的 TF Tree 可降低定位偏差與避障誤判，使機器人能更安全地在走廊、病房與護理站之間移動。

2.13 TurtleBot

TurtleBot 是一款基於 ROS 開發的開源移動機器人平台，由 Willow Garage 與 Clearpath Robotics 等公司推廣，廣泛應用於教育、研究與原型開發領域。TurtleBot 設計簡潔，具備模組化結構，便於快速整合各類感測器與控制器[!]，圖十一為 TurtleBot2 實體。



圖十一、AMR 載體 (TurtleBot2)

其硬體組成包括：

(1) 底盤驅動模組：例如 Kobuki，提供輪式移動能力，如圖十二所示。



圖十二、Kobuki 移動底盤實體照片

- (2) 感測元件：如 LiDAR、深度攝影機(Realsense)、IMU 等。
- (3) 運算模組：可搭載樹莓派、Jetson 或筆電作為 ROS 主控機。

TurtleBot 最大特色在於與 ROS 深度整合，提供官方支援的驅動、模擬環境(Gazebo)、SLAM 與導航套件等。本專題基於 TurtleBot 平台進行實作，透過其標準化的機構與軟體，降低開發難度並提升模組重用性，有效加速自走車平台的開發與測試流程[!]

第三章、實作或模擬過程

3.1 系統開發流程與功能配置

AMR 智慧導航系統之實作與模擬過程，內容涵蓋硬體平台建置、感測器資料擷取、ROS 通訊整合、SLAM 建圖、定位、路徑規劃、避障控制、Camera 輔助感知與巡檢任務流程設計。整體開發並非單一功能的堆疊，而是依照模擬驗證、硬體整合、感測資料確認、導航建置、感知功能加入、任務流程整合的順序逐步完成，使各模組能在相同 ROS 架構下協同運作。

系統核心以 ROS 作為軟體整合平台，搭配 TurtleBot2、Raspberry Pi 4、RPLiDAR A1 與 camera 建立 AMR 實作平台。開發流程初期先於模擬環境中建立機器人模型與室內場景，確認 ROS Topic、TF Tree、LiDAR 掃描資料與導航節點之間的連接關係。模擬階段可降低直接操作實體平台時的除錯負擔，也能先行驗證 SLAM 建圖、AMCL 定位、Costmap 與路徑規劃等導航流程。待模擬架構穩定後，再將相同邏輯移轉至實體 TurtleBot2 平台，確認實體感測器、底盤控制與 ROS 節點能正常配合。

在感測與導航配置方面，LiDAR 所產生之 /scan 資料提供給 SLAM、Costmap 與避障演算法使用，底盤里程計則提供 /odom 作為機器人移動狀態回饋。導航模組依據地圖、定位結果與障礙物資訊產生移動路徑，最後透過 /cmd_vel 將速度命令傳送至 TurtleBot2 底盤，使路徑規劃結果轉換為實際移動行為。此流程形成由感測輸入、資料處理、定位建圖、路徑規劃到控制輸出的完整導航鏈。

Camera 感知功能則作為導航系統的補強與應用擴充，除輔助避障外，亦支援顏色辨識等語意感知功能。透過 ROI 分區方式，可同時支援移動安全與巡檢任務判斷，使影像資料不僅作為畫面顯示，也能轉換為任務決策所需的資訊。整體流程以 LiDAR 導航為基礎，加入 Camera 輔助感知與影像辨識功能，形成具備感測融合概念的 AMR 智慧導航架構。

3.1.1 硬體平台建置

硬體平台以 TurtleBot2 作為自主移動載體，並以 Raspberry Pi 4 作為主要運算與控制核心，負責執行 ROS 節點、感測器資料接收與底盤控制指令傳遞。整體架構包含

行動電源、Raspberry Pi 4、TurtleBot2、RPLiDAR A1 及 Camera 等模組，如圖十三所示。各硬體元件透過 USB 連接與供電配置進行整合，使平台能同時具備移動控制、環境掃描與影像感知能力。

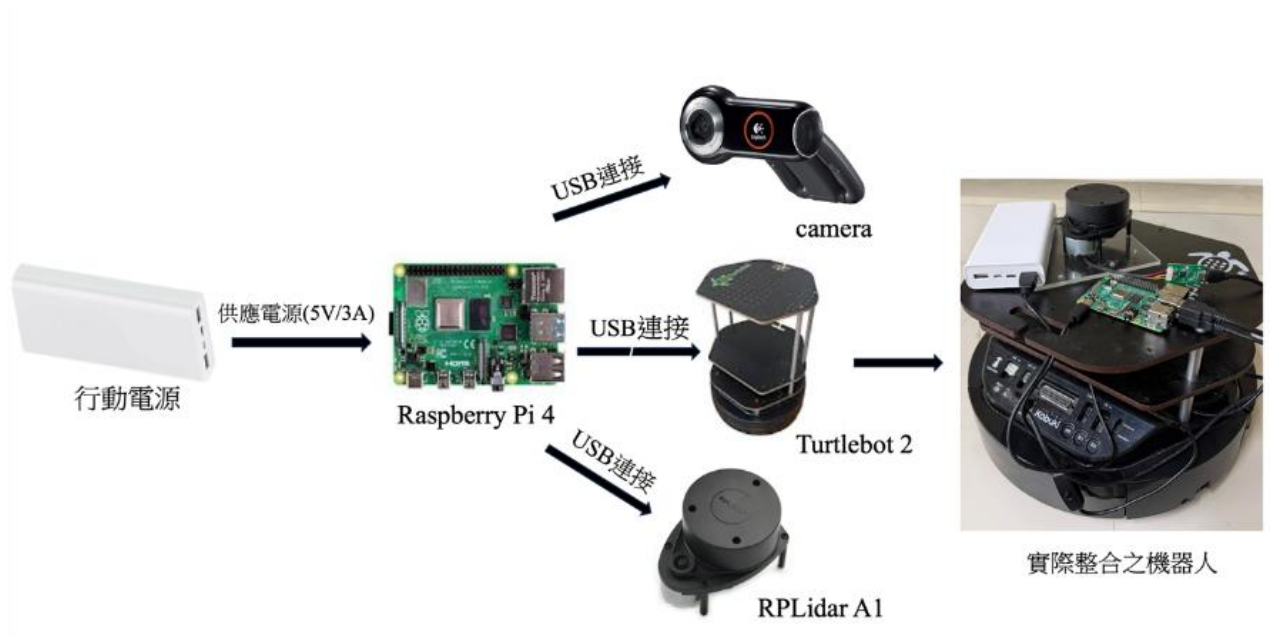
電源部分採用行動電源供應 Raspberry Pi 4 運作所需之 5V/3A 電力。由於 Raspberry Pi 4 同時需連接 LiDAR、Camera 與 TurtleBot2 底盤，穩定供電會直接影響感測資料擷取與 ROS 節點執行狀態。若供電不足，可能導致 USB 裝置斷線、節點異常關閉或資料更新不穩，因此硬體建置時需先確認電源輸出規格與實際負載是否符合系統需求。

Raspberry Pi 4 為各感測器與移動底盤之間的資料整合中心。RPLiDAR A1 透過 USB 連接至 Raspberry Pi 4，負責取得周圍環境距離資訊，後續再由 LiDAR driver 將掃描資料發布至 ROS Topic，提供 SLAM 建圖、障礙物判斷與路徑規劃使用。TurtleBot2 同樣透過 USB 與 Raspberry Pi 4 連接，使控制端能將速度命令傳送至 Kobuki 底盤，進而控制機器人前進、轉向或停止。

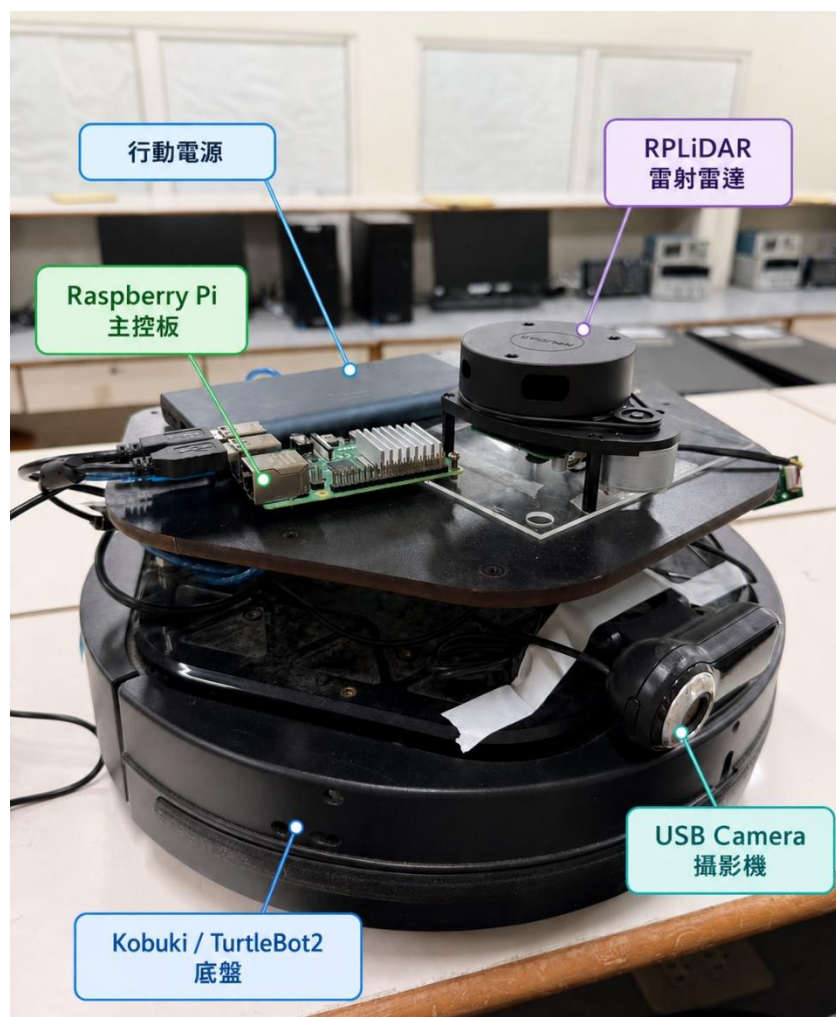
Camera 則安裝於平台前方，作為影像感知來源。提供後續視覺任務所需影像資料，依照不同任務需求，系統可擷取不同 ROI 區域進行處理；例如路面低矮障礙物判斷可使用畫面下半部區域，顏色或物件辨識則可依目標位置擷取指定區域。此配置使 camera 成為影像感知模組的資料來源，而非只服務於單一辨識功能。

TurtleBot2 採用 Kobuki 底盤作為移動機構，具備差動驅動能力，可依據 ROS 發布之速度命令執行直線移動與旋轉動作。Raspberry Pi 4 接收導航或避障模組產生的控制命令後，透過底盤通訊介面將線速度與角速度傳送至 TurtleBot2，使軟體層的路徑規劃結果能轉換為實際移動行為。此架構可將上層演算法與底盤控制分離，提升後續維護與功能擴充的彈性。

實際組裝時，Raspberry Pi 4、RPLiDAR A1、Camera 與行動電源皆固定於 TurtleBot2 上方平台，以減少移動過程中的線材晃動與感測器位置偏移。LiDAR 安裝位置需保持水平，以確保掃描平面穩定；Camera 則需朝向機器人前方，使其能取得行進方向之路面與巡檢目標影像。整合完成後，AMR 平台可同時支援 LiDAR 掃描、Camera 影像擷取、底盤控制與 ROS 資料通訊，作為後續 SLAM 建圖、自主導航、避障控制與醫療巡檢任務的硬體基礎。



圖十三、系統硬體架構圖



圖十四、實體系統架構圖

3.1.2 感測器資料擷取

感測器資料擷取階段的重點，在於確認各感測來源能否被系統正確讀取，並轉換為後續模組可使用的資料格式。由於 AMR 的導航、建圖、避障與影像辨識皆需依賴感測資料，因此在進入完整任務流程前，需先確認模擬環境與實體平台中的資料來源是否一致，避免後續整合時因資料格式、Topic 名稱或座標關係不一致而造成系統錯誤。

初期先於模擬環境中進行資料擷取練習。透過 Gazebo 建立室內場景與機器人模型後，系統可先取得模擬 LiDAR 掃描資料，並將掃描結果發布至 ROS Topic。此階段的目的並非分析導航成果，而是確認模擬感測器能產生與實體 LiDAR 類似的資料流，使後續 SLAM、Costmap 與避障模組能在實體操作前完成基本流程驗證。

在 LiDAR 資料部分，系統主要接收雷射掃描資訊，並以 /scan 作為後續模組的輸入資料。該資料提供機器人周圍障礙物距離與方向資訊，可供 SLAM 建圖、障礙物標記與避障判斷使用。實作時需先確認 /scan 是否能持續發布，再觀察掃描方向、資料更新頻率與環境輪廓是否合理。若掃描資料無法正確對應環境，後續建圖與路徑規劃即可能出現偏差。完成模擬資料確認後，再將相同的資料擷取流程移至實體平台。實體 LiDAR 啟動後，同樣需確認 /scan Topic 是否正常輸出，並檢查感測資料是否能反映周圍牆面、桌椅或其他障礙物的位置。此階段可將模擬環境中建立的檢查方式套用至實體操作，使系統由虛擬測試逐步過渡至真實環境，降低直接進行實體整合時的除錯負擔。

Camera 資料擷取則分為低矮障礙物輔助判斷與影像辨識兩項用途，依照任務需求選取不同 ROI 區域。感測資料擷取完成後，LiDAR 與 camera 分別提供不同型態的環境資訊。前者以距離資料描述周圍障礙物分布，後者則以影像資料補充近地面狀態與目標外觀資訊。兩者在資料型態與應用方向上不同，但可共同構成 AMR 的感知輸入來源。因此，此階段的工作重點並非分析最終測試結果，而是建立穩定的資料輸入流程。當模擬與實體環境中的 /scan、影像資料與相關 Topic 均能正常取得後，後續才能進一步進行 ROS 通訊整合、TF Tree 設定、SLAM 建圖、路徑規劃與巡檢任務流程設計。

3.2 ROS 通訊與座標整合

由於 AMR 同時整合 LiDAR、Camera、TurtleBot2 底盤、SLAM、導航與避障等多項功能，若各模組之間無法正確交換資料，或感測器座標與機器人本體座標未能對應，後續建圖、定位與路徑規劃皆可能受到影響。因此，ROS 通訊與座標整合可視為感測層與導航控制層之間的關鍵連接。

ROS 系統採用模組化設計，不同功能通常以獨立 Node 執行，並透過 Topic 傳遞資料。LiDAR 需將掃描資料發布至 /scan，底盤里程計需提供 /odom，導航或避障模組則會依據規劃結果發布 /cmd_vel 控制命令。camera 影像資料亦需透過對應的影像 Topic 傳入影像處理流程。透過此資料交換方式，各模組不需直接耦合，而是依照統一的資料介面進行連接，使系統具有較高的擴充性。除了 Topic 資料流之外，座標轉換也是系統整合過程中的重要環節。LiDAR 掃描資料以 laser_frame 為基準，camera 影像以相機座標或影像平面為基準，底盤運動則與 base_link、base_footprint 等機器人本體座標有關；地圖與定位結果則分別對應至 map 與 odom 座標系。若這些座標系之間缺乏正確轉換，系統即使能接收到感測資料，也無法準確判斷障礙物與機器人之間的相對位置。

在進入 SLAM 建圖、AMCL 定位與 costmap 設定前，需先確認 ROS Topic 是否正常發布與訂閱，並建立完整的 TF Tree。Topic 資料流負責處理資料如何傳遞，TF Tree 則負責處理資料位於哪一個座標系。兩者共同構成 AMR 系統的資料基礎，使 LiDAR 掃描、camera 影像、底盤移動與地圖資訊能在相同架構下協同運作。透過通訊架構與座標關係的整理，可確保後續模擬建圖、實體導航、低矮障礙物感知與病房巡檢任務能建立在一致且可追蹤的資料流程上。

3.2.1 Topic 資料流設計

在感測資料能穩定取得後，下一步為整理各模組於 ROS 系統中的資料傳遞方式。此階段的重點不在於重新說明各感測器功能，而是確認資料由哪一個節點產生、透過哪一個 Topic 傳送，以及後續由哪些模組接收使用。透過明確的 Topic 資料流設計，可降低節點之間直接耦合的情形，使導航、避障、影像感知與底盤控制能在一致的資料介面下運作。

資料流設計可分為感測輸入、狀態回饋、座標資訊、任務判斷與控制輸出等類別。感測輸入主要包含 LiDAR 掃描資料與 camera 影像資料；狀態回饋則包含底盤里程計與機器人移動狀態；座標資訊由 TF 相關資料維持各 frame 之間的關係；任務判斷則

接收影像辨識或障礙物判斷結果；控制輸出最後以速度命令形式送至底盤。透過分類整理，可使資料來源與資料用途更清楚，避免所有模組混雜在同一流程中。

LiDAR 掃描資料以 `/scan` 作為主要 Topic。此資料由 LiDAR driver 節點發布，後續可供避障演算法使用。為確保資料流穩定，整合時需確認 `/scan` 是否符合後續模組需求，以及 `frame_id` 是否能與 TF Tree 中的 `laser_frame` 對應。底盤狀態資料則以 `/odom` 提供機器人移動資訊。里程計資料可供定位、建圖與座標轉換使用，並與 `odom frame` 產生對應關係。若 `/odom` 更新不穩，導航系統可能無法正確追蹤機器人短時間內的位移變化。因此，在 Topic 設計階段，需確認底盤節點發布的里程計資料能與 TF Tree 及定位流程銜接，而不是只確認底盤能否移動。

控制命令以 `/cmd_vel` 作為底盤輸入 Topic。路徑規劃、避障演算法或手動控制皆可能產生速度命令，因此需避免多個節點同時對 `/cmd_vel` 發布互相衝突的指令。實作時應明確定義控制來源，例如在導航測試時由 `planner` 輸出速度命令，在避障測試時由避障節點接管控制。若不同控制模式未進行切換管理，可能造成機器人行為不一致，影響後續測試判斷。

Camera 影像資料在 Topic 設計中被視為影像輸入來源。影像節點負責發布相機畫面，後續由不同影像處理流程依需求擷取 ROI 或進行辨識。影像資料需先以穩定 Topic 形式輸入系統，後續模組才能依任務需求取得相同來源的影像資料。如此可使 camera 在不同應用之間共用資料來源，而不需為每一項功能重新建立獨立輸入流程。

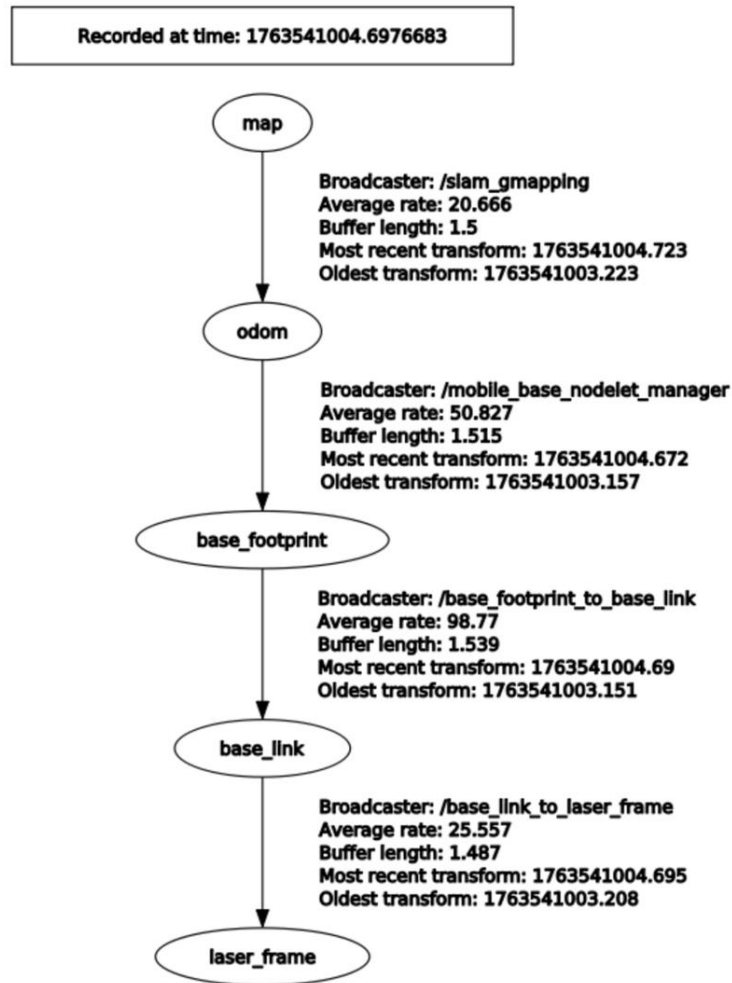
除感測與控制資料外，TF 相關 Topic 亦需納入整體資料流。`/tf` 負責發布動態座標轉換，使地圖、里程計、底盤與感測器 frame 能維持正確關係；若使用固定感測器安裝位置，則可透過靜態轉換維持其相對位置。Topic 資料流與 TF Tree 需同步檢查，因為資料即使能被訂閱，若缺乏正確座標對應，仍無法被導航或建圖模組正確使用。為確認資料流是否正常，實作過程中檢查 Topic 是否存在、資料是否持續更新，以及發布與訂閱關係是否符合設計。列出目前系統中的 Topic、觀察特定 Topic 內容，以及確認資料更新頻率。此流程可協助判斷問題來源是節點未啟動、Topic 名稱不一致、資料格式不符，或資料更新頻率不足。目的是建立各模組之間清楚且可追蹤的資料傳遞關係。LiDAR、camera、底盤、定位、規劃與控制模組各自負責不同資料，但必須透過統一的 Topic 介面連接，才能形成完整的 AMR 系統。

3.2.2 TF Tree 與座標轉換設定

在 Topic 資料流確認後，需進一步建立各座標系之間的轉換關係，使不同來源的感測資料能被放置於正確的空間位置。AMR 系統中同時包含地圖座標、里程計座標、底盤座標與感測器座標；若僅確認/scan、/odom 或影像資料能正常發布，仍不足以保證後續建圖與導航功能可正確執行。因此，在進入 SLAM、AMCL 與路徑規劃前，必須先完成 TF Tree 設定，確保各模組使用一致的座標基準。

實作過程中，座標系以移動機器人常見架構進行配置，主要包含 map、odom、base_footprint、base_link 與 laser_frame。其中，map 作為全域地圖座標系，對應於 SLAM 或定位系統所建立之環境地圖；odom 為里程計座標系，用於描述機器人短時間內的連續移動狀態；base_footprint 表示機器人底盤投影至地面平面的中心位置，適合用於二維導航；base_link 則代表機器人本體座標；laser_frame 為 LiDAR 安裝位置所對應的感測器座標系。透過上述座標分層，可使地圖、底盤與 LiDAR 掃描資料維持明確的空間關係。

如圖十五所示，系統實際建立之 TF Tree 依序形成 map → odom → base_footprint → base_link → laser_frame 的結構。此關係表示 LiDAR 掃描到的障礙物位置可由 laser_frame 轉換至機器人本體座標，再進一步對應至里程計座標與地圖座標。換言之，LiDAR 所取得的資料不只是單純的距離數值，而是能透過座標轉換被放入整體地圖與導航架構中，供 SLAM、costmap 與避障模組使用。



圖十五、AMR 的 Transform Tree

在設定過程中，首先需確認底盤節點是否能正確發布 odom → base_footprint 的轉換。此轉換來自機器人的移動狀態，會隨底盤前進、轉向或停止而更新，是導航系統追蹤機器人短時間位移的重要依據。若此轉換中斷，系統可能無法判斷機器人相對於起始位置的變化，進而影響地圖更新、定位估測與路徑規劃。其次，需設定機器人本體與 LiDAR 之間的固定轉換，也就是 base_link → laser_frame。由於 LiDAR 安裝於機器人平台上方，其位置相對於底盤中心固定不變，因此此轉換屬於靜態轉換。設定時需確認 LiDAR 的安裝方向與高度是否合理，避免掃描資料在 RViz 中出現方向相反、位置偏移或與機器人模型不一致的情形。若 laser_frame 的位置設定錯誤，系統可能會將障礙物標記在錯誤方向，使避障判斷與 costmap 更新產生偏差。

map → odom 的轉換則通常由 SLAM 或定位相關節點產生。進行建圖時，SLAM 會根據 LiDAR 掃描資料與里程計資訊估測機器人在地圖中的位置，並維持 map 與 odom

之間的關係；進入定位階段後，AMCL 亦會依據既有地圖與感測資料修正此轉換。此設定使機器人能同時保留里程計的短時間連續性，以及地圖座標的全域參考能力。

在模擬階段，TF Tree 可先用於檢查虛擬機器人模型、虛擬 LiDAR 與模擬地圖之間的座標關係。待座標結構穩定後，再將相同邏輯移至實體平台，確認實體 LiDAR、TurtleBot2 底盤與 ROS 節點之間的轉換是否一致。透過先模擬、後實體的方式，可在硬體操作前先排除 frame 命名、座標方向與轉換缺漏等問題，降低後續整合時的除錯成本。

實作時亦需注意 TF Tree 不應出現重複發布或斷裂情形。若同一組座標轉換由多個節點同時發布，可能導致資料衝突；若某個 frame 缺少父座標系，則會使整體座標鏈無法連接。為避免此類問題，整合過程中需統一 frame 命名，並確認各節點所發布的轉換關係是否符合系統設計。例如：LiDAR 資料需對應至 laser_frame，底盤移動需對應至 base_footprint 或 base_link，地圖定位則需與 map、odom 保持一致。

TF Tree 設定完成後，可透過 RViz 觀察機器人模型、雷射掃描點、地圖與座標軸之間的位置關係。若雷射掃描點能隨機器人移動而正確對應至環境輪廓，且各座標 frame 之間沒有明顯偏移，代表座標轉換已可支援後續建圖與導航流程。此階段的確認重點不在於評估建圖精度，而是確保感測資料、機器人本體與地圖之間具備可追蹤且一致的空間基準。SLAM 可根據正確的 LiDAR 安裝位置建立地圖，AMCL 可利用地圖與雷射掃描結果修正定位，costmap 則能將障礙物標記於正確區域。透過座標轉換的整理，感測資料不再只是分散於各節點中的數值，而能被整合為導航系統可使用的空間資訊，進一步支援 AMR 於醫療巡檢情境中的移動、避障與目標點導航。

3.3 模擬環境與導航建置

由於 AMR 導航功能涉及地圖建立、定位估測、路徑規劃、避障控制與底盤移動等多項模組，若一開始即於實體平台上進行完整測試，容易因硬體連接、感測器資料、座標轉換或參數設定等問題交互影響，增加除錯難度。因此，開發流程先以 Gazebo 與 RViz 建立可觀察、可重複測試的模擬環境，再逐步移轉至實體平台進行驗證。

模擬環境的主要目的，是在不受實體場地與硬體限制影響的情況下，先確認機器人模型、感測器資料流與導航模組是否能依照預期運作。透過 Gazebo 可建立室內場

景、牆面與障礙物，並載入 TurtleBot 模型進行移動模擬；透過 RViz 則可觀察 LiDAR 掃描資料、地圖建立狀態、機器人位置、路徑規劃結果與 TF 座標關係。兩者分別提供物理模擬與資料視覺化功能，使開發過程能更清楚地檢查各模組之間的連動狀態。

此階段的建置流程可分為四個主要部分。第一：建立 Gazebo 模擬場景，使機器人能在接近室內巡檢情境的空間中進行移動與感測資料取得。第二：利用 LiDAR 掃描資料進行 SLAM 建圖，使系統能由未知環境逐步生成可供導航使用的地圖。第三：於地圖建立後導入 AMCL 定位與 RViz 視覺化，確認機器人能在既有地圖中估測自身位置。第四：設定 costmap 與路徑規劃，使 AMR 能根據地圖、障礙物與目標點產生可行移動路徑。

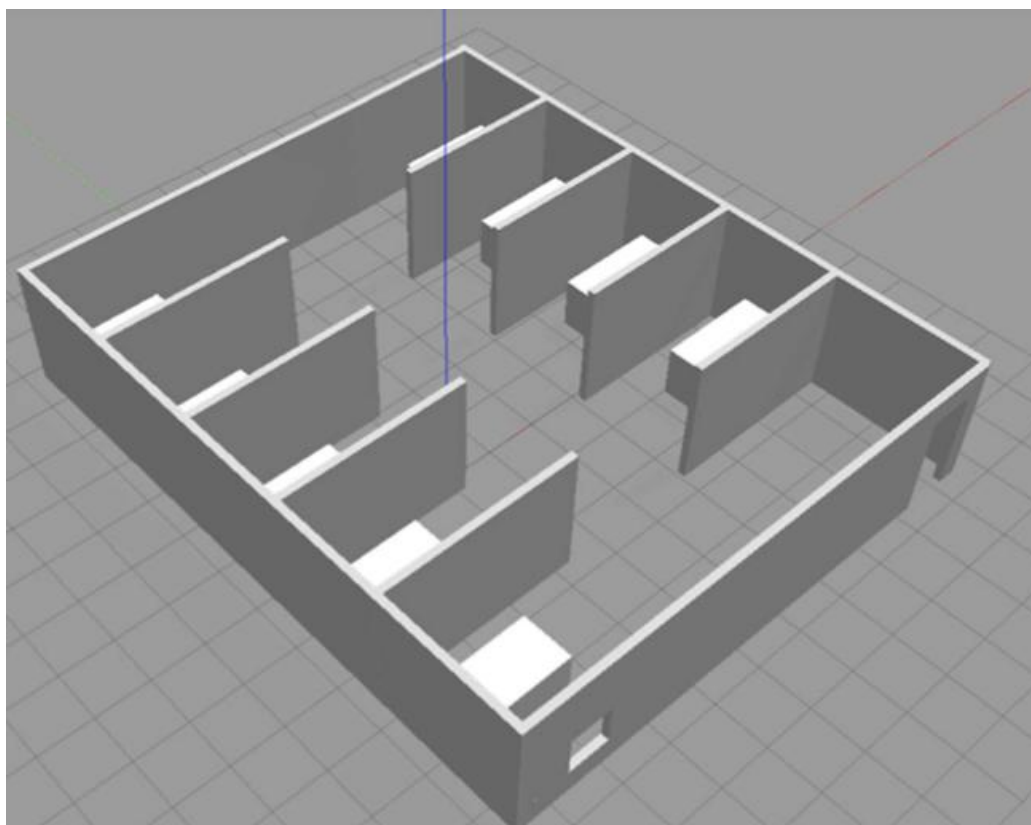
在模擬階段完成基本流程後，相關設定可作為實體平台操作前的參考依據。例如：Topic 名稱、TF frame 命名、地圖載入方式、導航節點啟動順序與 costmap 參數架構，皆可先於模擬中確認，再依照實體感測器與場地條件進行調整。如此可避免在實體測試時同時面對環境、硬體與程式問題，也能使系統開發由單一模組測試逐步進入整合測試。模擬環境與導航建置並非最終成果展示，而是 AMR 導航功能形成前的重要中介階段。透過先模擬、後實體的開發方式，可先建立完整的資料流與導航流程，再將其應用於真實平台。

3.3.1 Gazebo 模擬環境建置

在進行實體平台操作前，先以 Gazebo 建立模擬環境，作為導航流程與感測資料驗證的前置步驟。由於 AMR 系統同時包含感測器資料、座標轉換、建圖、定位、路徑規劃與底盤控制等模組，若直接於實體平台上測試，容易因多項因素同時發生而增加除錯困難。因此，開發初期先於虛擬環境中確認機器人模型、場景配置與感測資料是否能依照預期運作，再將相關流程移轉至實體平台。

Gazebo 模擬環境主要用於建立接近室內巡檢情境的測試空間。場景中可配置牆面、走廊、轉角與障礙物，使機器人能在具有限制邊界的環境中移動，而非僅在空白平面上執行控制命令。此配置有助於模擬病房巡檢時可能遇到的室內動線，例如從起始點進入走廊、接近病房門口、繞過障礙物或返回指定位置等情境。透過模擬場景的建立，可先確認導航流程中的基本空間關係，再進一步加入建圖與路徑規劃功能。

如圖十六所示，Gazebo 中載入 TurtleBot 模型後，平台可在模擬場域中執行移動與感測資料產生。此階段並非著重於最終導航成果，而是用來確認模型是否能被正確生成，機器人能否接收控制命令，以及模擬感測器資料是否能進入 ROS 系統。當機器人模型、場景物件與感測資料皆能正常啟動後，後續才可進行 SLAM 建圖與路徑規劃流程。



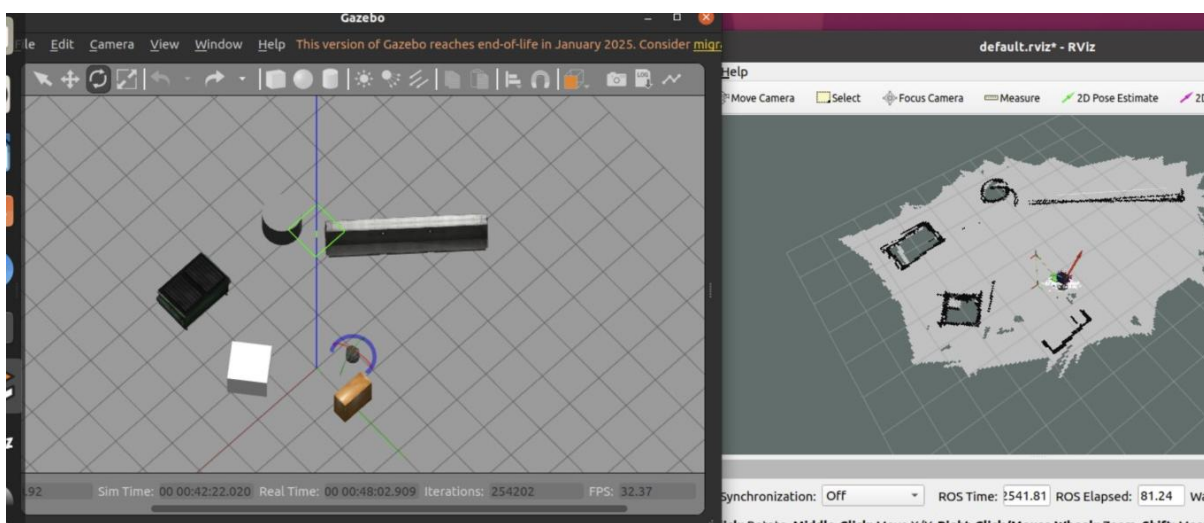
圖十六、Gazebo 模擬畫面

模擬場景建立後，需進一步確認 LiDAR 模擬資料是否能與 ROS Topic 對應。虛擬 LiDAR 會依據 Gazebo 場景中的牆面與障礙物產生掃描資料，並透過 /scan 提供給後續模組使用。此流程可使 SLAM、Costmap 與避障節點在未接入實體 LiDAR 前，先以模擬資料完成基本串接。若模擬環境中的 /scan 無法正常發布，或掃描方向與場景不一致，後續建圖與導航流程即無法順利進行。

完成感測資料確認後，Gazebo 模擬環境可作為導航演算法初步驗證的平台。機器人在場景中移動時，系統可同步觀察其路徑、速度命令與感測資料變化，並檢查是否存在模型碰撞、移動方向錯誤或資料流中斷等問題。相較於直接操作實體機器人，模擬環境可降低硬體碰撞風險，也方便反覆測試不同場景配置與參數設定。Gazebo 與

RViz 在此階段扮演不同角色。Gazebo 主要呈現機器人與環境之間的物理模擬關係，包含模型位置、場景物件與移動狀態；RViz 則用於觀察 ROS 資料，例如：LiDAR 掃描、地圖生成、機器人座標與路徑規劃結果。兩者搭配使用，可同時檢查模擬世界中的機器人行為與 ROS 系統中接收到的資料，使開發過程更容易定位問題來源。

圖中左側為 Gazebo 模擬環境，用於建立機器人、障礙物與場域配置；右側為 RViz 視覺化畫面，可呈現 LiDAR 掃描資料、機器人位置與 SLAM 建圖結果。透過兩者搭配，可在實體測試前先確認模擬場域與建圖流程是否正常運作。



圖十七、Gazebo 模擬時的虛擬世界(左)與 RViZ(右)

模擬環境建置完成後，相關設定可作為實體平台測試前的參考。包含場景中障礙物配置、機器人模型尺寸、感測器資料流、Topic 名稱與導航啟動順序，皆可先於模擬階段整理。當模擬流程能穩定執行後，再將相同架構套用至 TurtleBot2 與 RPLiDAR A1 的實體平台，可有效降低實體整合時的風險與除錯成本。Gazebo 模擬環境建置是 AMR 導航功能形成前的重要步驟。透過虛擬場景，可先完成機器人模型、感測資料與 ROS 節點的初步驗證，並為後續 SLAM 建圖、AMCL 定位、Costmap 設定與路徑規劃提供測試基礎。此流程使系統開發由模擬逐步銜接至實體操作，符合先驗證架構、再進行整合的開發邏輯。

3.3.2 SLAM 建圖流程

此階段的目的是在於使 AMR 能依據 LiDAR 掃描資料與自身移動資訊，逐步建立室內環境地圖，作為後續定位、路徑規劃與巡檢任務的基礎。建圖流程先於模擬環境

中進行，待資料流與座標關係確認後，再移轉至實體平台，使開發過程能由可控環境逐步銜接至真實場域。SLAM 建圖開始前，需先完成基本確認，確認感測資料、里程計資訊與座標轉換皆能正常運作，避免地圖生成過程中出現重疊、偏移或斷裂等問題。

如圖十八所示，SLAM 建圖流程由啟動 ROS 系統開始，接著載入模擬場景或實體 AMR 平台，再啟動 LiDAR 與 SLAM 相關節點。當 LiDAR 開始發布/scan 資料後，SLAM 節點會根據連續掃描結果與機器人移動狀態更新地圖。此時可透過 RViz 同時觀察機器人位置、雷射掃描點與地圖輪廓，確認系統是否能正確接收環境資訊。



圖十八、SLAM 建圖實作流程圖

在模擬階段，AMR 於 Gazebo 場景中移動，透過虛擬 LiDAR 取得牆面、轉角與障礙物等環境資訊。操作時需控制機器人沿著場景邊界與主要通道緩慢移動，使 LiDAR 能從不同角度掃描環境。若機器人移動過快，可能導致連續掃描資料之間差異

過大，使地圖邊界不穩定；若只在單一區域停留，則地圖會缺乏完整空間資訊。因此，建圖時需讓機器人均勻通過主要路徑，使場景輪廓逐步補齊。

建圖過程中，RViz 是主要的觀察工具。透過 RViz 可即時檢查雷射掃描點是否與環境邊界相符，也能觀察地圖是否隨機器人移動逐漸擴展。當地圖線條出現明顯偏移或重疊時，通常需回頭檢查里程計資料、TF 轉換、LiDAR 掃描方向或機器人移動速度。此階段的重點不在於分析最終地圖精度，而是確認地圖能依照感測資料逐步生成，並能作為後續導航流程的輸入。

完成模擬建圖後，可將相同流程移至實體平台。實體建圖時，由 RPLiDAR A1 取得真實環境掃描資料，TurtleBot2 底盤提供移動資訊，Raspberry Pi 負責執行 ROS 節點與資料傳遞。與模擬環境相比，實體操作需額外注意更多模擬操作時沒有的外部環境等因素。為降低誤差，機器人移動時應保持適當速度，並避免在狹窄區域快速旋轉，使 LiDAR 能持續取得穩定掃描資料。

建圖完成後，需將地圖儲存為後續導航可讀取的格式。一般會儲存為影像地圖檔與參數檔，例如：map.pgm 與 map.yaml。其中，影像地圖用於記錄可通行區域、障礙區域與未知區域；參數檔則保存地圖解析度、原點位置與其他地圖設定。後續進行 AMCL 定位與路徑規劃時，系統會載入此地圖，使 AMR 能在既有環境中估測自身位置並規劃前往目標點的路徑。在此流程中，SLAM 建圖並非單一節點的獨立工作，而是由 LiDAR 掃描、底盤移動、座標轉換與視覺化工具共同支援。LiDAR 提供環境距離資料，底盤移動使感測器能從不同位置觀察場域，TF Tree 負責維持資料的空間一致性，RViz 則協助開發者檢查地圖生成狀態。當上述環節能穩定配合後，才可進一步進行 AMCL 定位、Costmap 設定與目標點導航。

SLAM 建圖流程是 AMR 從感測資料進入導航應用的關鍵步驟，透過先模擬、後實體的方式，可先建立完整建圖流程，再依照真實環境條件進行調整。此階段完成後，系統便具備可供定位與路徑規劃使用的環境地圖。

3.3.3 AMCL 定位與 RViz 視覺化

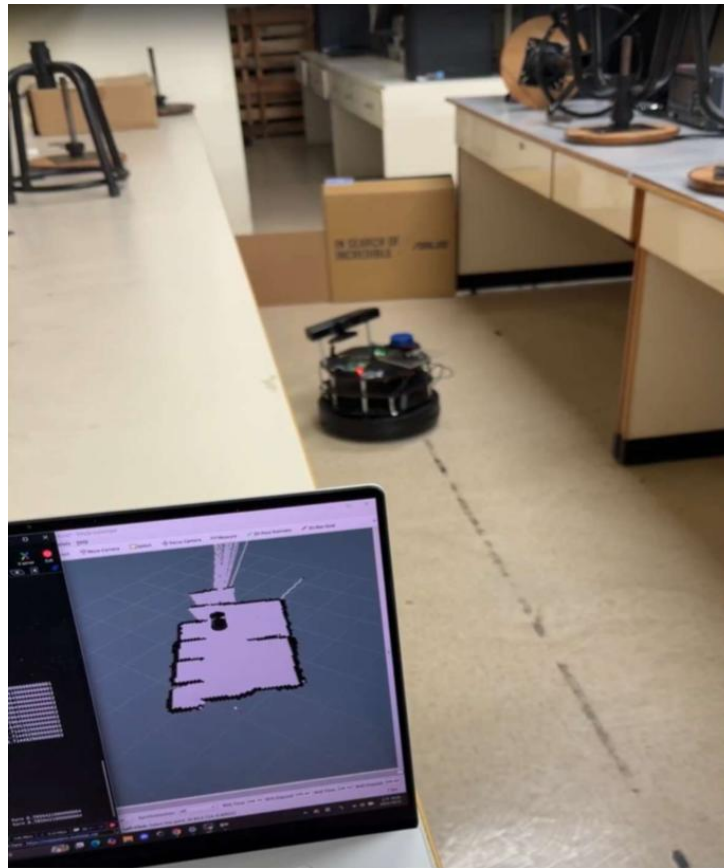
完成 SLAM 建圖並儲存地圖檔後，後續流程需將地圖載入導航系統，並建立機器人在地圖中的定位能力。此階段以 AMCL 作為定位模組，搭配 RViz 進行視覺化觀

察，使開發者能確認機器人位置、雷射掃描資料、地圖邊界與座標系是否能正確對應。相較於 SLAM 建圖階段著重於建立未知環境地圖，AMCL 定位階段則是在既有地圖中估測機器人目前所在位置，為後續目標點導航與路徑規劃提供基準。

在實作流程上，首先需載入前一階段建立之地圖檔，包含地圖影像檔與對應參數檔。地圖載入後，系統會將其作為全域參考環境，提供 AMCL、Costmap 與路徑規劃模組使用。接著啟動底盤里程計、LiDAR 掃描資料、TF Tree 與 AMCL 節點，使定位模組能同時接收地圖資訊、機器人移動狀態與雷射感測資料。此流程可使 AMR 從單純擁有地圖，進一步具備在地圖中判斷自身位置的能力。

AMCL 啟動後，需在 RViz 中檢查定位相關資訊是否正確顯示。RViz 可同時呈現地圖、機器人模型、雷射掃描點、座標軸與定位估測結果，因此適合作為定位流程中的主要觀察工具。若雷射掃描點能大致貼合地圖中的牆面或障礙物邊界，代表感測資料與地圖之間具有合理對應；若掃描點與地圖明顯錯開，則需檢查初始位置設定、TF Tree、LiDAR frame 或里程計資料是否存在問題。

如圖十九所示，AMR 可於已建立之地圖中透過 RViz 顯示其相對位置，並將實體平台運行狀態與地圖資料同步呈現。此畫面可用於確認機器人是否已被放置於正確地圖座標中，也能觀察 LiDAR 掃描資料是否隨機器人移動而更新。透過 RViz 的視覺化功能，可在正式進行目標點導航前，先檢查定位、感測與座標轉換是否具備基本一致性。定位流程中，初始位置設定是一項重要步驟，若機器人實際位置與系統估測位置差距過大，AMCL 可能需要較長時間才能收斂，甚至造成導航路徑規劃錯誤。因此，在啟動定位後，需先於 RViz 中指定或調整機器人初始姿態，使地圖中的機器人位置接近實際擺放位置。完成初始定位後，再透過短距離移動或原地旋轉，使 LiDAR 掃描資料與地圖特徵進一步匹配，提高定位穩定度。



圖十九、AMR 實際運行於建置地圖中之 RViz 畫面

在模擬階段，AMCL 定位主要用於檢查已儲存地圖、虛擬 LiDAR、機器人模型與 TF Tree 是否能正常配合。待模擬流程穩定後，再將相同流程移轉至實體平台。實體定位時需額外注意環境是否與建圖時一致，若出現明顯變化，可能影響雷射掃描與地圖之間的匹配程度。為降低定位誤差，實體測試時應盡量保持建圖環境與定位環境一致，並確認 LiDAR 掃描範圍未被機器人本體或線材遮擋。

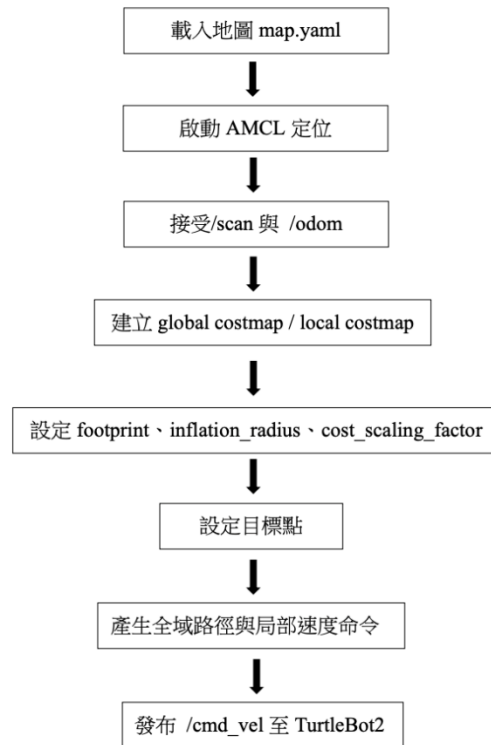
完成定位確認後，RViz 仍可作為後續導航流程的監看介面。當機器人接收目標點後，可在 RViz 中觀察全域路徑、局部路徑、Costmap 障礙區域與機器人移動方向。此功能使開發者能在同一畫面中檢查地圖是否正確載入、定位是否穩定、路徑是否合理以及感測資料是否持續更新。因此，RViz 不僅用於顯示定位結果，也扮演整合導航流程時的重要除錯工具。AMCL 定位與 RViz 視覺化是由建圖流程銜接至自主導航流程的關鍵階段。地圖提供全域環境基準，LiDAR 與里程計提供即時狀態資訊，AMCL 負責估測機器人於地圖中的位置，RViz 則協助觀察定位與導航資料是否一致。完成此階段後，AMR 才能在既有地圖上進行目標點導航、Costmap 建立與醫療巡檢任務規劃。

3.3.4 Costmap 與路徑規劃設定

完成地圖建立與 AMCL 定位流程後，導航系統需進一步設定 Costmap 與路徑規劃模組，使 AMR 能根據環境地圖、即時障礙物資訊與目標點產生可執行的移動路徑。此階段的工作重點在於將前面建立的地圖、LiDAR 掃描資料、里程計資訊與機器人外型尺寸整合至導航架構中，並調整相關參數，使系統能在模擬環境與實體場域中進行目標點移動與避障。

在導航流程中，costmap 扮演連接感測資料與路徑規劃的中介角色。系統載入 SLAM 建立之地圖後，先以靜態地圖作為全域環境基準，再結合 LiDAR 所發布的/scan 資料更新機器人周圍障礙物狀態。由於 AMR 在移動過程中不只需要知道目標點方向，也必須即時判斷周圍是否存在障礙物，因此設定 costmap 時需同時考慮全域路徑規劃與局部避障需求。設定流程首先需確認地圖檔已正確載入，並使 AMCL 能提供機器人在地圖中的位置估測。若定位資訊不穩定，global costmap 會難以判斷機器人目前所在位置，進而影響路徑規劃結果。且系統需確認/scan 與/odom 能被導航模組接收，使 local costmap 可依據雷射掃描與機器人移動狀態更新周圍環境資訊。此處的設定延續前面 Topic 資料流與 TF Tree 的整合結果，因此需確保各 frame 與資料來源皆能正常對應。

Costmap 與路徑規劃的設定流程如圖二十所示。系統由地圖載入與定位確認開始，接著接收 LiDAR 與里程計資料，建立 global costmap 與 local costmap，再依據機器人外型、障礙物膨脹範圍與代價衰減參數進行路徑規劃設定，最後由規劃器產生移動路徑與速度命令。



圖二十、costmap 與路徑規劃設定流程圖

機器人外型參數是 costmap 設定中的重要項目。由於 TurtleBot2 具有實際寬度與底盤範圍，若系統只將機器人視為單一點，路徑規劃時可能產生過於貼近牆面或障礙物的路線。因此，需依照平台尺寸設定 footprint，使導航模組能將機器人本體範圍納入碰撞判斷。若 footprint 設定過小，可能增加碰撞風險；若設定過大，則可能使可通行空間被過度限縮，導致機器人難以通過狹窄區域。

除 footprint 外，inflation radius 與 cost scaling factor 亦會影響路徑規劃行為。inflation radius 用於在障礙物周圍建立安全緩衝區，使機器人不會緊貼牆面或障礙物移動；cost scaling factor 則影響障礙物周圍代價下降的速度。需依照場域寬度、機器人尺寸與避障需求調整兩者，使路徑既能保持安全距離，又不至於因代價區域過大而造成無法通行。先前實作過程中也曾發現，若 costmap 參數設定不當，可能造成旋轉時持續判定碰撞，或使規劃路徑偏離實際可行區域，因此此階段需反覆檢查參數與機器人行為是否一致。

在更新頻率設定上，global costmap 與 local costmap 需依照系統負載與導航需求調整。若更新頻率過低，障礙物資訊可能無法即時反映於 Costmap 中；若更新頻率過

高，則可能增加 Raspberry Pi 運算負擔，影響其他 ROS 節點執行。由於需同時處理 LiDAR、底盤、Camera 與導航節點，參數設定時需在即時性與運算負載之間取得平衡，避免因資料更新延遲或運算壓力造成導航不穩。

完成 costmap 設定後，接續進行目標點路徑規劃。當系統接收到目標位置後，global planner 會根據 global costmap 產生前往目標點的全域路徑；local planner 則根據 local costmap 與機器人當前狀態，產生短時間內可執行的速度命令。速度命令最後透過 /cmd_vel 發布至 TurtleBot2 底盤，使規劃結果轉換為實際移動行為。此流程將地圖、定位、感測與控制串接在同一導航架構中，是 AMR 能完成目標點移動的主要依據。

在模擬階段，costmap 與路徑規劃設定先於 Gazebo 場景中進行初步確認。透過模擬環境，可觀察機器人在牆面、轉角與障礙物附近的移動趨勢，並檢查路徑是否過於貼近障礙物或出現無法規劃的情形。待基本設定穩定後，再將相同架構套用至實體平台，並依照真實場地條件調整 footprint、inflation radius、cost scaling factor 與更新頻率等參數。costmap 與路徑規劃設定是由已建立地圖轉向可執行導航的重要步驟。SLAM 提供環境地圖，AMCL 提供機器人位置，LiDAR 提供即時障礙物資訊，costmap 將上述資料轉換為可供規劃器判斷的通行成本，最後由路徑規劃器產生移動路徑與速度命令。完成此設定後，AMR 才能在後續醫療病房巡檢流程中依照目標點移動，並在遇到障礙物時進行路徑調整與避障。

3.3.5 自主探索與避障演算法實作

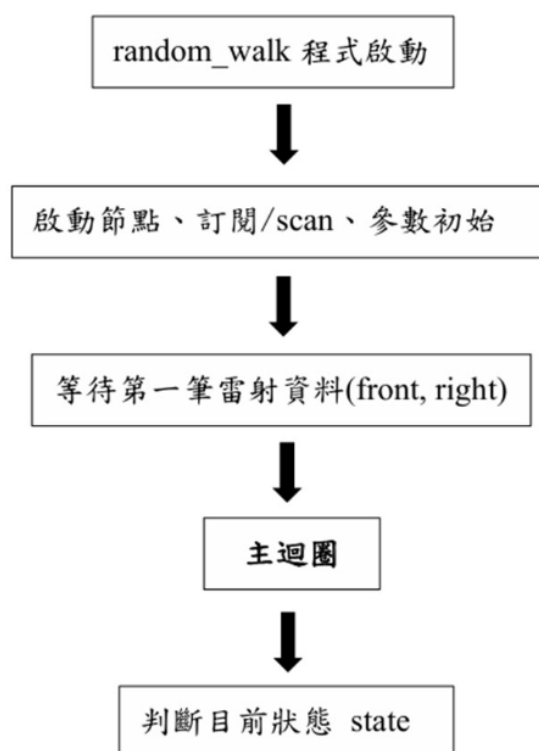
在 SLAM 建圖階段，AMR 尚未具備完整環境地圖，也無法直接依賴既有地圖進行目標點導航。若完全由人員手動控制機器人移動，雖然可以完成建圖，但會降低系統自動化程度，也無法呈現 AMR 在未知環境中自主探索的能力。因此，系統設計 random_walk 自主探索演算法，使機器人能在陌生場域中依據 LiDAR 即時距離資料進行移動判斷，於建圖過程中自行前進、沿牆與避障，協助 SLAM 取得更完整的環境掃描資料。

random_walk 的主要目的不是取代地圖完成後的正式導航系統，而是在地圖尚未建立完整之前，提供 AMR 基本的自主移動能力。此演算法讓機器人不必依賴人工遙控，即可在未知空間中持續探索環境。機器人移動過程中，LiDAR 會不斷取得周圍牆面、

通道與障礙物資訊，SLAM 節點則根據連續掃描資料逐步建立地圖。透過自主探索方式，可使建圖流程更接近全自動化 AMR 的設計目標。

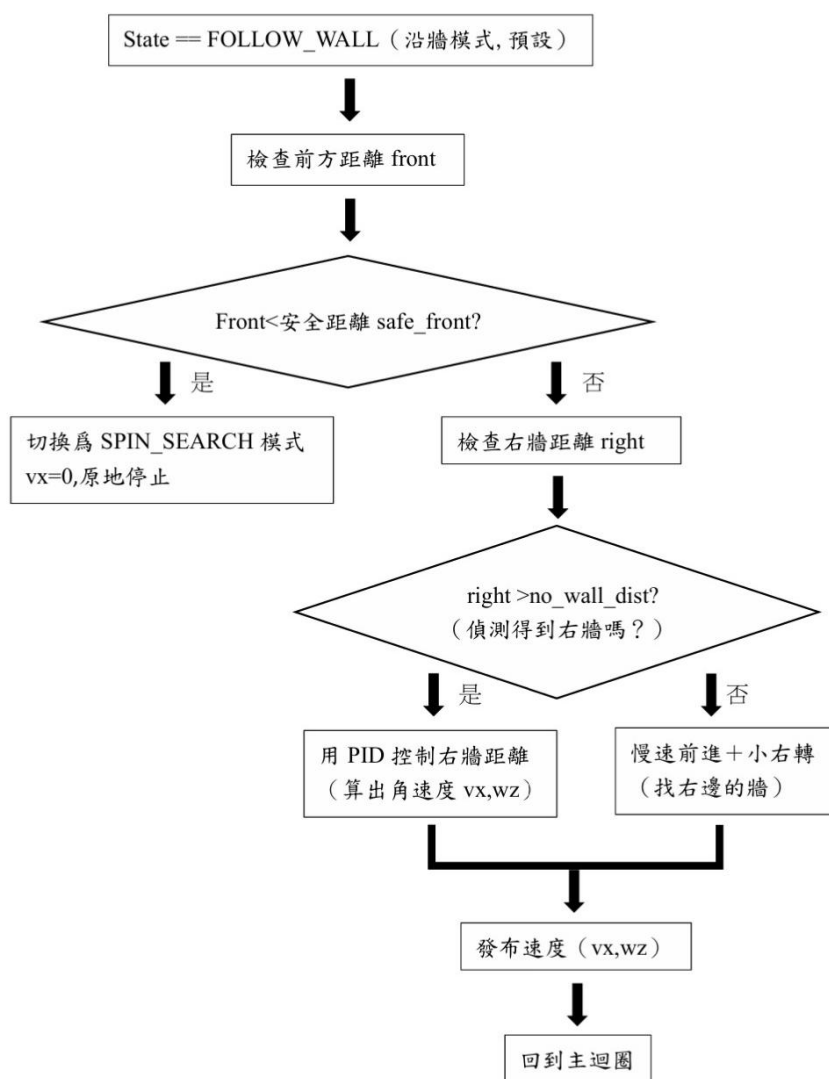
演算法主要根據 LiDAR 掃描資料中的前方距離與右側距離進行判斷。前方距離用於確認機器人是否能繼續前進，右側距離則用於判斷機器人與牆面或通道邊界之間的相對位置。藉由這兩項距離資訊，系統可在 FOLLOW_WALL 沿牆模式與 SPIN_SEARCH 旋轉搜尋模式之間切換，使 AMR 在探索未知環境時不會只依固定方向前進，而能依據周圍空間狀態調整行為。

random_walk 主程式流程如圖二一所示。程式啟動後，系統先完成節點初始化、訂閱/scan，並設定相關判斷參數。當第一筆有效的雷射資料被接收後，程式進入主迴圈，持續讀取前方與右側距離，再依據目前狀態 state 進行模式判斷。此流程使機器人能在建圖期間持續更新自身移動行為，並透過 LiDAR 掃描資料維持與環境邊界之間的基本安全距離。



圖二一、random_walk 自主避障主程式流程圖

在 FOLLOW_WALL 沿牆模式中，系統先檢查前方距離 front 是否低於安全距離 safe_front，流程如圖二十二所示。若前方距離不足，表示機器人正接近障礙物或牆面，演算法會停止線速度並切換至 SPIN_SEARCH 模式，避免 AMR 持續向前造成碰撞。若前方空間仍可通行，則進一步檢查右側距離 right，用以判斷機器人是否能以右側牆面作為參考邊界。



圖二十二、random_walk 中 FOLLOW_WALL 沿牆模式流程圖

當右側距離可被穩定量測時，系統會以目前右側距離與目標牆距之間的誤差作為控制依據，計算機器人所需的角速度修正量，使 AMR 在沿牆移動時能維持較穩定的側向距離。如圖二十三所示，此處採用 PID 架構進行右牆距離控制，但由於積分權重 Ki 設為 0，實際控制主要由比例項與微分項影響。比例項用於根據目前距離誤差產生修

正方向，微分項則用於抑制誤差變化過快所造成的擺動，使機器人沿牆時不會因單次距離變化而產生過度轉向。

```
# ===== PID 參數 (右牆距離控制) =====  
# 使用PD控制, Ki 權重設 0 (幾何限制)  
self.Kp = rospy.get_param("~Kp", 2.0)  
self.Ki = rospy.get_param("~Ki", 0.0)  
self.Kd = rospy.get_param("~Kd", 0.3)
```

圖二十三、於 random_walk 演算法的 PID 參數(Integral 權重為 0)

(Kp 為 Proportional 權重, Ki 為 Integral 權重, Kd 為 Derivative 權重)

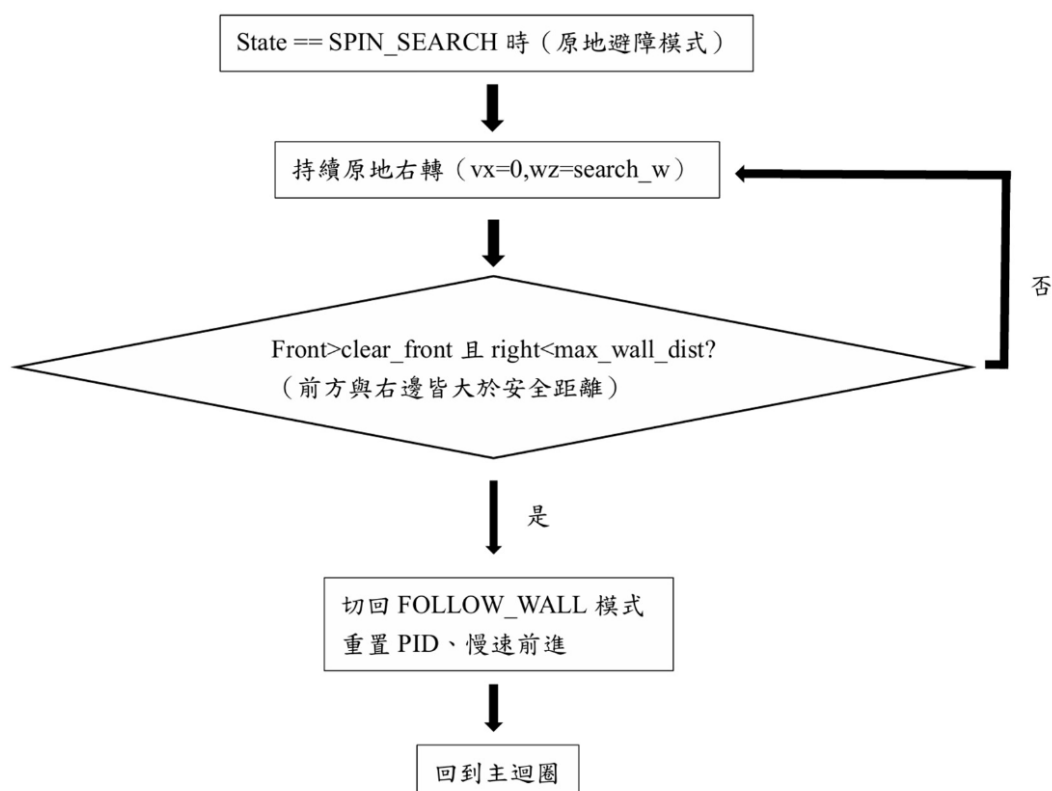
如圖二十四所示，程式中先讀取右側距離 right，並以目前右側距離與目標距離 target_wall 之差作為誤差值。若右側距離無法取得有效數值，系統會以預設距離作為替代值，避免控制器因無限值或無效資料而中斷。接著，程式計算誤差變化量，並依據比例項、積分項與微分項組合出角速度 w。因為 Ki 設為 0，所以積分項不參與實際修正，可降低長時間累積誤差造成控制輸出過大的風險。

```
# PID 計算 (控制右牆距離)  
def update_pid(self, dt):  
    r = self.right  
    if not math.isfinite(r):  
        r = self.no_wall_dist * 2.0  
  
    # 誤差：目前右牆距離 - 目標距離  
    error = r - self.target_wall  
  
    # 積分項  
    self.int_err += error * dt  
    self.int_err = clamp(self.int_err, -self.int_max, self.int_max)  
  
    # 微分項  
    der = (error - self.prev_err) / dt if dt > 0 else 0.0  
    self.prev_err = error  
  
    # PID 輸出 (前面加負號：牆太遠 → 右轉；牆太近 → 左轉)  
    w = -(self.Kp * error + self.Ki * self.int_err + self.Kd * der)  
  
    # 限制最大角速度  
    w = clamp(w, -self.max_ang, self.max_ang)  
    return w
```

圖二十四、PD 控制器實行於 random_walk 演算法中(w 為角速度)

控制器輸出的角速度會再經由最大角速度限制，避免機器人因修正量過大而急速旋轉。當機器人距離右牆過遠時，角速度會使其逐漸向右修正；當距離過近時，則會向左修正，使底盤與牆面保持安全間距。若右側未偵測到可追蹤牆面，系統會改以慢速前進並略向右轉，嘗試重新取得右側邊界。最後，演算法將線速度與角速度發布至 `/cmd_vel`，由 TurtleBot2 底盤執行對應移動。

SPIN_SEARCH 旋轉避障模式則用於處理前方受阻的情況，如圖二十五所示。當系統切換至此模式時，AMR 會停止前進並原地右轉，持續檢查前方距離與右側距離是否恢復至安全條件。若前方與右側皆符合可通行判斷，演算法便切回 FOLLOW_WALL 模式，並以較慢速度重新進入沿牆移動。若條件仍未滿足，機器人會維持旋轉搜尋，直到找到可行方向為止。



圖二十五、random_walk 中 SPIN_SEARCH 旋轉避障模式流程圖

在 ROS 架構中，此避障演算法透過 `/scan` 與 `/cmd_vel` 形成感測至控制的閉迴路。LiDAR driver 持續發布 `/scan`，演算法節點讀取距離資料後判斷目前狀態，並輸出對應的速度命令。速度命令發布至 `/cmd_vel` 後，由底盤節點轉換為實際移動，使 AMR 能在建圖期間依據周圍環境即時修正移動方向。

參數設定會直接影響避障行為，safe_front 若設定過小，機器人可能過度接近障礙物；若設定過大，則容易過早進入旋轉模式，使移動效率降低。右側目標牆距會影響沿牆時的路徑位置，若距離設定不合理，機器人可能過度靠近牆面或偏離通道。角速度上限則用於限制修正幅度，避免控制輸出過大造成底盤擺動。實作時需依據場地寬度、LiDAR 掃描狀況與 TurtleBot2 底盤反應調整相關參數，使探索行為能在安全性、掃描完整度與移動穩定性之間取得平衡。

3.4 Sensor Fusion 感知架構設計

完成導航與避障架構後，系統進一步加入 Camera 影像資料，使 AMR 的環境感知不僅依賴單一感測來源。LiDAR 可提供穩定的距離量測，適合用於 SLAM 建圖、定位、costmap 更新與主要避障判斷；camera 則可取得前方影像資訊，補充 LiDAR 固定掃描平面不易涵蓋的低矮障礙物，同時支援物件辨識、口罩辨識與顏色辨識等語意感知功能。兩者在資料型態與適用情境上不同，因此可透過 Sensor Fusion 概念形成互補式感知架構。並非將 LiDAR 與 Camera 視為彼此取代的感測器，而是依照各自特性分配任務。LiDAR 主要負責距離與空間結構資訊，在光線不足或背景複雜的情況下仍能提供穩定掃描資料；Camera 則負責取得視覺特徵，可觀察車體前方近地面區域，也能進一步辨識顏色、物件或人員配戴狀態。透過此分工，系統可降低單一感測器限制所造成的判斷缺口。

Camera 影像資料依任務需求進行 ROI 切分如表一式。在影像處理部分，系統依不同任務需求採用不同 ROI 擷取策略。低矮障礙物辨識主要針對 AMR 即將通過的近距離路面區域，因此固定擷取畫面下方 30% 作為判斷範圍，使模型能專注於車體前方低矮障礙物是否會影響移動。相較於使用完整影像，下方 30% ROI 可減少牆面、遠方背景與畫面上方物件干擾，也較能對應實際行進路徑上的通行安全。

除低矮障礙物辨識外，顏色辨識、口罩辨識與物件辨識則不採固定區域，而是依目標位置與任務需求使用動態 ROI。由於顏色標籤、檢測試紙、口罩或目標物在畫面中的位置可能隨機器人距離、拍攝角度與場域配置而改變，若固定取畫面某一區域，可能造成目標未落入 ROI 或擷取到過多無關背景。因此，系統會依辨識目標的位置或畫面內容調整 ROI 範圍，使 Camera 能在不同任務情境下取得較有效的影像資訊。

Camera 任務	ROI 選取方式	主要用途
低矮障礙物輔助避障	畫面下方 30%ROI	觀察車體前方近地面區域，補足 LiDAR 固定掃描平面不易涵蓋的低矮物體。
顏色辨識	動態 ROI	辨識標籤、試紙或巡檢目標之顏色狀態，作為任務判斷依據。
物件辨識	動態 ROI	擷取可能出現物件的區域，降低背景干擾並提高影像處理效率。
口罩辨識	人臉動態 ROI	判斷人員配戴狀態，作為 Camera 語意感知功能之一。

表一、Camera ROI 任務對照表

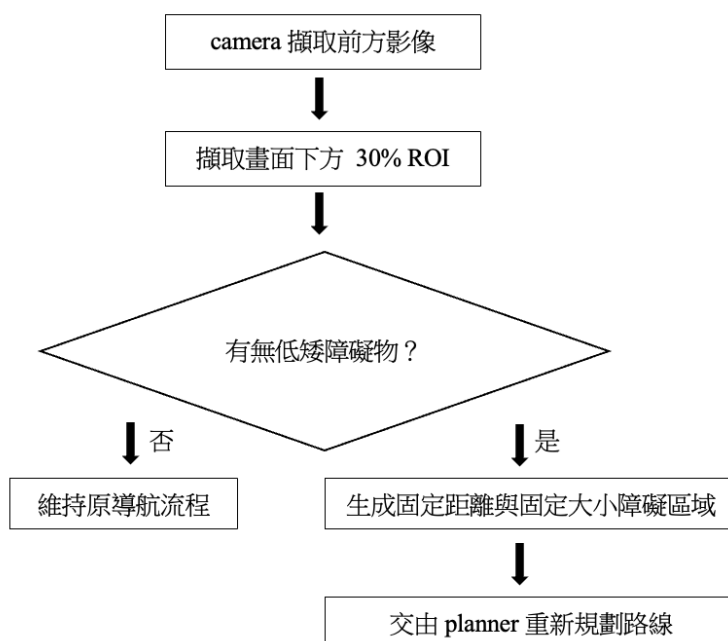
除低矮障礙物與顏色辨識外，Camera 亦可延伸至口罩辨識與物件辨識等語意感知任務。此類任務不採用固定 ROI，而是依畫面中的目標位置進行動態擷取。以口罩辨識為例，系統可先偵測畫面中人臉所在區域，再擷取人臉附近 ROI 進行配戴狀態判斷；物件辨識則可依目標物可能出現的位置調整擷取範圍，以減少無關背景對模型判斷的干擾。透過動態 ROI 設計，Camera 可依不同任務選擇適當的影像區域，使同一感測器能支援低矮障礙物、顏色標籤、口罩配戴與物件狀態等多種感知需求。

3.4.1 Camera 輔助低矮障礙物避障流程

在 Sensor Fusion 感知架構中，Camera 主要用於補充 LiDAR 固定掃描平面以外的影像資訊。相較於前面章節已說明的感測器資料擷取與 ROI 分區，此重於 Camera 影像如何被轉換為導航系統可使用的輔助避障資訊。並非取代 LiDAR-based 避障，而是針對 LiDAR 不易涵蓋的近地面低矮物體，提供額外判斷依據，使 planner 在規劃路徑時能將可能障礙區域納入考量。輔助低矮障礙物避障的輸入來源為機器人前方影像，讓模型專注於 AMR 即將通過的近距離路面範圍，而不是對整張影像進行廣泛判斷。

在目前實作條件下，camera 輔助避障採用有障礙物與無障礙物的二分類方式，如圖二十六所示。模型不直接估測障礙物精確距離或形狀，而是判斷前方 ROI 內是否存在可能影響移動的小型低矮物體。此設計與 RGB Camera 的特性有關，因為一般 RGB camera 無法像深度相機一樣直接取得物體距離；同時，Raspberry Pi 的運算效能有限，若要求系統在移動過程中即時完成高頻率影像推論，容易造成延遲並影響導航穩定性。因此，本階段以出發前或低速移動前的偵測為主要流程。當 camera 判斷前方 ROI 存在低矮障礙物時，系統會依據相機固定安裝角度與視野範圍，將該障礙物視為位於車體前方固定距離內的障礙區域。由於模型輸出僅表示有無障礙物，而非精確深度與尺寸，

因此實作上採用固定大小與固定距離的方式建立輔助障礙資訊。此方法雖無法描述障礙物的完整幾何形狀，但可作為 planner 的附加限制，使路徑規劃時避免直接通過該區域。



圖二十六、camera 輔助低矮障礙物避障流程圖

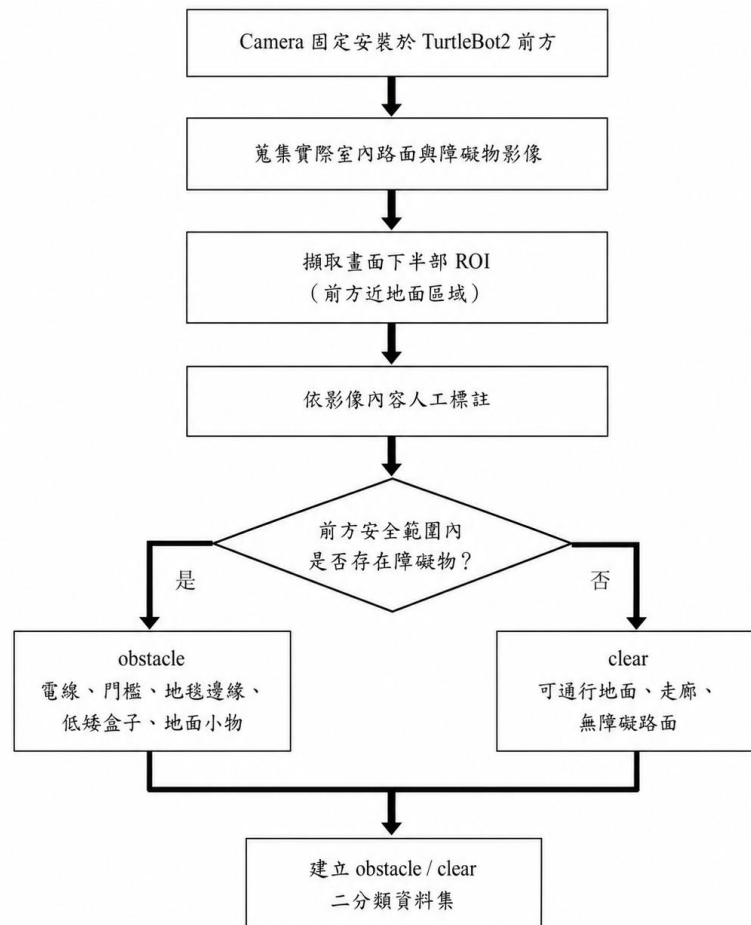
完成輔助障礙區域建立後，系統將其提供給路徑規劃流程使用。若 camera 未偵測到低矮障礙物，AMR 依照原本 LiDAR、costmap 與 planner 的導航流程移動；若 camera 偵測到障礙，則 planner 會在規劃時避開前方固定障礙區域，重新產生較安全的移動路徑。透過此方式，camera 的影像判斷結果能轉換為導航層可使用的資訊，而非僅停留在畫面辨識結果。

此流程的核心價值在於驗證 LiDAR 與 camera 的互補性。LiDAR 仍是主要的距離量測與導航感測器，能在光線不足或影像辨識條件不佳時提供穩定掃描資料；camera 則針對 LiDAR 可能未掃描到的低矮物體提供補充資訊。兩者透過 costmap 與 planner 進行功能層級整合，使 AMR 在面對室內小型障礙物時具備更完整的感知來源。需要注意的是，此階段仍屬於輔助避障流程，並非完整的即時動態視覺避障系統。受限於 Raspberry Pi 運算延遲、RGB Camera 缺乏深度資訊，以及固定視角假設，camera 目前較適合用於出發前或低速狀態下的近距離障礙物判斷。

3.4.2 有無障礙物辨識模型訓練

在 Camera 輔助低矮障礙物避障流程設計後，進一步建立可判斷前方路面是否存在低矮障礙物的影像辨識模型。本節主要說明有無障礙物辨識模型之資料收集、影像前處理、模型訓練與輸出應用方式。此模型補足 2D LiDAR 掃描高度固定時，對於近地面低矮物體可能無法完整偵測的限制。由於本專題使用一般 RGB Camera，無法直接取得障礙物的精確深度與距離，因此模型不設計為遠、中、近距離分類，而是將任務簡化為 obstacle 與 clear 兩類，用以判斷 AMR 前方安全範圍內是否存在可能影響移動的障礙物。

在資料收集方面，本模型比照 RGB+HSV 六通道 CNN 顏色辨識模型的建立流程，先由 Camera 拍攝實際場景影像，再依照影像內容進行分類標記。有無障礙物資料集建立流程如圖二十七所示，在收集資料時，Camera 固定安裝於 TurtleBot2 前方，使拍攝角度與實際巡檢時的視角一致，避免訓練資料與實際輸入畫面差異過大，其中 obstacle 類包含電線、地毯邊緣、門檻、低矮盒子、紙團、塑膠袋或其他可能低於 LiDAR 掃描高度的小型地面障礙物；clear 類則包含一般可通行地面、走廊、病房通道或未放置障礙物之前方路面。為提升模型在實際環境中的適應能力，資料收集時需涵蓋不同地板材質、光線條件、障礙物位置、障礙物大小與拍攝背景，使模型不只記住單一場景，而能學習分辨前方路面是否可通行的影像特徵。



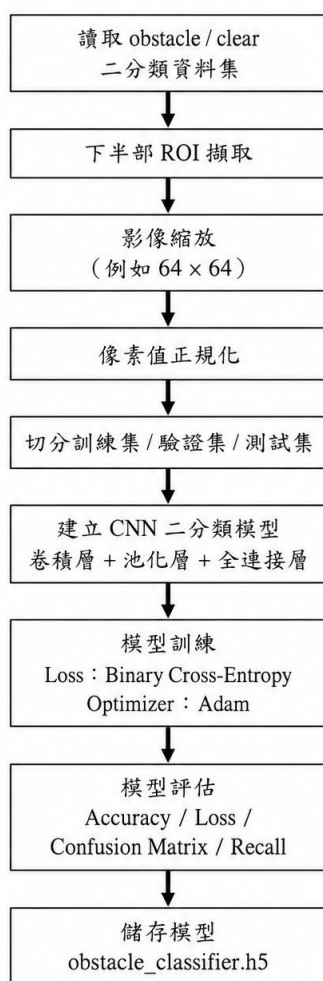
圖二十七、有無障礙物資料集建立流程圖

在影像前處理方面，低矮障礙物主要出現在機器人前方近地面區域，因此系統不直接使用完整影像作為模型輸入，而擷取畫面下半部 30% 作為 ROI。此設計可使模型專注於 AMR 即將通過的路面範圍，減少牆面、天花板、遠方背景或無關物件造成的干擾。擷取下半部 ROI 後，系統將影像統一縮放至符合 CNN 模型輸入格式，將影像像素值正規化，使其數值範圍轉換至模型較容易學習的區間。透過 ROI 擷取、尺寸統一與正規化處理，可確保訓練階段與即時辨識階段使用相同的資料格式，降低因輸入格式不一致而造成的辨識誤差。

本模型採用卷積神經網路 CNN 作為二分類架構，透過卷積層擷取影像中的邊緣、紋理、形狀與局部區域特徵，再透過池化層降低特徵維度並保留主要資訊，最後由全連接層完成分類判斷。輸出層可採用單一節點搭配 sigmoid 函數，輸出值代表前方 ROI 屬於 obstacle 的機率。若輸出機率高於設定門檻，系統判定前方存在低矮障礙物；若低於門檻，則判定目前路面為 clear。訓練時使用 binary cross-entropy 作為損失函數，並搭

配 Adam 優化器進行模型參數更新，使模型逐步學習 obstacle 與 clear 兩類影像之間的差異。

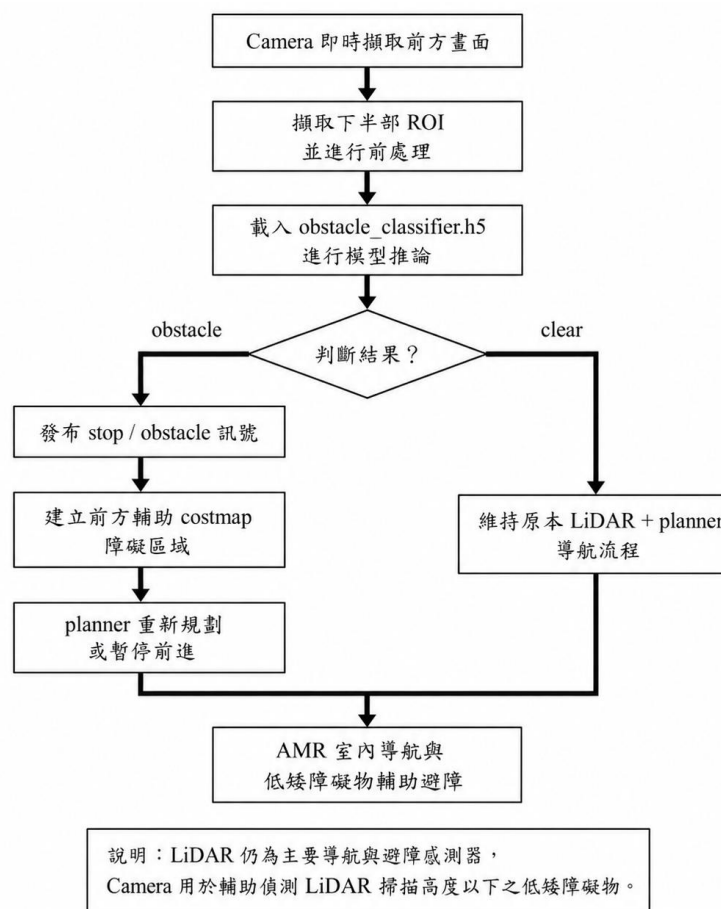
模型訓練流程如圖二十八所示，訓練集用於更新 CNN 模型參數，驗證集則用於觀察模型對未參與訓練資料的辨識能力。完成訓練後，可透過訓練準確率、驗證準確率、loss 曲線與混淆矩陣評估模型效果。



圖二十八、有無障礙物 CNN 模型訓練流程圖

有無障礙物即時辨識與導航整合流程如圖二十九所示，在與導航系統整合時，模型輸出可有兩種使用方式。第一種方式為 stop / clear 狀態判斷，當模型偵測到前方下半部 ROI 存在低矮障礙物時，系統可先暫停 AMR 移動，避免機器人直接前進造成碰撞或卡住；當判斷為 clear 時，則允許機器人繼續依照原路徑移動。第二種方式則是將 obstacle 結果轉換為輔助 costmap 資訊。由於 RGB Camera 無法提供精確深度，因此本專題以固定視角、固定距離與固定大小的方式，在 AMR 前方建立一個輔助障礙區域，

提供 planner 作為路徑規劃參考。此方式雖無法完整描述障礙物的實際幾何形狀，但可使 Camera 的辨識結果從單純畫面分類轉換為導航層可使用的避障資訊。



圖二十九、有無障礙物即時辨識與導航整合流程圖

整體而言，有無障礙物辨識模型訓練是 Camera 輔助避障流程中的關鍵步驟，此模型仍屬於輔助避障功能，主要適用於出發前檢查或低速移動狀態下的近距離判斷；LiDAR 仍作為 AMR 的主要導航與避障感測器。透過 LiDAR 與 Camera 的功能互補，本系統可在原有 SLAM、AMCL、costmap 與 move_base 導航架構上，進一步提升室內巡檢時的環境感知完整性與移動安全性。

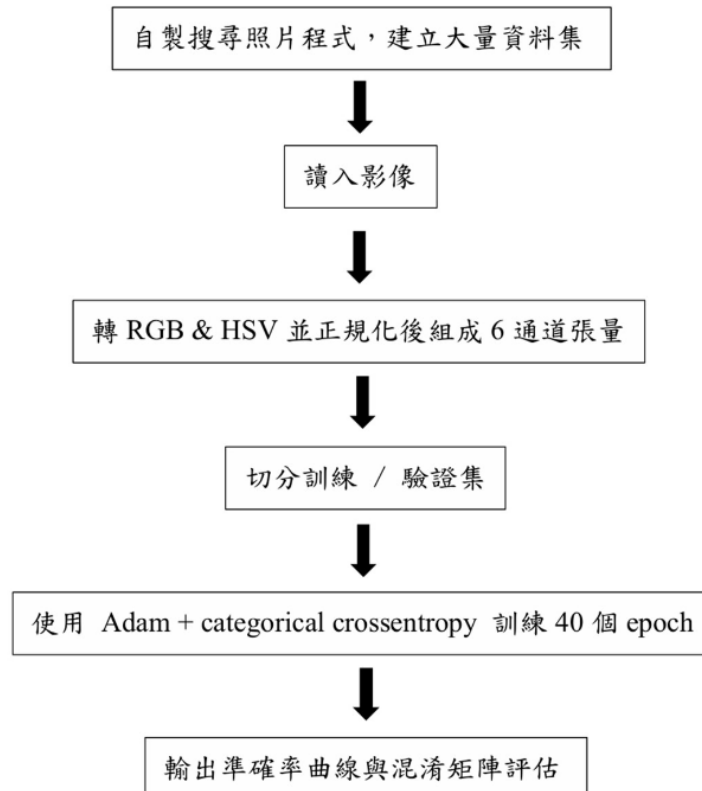
3.4.3 RGB+HSV 六通道 CNN 顏色辨識流程實作

進一步將影像資料應用於顏色辨識流程，使 AMR 在巡檢任務中能判斷指定目標物或模擬生理試紙之顏色狀態。在顏色類別設計上，模型所採用之顏色樣本並非任意選取，而是參考醫療檢測與室內巡檢情境中常見之顏色型態進行規劃。例如：尿液試紙、常見臨床檢測試紙、狀態指示標籤與設備燈號，常以紅、黃、綠、藍、黑、白、

灰及其深淺 variations 作為判讀或狀態標示依據。因此，本研究以多種常見深淺色系作為資料集基礎，使顏色辨識模組較能對應實際應用需求，而非僅作為一般性影像分類展示。資料建立時，先以 camera 擷取不同顏色樣本影像，再依類別進行整理，使後續模型能學習各顏色在實際攝影機畫面中的表現。由於相機影像會受到光線、角度與物體材質影響，因此資料蒐集時需盡量涵蓋不同拍攝條件，以降低模型只適用於單一場景的情形。

影像前處理是此流程中的關鍵步驟。camera 原始影像通常以 BGR 格式讀取，先將其轉換為 RGB 影像，並將影像尺寸統一調整為 64×64 ，以符合模型輸入格式。接著，再將 RGB 影像轉換為 HSV 影像，最後將 RGB 三通道與 HSV 三通道沿通道方向串接，形成六通道輸入資料。此處的處理重點在於使訓練資料與即時辨識資料採用相同格式，避免因前處理不一致造成模型輸入偏差。

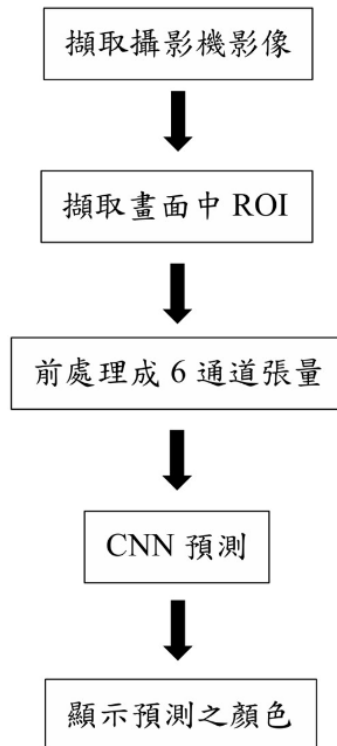
模型訓練流程如圖三十所示。資料集完成後，系統先讀取各類別影像並進行標籤編碼，接著執行 RGB 與 HSV 前處理，再將資料切分為訓練集與驗證集。模型架構採用 Keras Sequential 方式建立，以多層 Conv2D 與 MaxPooling2D 萃取影像特徵，最後接續全連接層與 softmax 輸出層進行 11 類顏色分類。訓練完成後，模型儲存為 color11_classifier.h5，供後續即時辨識程式載入使用。



圖三十、11 色辨識模型 color11_classifier.h5 之訓練程式流程圖

在即時辨識流程中，系統先啟動 camera 並取得連續影像，再從畫面中擷取固定 ROI 區域作為辨識範圍。選用固定 ROI 的目的，是減少背景干擾，使模型主要判斷指定區域內的顏色目標。擷取後的 ROI 會經過與訓練階段相同的影像尺寸調整、RGB 轉換、HSV 轉換與六通道串接，確保即時輸入資料與訓練資料具備一致格式。

即時辨識程式流程如圖三十一所示。完成前處理後，六通道影像會輸入已訓練完成的 CNN 模型，模型輸出各顏色類別的預測機率，系統再選取機率最高的類別作為辨識結果。辨識結果會以文字方式顯示於畫面上，並搭配框線標示 ROI 區域，使操作者能確認系統目前判斷的顏色位置與類別。此流程可將 camera 影像轉換為明確的顏色分類結果，供後續巡檢任務使用。



圖三十一、色辨識模型 color11_classifier.h5 之辨識程式流程圖

在 AMR 巡檢應用中，顏色辨識模組可作為任務判斷的輸入之一。當機器人抵達指定巡檢點後，Camera 可擷取目標區域影像，並透過 color11_classifier.h5 模型輸出顏色類別。若辨識結果屬於預先設定之異常顏色，系統可進一步觸發任務切換，例如標記該巡檢點、返回指定位置或前往護理站回報；若結果屬於正常狀態，則可繼續執行下一巡檢點任務。此設計使影像辨識不只停留在單純分類，而能與巡檢流程產生連結。RGB+HSV 六通道 CNN 顏色辨識流程將 Camera 影像轉換為可供任務判斷使用的分類資訊。訓練階段負責建立顏色分類模型，即時辨識階段則負責將模型應用於實際影像輸入。透過資料集建立、前處理一致化、模型訓練與 ROI 即時辨識，Camera 模組可進一步支援 AMR 的醫療巡檢應用，為後續結果分析中的準確率、混淆矩陣與實際辨識畫面提供測試基礎。

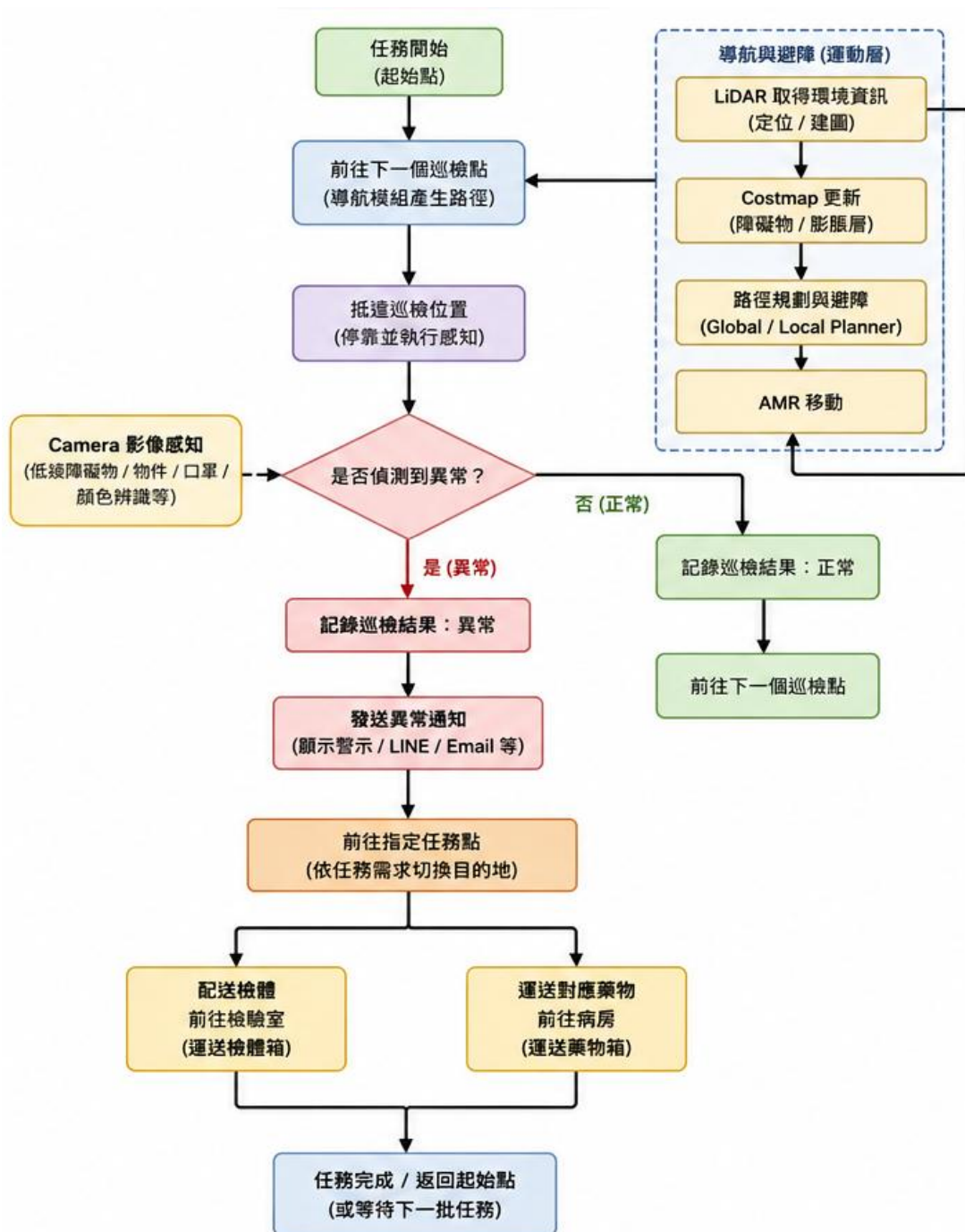
3.5 醫療病房巡檢任務流程

完成導航、避障與感知架構後，系統需進一步透過具體應用情境呈現整合效果。以醫療巡檢作為應用案例，規劃 AMR 在室內場域中由起始點出發，依序前往指定巡檢

點，並依據感知結果決定後續任務方向。此流程並非建立完整醫院派工系統，而是用於展示導航、感測融合與影像辨識功能如何被組織成可執行的任務流程。

醫療巡檢流程可視為多個目標點導航任務的組合。系統先將起始點、病房巡檢點與指定任務點設定為地圖中的可達位置，再依照場域動線安排移動順序。AMR 每次接收一個目標點後，由導航模組產生移動路徑，並在抵達指定位置後執行感知判斷。透過此方式，可將複雜巡檢任務拆解為連續且可追蹤的單點導航流程，降低整合與除錯難度。在此應用流程中，LiDAR 主要支援移動過程中的定位、建圖、路徑規劃與避障判斷；Camera 則於特定任務階段提供影像感知資料，例如：較低的障礙物輔助判斷、物件辨識、口罩辨識或顏色辨識。感知結果不直接取代導航流程，而是作為任務判斷條件，使系統能依照不同狀態切換下一個目標點。例如：當巡檢結果判定為正常時，AMR 可繼續前往下一巡檢點；若偵測到預設異常條件，則可標記該狀態，並切換至指定任務點或等待人工確認。

醫療巡檢應用流程是 Sensor Fusion AMR 系統的展示場景。透過巡檢點位規劃與任務切換設計，可說明系統如何由底層感測、導航與控制，延伸至具備應用意義的巡檢任務。

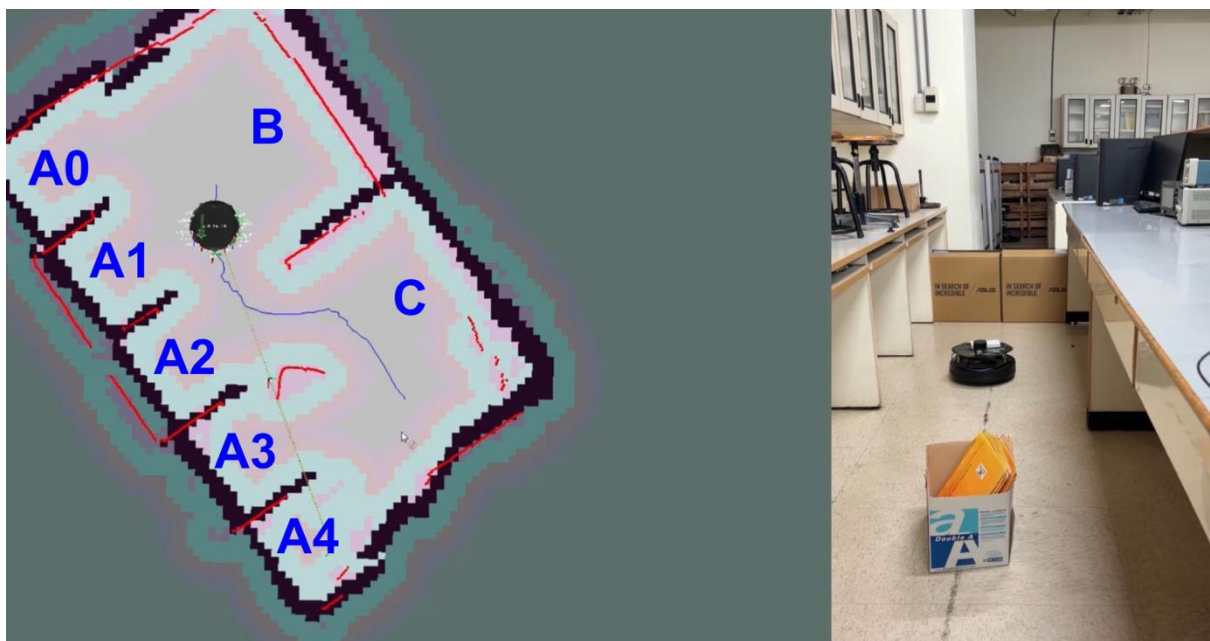


圖三十二、醫療病房巡檢任務流程圖

3.5.1 巡檢點位與移動順序規劃

巡檢點位規劃的目的，在於將醫療應用場景中的功能區域轉換為 AMR 可執行的導航目標。由於巡檢任務並非單一目標點移動，而是由多個定點任務串接而成，因此需先在已建立的地圖上標示各巡檢位置與指定任務點，使機器人能依照任務順序逐一前往，並於抵達後執行對應的感知或狀態判斷。

如圖三十三所示，A0 至 A4 為病房巡檢點，代表 AMR 於巡檢流程中需依序前往之主要位置；B 點為檢驗中心，可作為檢測結果異常或需進一步處理時的指定任務點；C 點為配藥房，可用於模擬藥品領取、配送或任務切換後之前往位置。透過此點位配置，巡檢流程不只包含病房之間的定點移動，也能依據不同狀態切換至檢驗中心或配藥房，使任務設計更接近實際醫療場域中的移動需求。



圖三十三、實機運行與 RViz 畫面

(A0~A4 為病房巡檢點，B 點為檢驗中心，C 點為配藥房)

在點位安排上，病房巡檢點主要分布於地圖左側可通行區域旁，模擬病房沿走廊排列的場景。此配置可使 AMR 依照固定順序完成巡檢，減少不必要的折返與路徑交錯。每一個病房巡檢點皆可視為一次獨立任務，機器人抵達後可短暫停留，並依任務需求執行影像擷取、狀態辨識或定點確認。完成該點任務後，系統再載入下一個目標點，使多點巡檢能以連續的單點導航方式完成。

B 點與 C 點則屬於延伸任務點，並非一般巡檢順序中的固定病房點位。B 點設定為檢驗中心，主要用於模擬巡檢過程中若出現需檢驗或需回報之狀態，AMR 可將後續目標切換至該位置；C 點設定為配藥房，可用於模擬藥品相關任務或指定物資位置。此設計使系統具備事件驅動任務切換的雛形，也能呈現 AMR 在醫療場域中不只執行巡邏，還能依狀態改變移動目標。

點位設定時需同時考量導航可行性與任務需求。若目標點過於靠近牆面、障礙邊界或狹窄轉角，機器人抵達後可能缺乏足夠的轉向空間，也可能影響定位穩定度與影像擷取角度。因此，巡檢點應設置於可通行區域內，並與牆面保持適當距離，使 AMR 能順利到達、停留與離開。對於需要進行影像辨識的點位，也應保留足夠的拍攝距離，避免畫面過近或角度受限。

任務順序可依場域動線由 A0 至 A4 依序進行，完成病房巡檢後再依結果決定是否前往 B 點或 C 點。若各巡檢點皆未觸發異常條件，系統可依照原路徑繼續完成剩餘巡檢；若某一點位的感知結果符合預設條件，則可暫停原巡檢順序，改以前往檢驗中心或配藥房作為下一目標。此方式能將固定路線巡檢與條件式任務切換結合，使 AMR 具備較彈性的應用流程。

3.5.2 異常辨識與任務切換設計

完成巡檢點位與任務順序規劃後，系統需進一步加入感知結果判斷，使 AMR 不只是依照固定順序前往各病房巡檢點，也能依據巡檢結果改變後續任務方向。此設計的重點，在於將 Camera 影像辨識結果轉換為任務控制條件，使巡檢流程具備事件驅動能力。當巡檢點狀態正常時，系統依照原先設定的順序前往下一個病房點位；當偵測到預設異常條件時，則暫停原巡檢順序，並切換至指定任務點。

在任務執行過程中，AMR 抵達巡檢點後會進入短暫停留狀態，並啟用對應的影像感知功能。以顏色辨識任務為例，Camera 會擷取指定 ROI 區域影像，將影像輸入顏色辨識模型後取得分類結果與信心值。系統再根據預先設定的異常條件進行判斷；若辨識結果屬於一般狀態，該巡檢點即被記錄為正常，並繼續前往下一病房；若辨識結果符合異常條件，則將該點標記為異常，並啟動後續任務切換流程。

在醫療巡檢應用中，顏色辨識結果可用於模擬檢測試紙、狀態標籤或警示色之判讀。若某一巡檢點辨識出指定異常顏色，該結果可被視為需要進一步處理的任務事件。此時，系統不再單純依序前往下一病房，而是依照任務邏輯切換至 B 點檢驗中心，模擬檢體配送、檢測回報或進一步確認流程。完成 B 點任務後，系統可再前往 C 點配藥房，模擬藥品領取、物資配送或後續處理。最後，AMR 可返回原巡檢點或接續未完成之巡檢任務，使異常事件處理形成完整閉環。

從任務邏輯來看， $A1 \rightarrow B \rightarrow C \rightarrow A1$ 的流程可代表一次完整的異常事件處理鏈。A1 為異常觸發點，B 點對應檢驗中心，C 點對應配藥房，最後返回 A1 則可視為回到原任務位置或進行後續確認。此流程將影像辨識、任務判斷與目標點導航串接起來，使 AMR 不再只是固定順序巡邏，而能根據現場狀態調整後續任務。

從系統架構角度來看，異常判斷、任務切換與導航控制分屬不同層級之功能模組。Camera 僅負責擷取影像並輸出辨識結果，不直接控制機器人移動；任務管理模組負責接收辨識結果、更新任務狀態並決定下一目標點；導航模組再依據目標點變更結果進行路徑規劃與移動控制。此種模組化設計可降低影像辨識、任務邏輯與底盤控制之間的耦合程度，使系統日後若需新增其他異常條件，例如口罩辨識、物件辨識或特定標籤辨識時，仍可沿用相同之任務切換機制，而無須大幅修改導航控制架構。

異常狀態判斷亦需兼顧流程穩定性。若單張影像即直接作為任務切換依據，可能因瞬時光線變化、相機晃動或辨識誤差造成誤判。因此，系統可透過連續有效畫面比例、有效幀數統計或時間條件設計，提高判斷可信度，再決定是否觸發異常流程。此種設計可使任務切換不致過於敏感，同時提升巡檢任務之穩定性與可重現性。

此種邏輯並不同於完整醫療警報系統，但已能呈現智慧巡檢平台由感知結果引導任務流程之雛形。若後續需發展為實際通知系統，則可再進一步整合資料上傳、使用者介面、遠端監控或訊息通知功能。透過異常辨識與任務切換設計，Camera 感知結果得以融入 AMR 巡檢流程，使系統由固定路線導航延伸為具備狀態判斷與流程調整能力的智慧巡檢架構。

第四章、測試或模擬分析結果與討論

測試重點依功能層級整理，首先檢查硬體平台、ROS 資料傳遞與座標關係是否能正常協同；接著分析地圖建立與定位結果，確認導航基礎是否穩定；再觀察路徑規劃、LiDAR 避障與 Camera 輔助感知在不同情境下的表現。影像辨識部分則以訓練曲線、混淆矩陣與實際辨識狀況評估模型效果，最後再討論巡檢應用流程與整體限制。

4.1 系統整合測試結果

系統整合測試主要確認 AMR 平台於模擬與實體操作中，各硬體模組、ROS 節點、感測資料與控制命令是否能共同運作。測試時，AMR 可正常啟動底盤控制、LiDAR 掃描、camera 影像輸入與 ROS 相關節點。LiDAR 資料能持續輸出至 /scan，底盤狀態能透過 /odom 回傳，速度控制命令可經由 /cmd_vel 傳送至 TurtleBot2，使機器人執行移動控制。camera 影像資料亦可作為後續影像感知模組的輸入來源，供低矮障礙物輔助判斷與語意辨識流程使用。

表二為系統整合測試項目與結果整理，測試內容包含感測器資料、底盤控制、座標轉換、影像輸入與導航模組連接等項目。結果顯示，各模組可於同一 ROS 架構下進行資料交換，並能支援後續 SLAM 建圖、定位、路徑規劃與感測融合測試。

測試項目	確認內容	測試結果
LiDAR 資料輸入	/scan 是否能持續發布	可正常取得雷射掃描資料
底盤里程計	/odom 是否能回傳移動狀態	可取得底盤移動資訊
速度控制命令	/cmd_vel 是否能控制底盤	可控制 AMR 前進與轉向
Camera 影像輸入	影像資料是否能被系統讀取	可取得前方影像
TF 座標資料	各 frame 是否能連接	可形成基本座標關係
導航模組連接	地圖、定位、規劃資料是否能銜接	可進入後續導航測試

表二、系統整合測試項目與結果

AMR 已能將感測輸入、狀態回饋與控制輸出串接於同一系統中。LiDAR 與底盤資料提供導航所需的距離與移動狀態，Camera 則提供額外影像來源，使系統具備進一步進行 Sensor Fusion 測試的條件。

4.2 SLAM 建圖與定位結果分析

SLAM 建圖結果主要用於確認 AMR 是否能依據 LiDAR 掃描資料建立可供導航使用的室內地圖。由於地圖品質會直接影響 AMCL 定位、Costmap 建立與 planner 規劃結果，因此此項測試是整體導航功能中相當關鍵的一環。

如圖三十四所示，地圖中黑色區域呈現牆面與障礙物邊界，淺色區域則代表機器人已探索且可供導航參考的空間。從整體輪廓可看出，SLAM 建圖已能描繪出室內場域的主要結構，包含外圍牆面、內部隔間與通道形狀。結果表示 LiDAR 掃描資料能被系統有效整合，並轉換為具有空間結構的地圖成果，使 AMR 不再只依靠即時感測資料判斷周圍環境，而是能以已建立地圖作為後續導航基礎。



圖三十四、RViz 中已建構之實驗場景

對於室內導航而言，外圍牆面與通道輪廓是地圖中最重要的參考特徵，因為 AMCL 定位時會利用 LiDAR 掃描結果與既有地圖進行比對；若牆面輪廓能在地圖中穩定呈現，定位模組便較容易估測機器人目前所在位置。換言之，地圖不僅是環境紀錄，也會成為後續定位與路徑規劃的共同依據。

從可通行區域來看，圖中淺色區域形成連續空間，表示機器人在建圖過程中已探索到主要移動範圍。這對後續 global costmap 與路徑規劃具有重要意義，因為 planner 需要根據地圖判斷哪些區域可通行、哪些區域應避免進入。若地圖中可通行區域中斷或邊界不清，系統可能產生過於保守的路徑，甚至出現無法規劃路線的情況。然而，圖中部分牆面邊界可觀察到鋸齒狀或局部不平整的情形，尤其在轉角、窄通道或內部障礙物附近較為明顯。此現象與 LiDAR 掃描解析度、機器人移動速度、轉向過程中的掃描角度變化，以及里程計累積誤差有關。當機器人旋轉或通過狹窄區域時，連續掃描資料之間若存在微小偏差，地圖邊界便可能出現不完全重合的情形。此類誤差在室內建圖中相當常見，通常不會完全阻止導航，但會影響地圖的平滑程度與定位穩定性。

從定位角度來看，地圖品質會影響 AMCL 對機器人位置的估測。若地圖中的牆面與實際環境特徵一致，LiDAR 掃描點便能與地圖輪廓較好地重合，定位結果也會較穩定；反之，若地圖邊界扭曲、重疊或缺少關鍵特徵，定位時可能出現位置偏移或收斂速度較慢的情況。因此，此地圖雖已能呈現主要環境結構，但在正式進行長距離巡檢任務前，仍需確認定位結果是否能持續與地圖維持一致。

對於醫療巡檢應用而言，SLAM 地圖的價值在於提供任務點位與移動路徑的空間基準。巡檢點、指定任務點與返回位置皆需建立在可通行地圖之上，若地圖與實際環境差異過大，目標點設定與路徑規劃便可能失準。因此，SLAM 建圖結果不只是單一功能成果，也會連帶影響後續巡檢流程能否順利執行。若要進一步提升地圖品質，可在建圖時降低機器人移動與旋轉速度，增加轉角及狹窄區域的掃描完整度，並避免在感測資料尚未穩定時快速切換方向。透過較穩定的移動方式與更完整的場域掃描，可減少地圖邊界偏移與局部重疊，使地圖更適合長時間導航與巡檢應用。

4.3 路徑規劃與 LiDAR 避障測試結果

完成 SLAM 地圖建立與 AMCL 定位後，導航系統需進一步確認 AMR 是否能依照目標點產生可執行路徑，並在移動過程中依據 LiDAR 掃描資料避開障礙物。此節主要分析地圖完成後之目標點導航與 LiDAR-based 避障表現，與前述建圖階段所使用之 random_walk 自主探索演算法不同。random_walk 主要用於未知地圖尚未完成時協助機器人自主探索；而本節所討論的路徑規劃與避障，則是在地圖已建立、定位已啟用後，由 navigation stack、costmap 與 planner 共同完成的導航結果。

在導航測試中，AMR 需先取得目前於地圖中的位置，再由系統接收目標點並產生前往該位置的規劃路徑。若地圖輪廓與定位結果穩定，global planner 可依據全域地圖產生由目前位置前往目標點的路徑；local planner 則會根據機器人周圍即時感測資訊，調整短距離移動方向與速度命令。此流程可將地圖、定位、路徑規劃與底盤控制串接起來，使 AMR 不只是完成地圖建立，也能在地圖中執行具體移動任務。

如圖三十五所示，RViz 畫面可呈現 AMR 於地圖中的位置、costmap 區域與規劃路徑。圖中的黑色邊界代表牆面或障礙物輪廓，彩色區域則為 costmap 對障礙物與安全距離的標記結果。當目標點被送出後，系統會依據地圖中的可通行區域規劃移動路線，使機器人避開牆面與不可通行區域。此結果表示 SLAM 地圖、AMCL 定位與 move_base 導航流程已能形成基本銜接，具備目標點導航能力。



圖三十五、AMR 於 RViz 中路徑規劃與 costmap 顯示結果

從測試畫面可觀察到，costmap 會在牆面與障礙物周圍形成安全緩衝區，使 planner 在規劃路徑時不會過度貼近環境邊界。此設定對 TurtleBot2 這類具實際底盤寬度的 AMR 相當重要，因為機器人本體並非單一點，若路徑過於靠近牆面或障礙物，即使中心點沒有碰撞，底盤邊緣仍可能擦撞環境。因此，costmap 中的障礙物標記與 inflation 區域可視為導航安全性的基礎條件。

在無明顯障礙物阻擋的情境下，AMR 可依照規劃路徑朝目標點移動。若定位穩定且地圖邊界清楚，路徑通常能保持在可通行區域內，並避免穿越牆面或已知障礙區域。此現象代表 global costmap 與 global planner 能根據已建立地圖判斷可行方向，local planner 則能依照機器人當前姿態與周圍空間產生短距離控制命令。從測試結果來看，導航流程已能支援基本目標點移動，符合後續病房巡檢與任務切換所需的基礎功能。

若測試場域中存在障礙物，LiDAR 掃描資料會反映周圍牆面、桌椅、箱體或其他高於 LiDAR 掃描平面的物體。這些資訊會被更新至 local costmap，使 planner 在規劃過程中避開障礙區域。當障礙物位於路徑附近時，AMR 不會只依照原本直線方向前進，

而是會依據 costmap 的障礙標記重新調整局部移動方向。此結果顯示 LiDAR 在導航階段仍是主要的平面避障感測來源，可協助系統處理一般高度的室內障礙物。

路徑規劃結果會受到地圖品質、定位穩定度、costmap 參數與機器人外型設定影響。若地圖邊界不平整，planner 可能將部分可通行區域判斷為障礙或未知區域，使路徑產生繞行；若 AMCL 定位發生偏移，機器人實際位置與地圖位置不一致，也可能導致路徑規劃方向與實際場域不完全對應。此外，inflation radius 若設定過大，雖然能提升安全距離，但可能使狹窄通道被判定為不易通行；若設定過小，路徑則可能過度靠近牆面與障礙物。這些現象說明路徑規劃結果並非只取決於目標點設定，而是由地圖、定位、感測資料與參數共同影響。

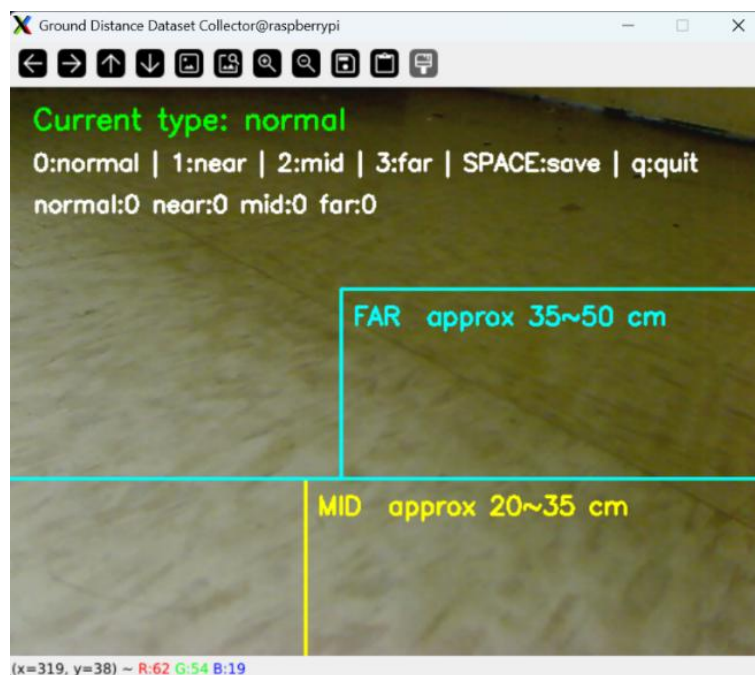
在 LiDAR 避障測試中，系統對一般高度障礙物具有較穩定的反應。當障礙物進入 LiDAR 掃描範圍後，local costmap 可將其標記為障礙區域，使 planner 產生避讓行為。此流程適合處理牆面、箱體、桌腳或其他能被 2D LiDAR 掃描平面偵測到的物體。若障礙物高度足以被 LiDAR 掃描到，系統通常能透過 costmap 更新與路徑修正降低碰撞風險。然而，LiDAR-based 避障仍存在感測限制。由於 RPLiDAR A1 為 2D 掃描方式，主要取得固定高度平面上的距離資訊，對於低於掃描平面的電線、紙團、低矮門檻或地面小物較不易完整偵測。此限制並不代表 LiDAR 導航無效，而是說明單一感測器在室內複雜環境中仍有盲區。因此，下節再進一步加入 Camera 輔助低矮障礙物避障結果，分析影像感知如何補足 LiDAR 對近地面障礙物感測不足的問題。

從測試結果可知，路徑規劃與 LiDAR 避障已能支援 AMR 在已知地圖中的基本導航任務。當目標點設定合理、定位狀態穩定，且障礙物能被 LiDAR 掃描到時，系統可產生可行路徑並依據環境狀態修正移動方向。這項成果為後續多點巡檢、異常狀態任務切換與 Camera 輔助避障提供基礎。後續改善方向則可針對 costmap 參數、local planner 平順度、狹窄通道通行能力與多感測器整合進一步調整，使導航行為更加穩定。

4.4 Camera 輔助低矮障礙物避障測試結果

本節針對 Camera 輔助低矮障礙物避障功能進行系統層測試，影像辨識結果轉換為 AMR 導航流程可使用的安全判斷資訊，由於 RPLiDAR A1 為 2D LiDAR，主要掃描固定高度之水平平面，對於電線、紙團、低矮物體、門檻或地面小物等低於掃描高度

之障礙物，無法完整反映於 /scan 資料與 costmap 中。本系統加入 Camera 作為輔助感知來源，判斷 AMR 前方近地面區域是否存在 LiDAR 不易偵測之障礙物。



圖三十六、Camera 分層 ROI 偵測區域示意圖

在測試下，Camera 影像以下半部 ROI 作為主要偵測區域。此區域對應 AMR 即將通過的近距離路面範圍，比完整影像更能反映低矮障礙物對移動安全性的影響。若使用整張影像進行判斷，牆面、桌面、遠方背景或畫面上方物件可能干擾模型輸出；下半部 ROI 可讓系統更專注於機器人前方實際行進區域，使影像辨識結果更容易與導航安全判斷產生對應。



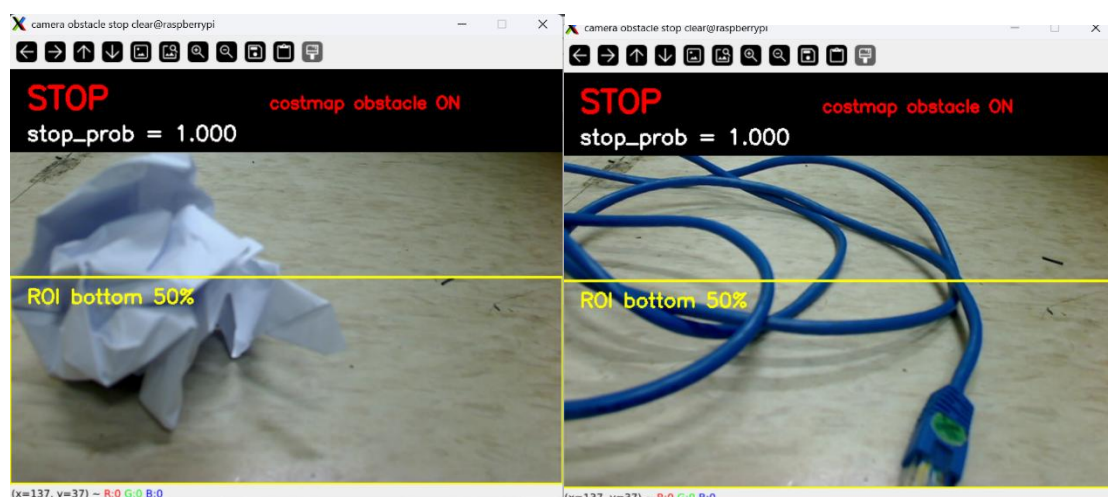
圖三十七、CLEAR 狀態辨識結果

當下半部 ROI 內為正常地面或可通行路面時，系統會輸出 CLEAR 判斷，並維持 costmap clear 狀態。結果表示 Camera 未偵測到需要停車或避開之低矮障礙物，AMR 可繼續依照原本由 LiDAR、SLAM、AMCL、costmap 與 planner 所組成的導航流程前進。CLEAR 狀態的不只是沒有障礙物，是避免系統在正常地面上頻繁誤停，使 AMR 能保持基本導航連續性。

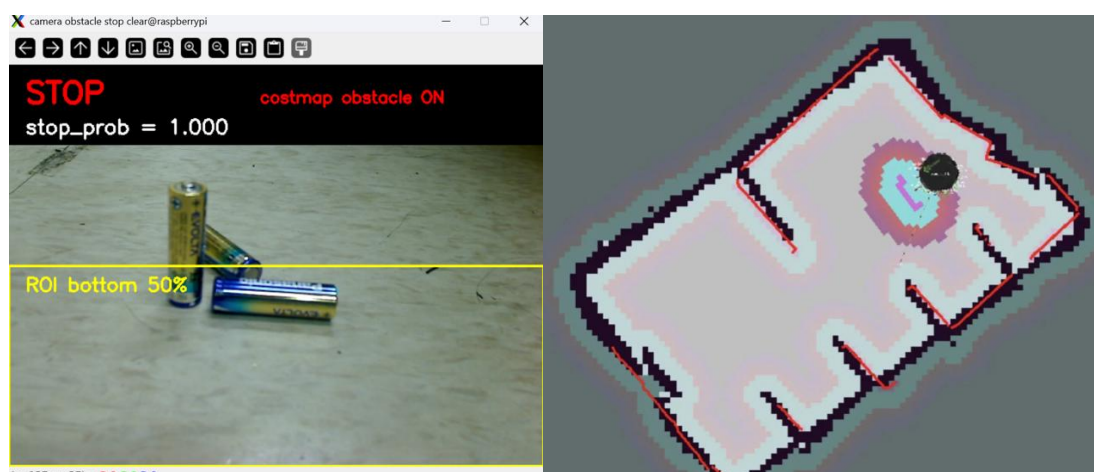


圖三十八、CLEAR 狀態

當電路板、電池、紙團、線材等低矮物體進入下半部 ROI 時，系統會輸出 STOP 或 obstacle 判斷，並可啟用 costmap obstacle 狀態或發布 stop 訊號。物體在室內環境中可能低於 LiDAR 掃描平面，或因體積小而不易被 2D LiDAR 完整偵測，但仍可能造成 AMR 卡住、捲入或碰撞風險。測試結果顯示，Camera 可透過影像特徵補足 LiDAR 對近地面障礙物感測不足的問題，使系統不只依賴水平距離掃描，也能取得前方地面狀態資訊。



圖三十九、STOP 狀態辨識結果



圖四十、障礙物偵測模型判斷結果(左)與 camera 產生之局域 cost(右)

完成影像判斷後，系統會將 Camera 結果轉換為導航層可使用的輔助資訊。當判斷為 CLEAR 時，AMR 維持原本路徑規劃與避障流程；當判斷為 STOP 或 obstacle 時，系統可依據相機固定安裝角度與前方視野範圍，在 AMR 前方建立固定距離與固定大小的輔助障礙區域，或直接暫停機器人移動。此方法雖然無法像深度相機一樣取得

障礙物的精確距離、形狀與實際佔用範圍，但可將 Camera 的影像辨識結果轉換為 planner 可參考的避障資訊，使影像感知不只停留在畫面分類，而能實際參與 AMR 的移動決策。

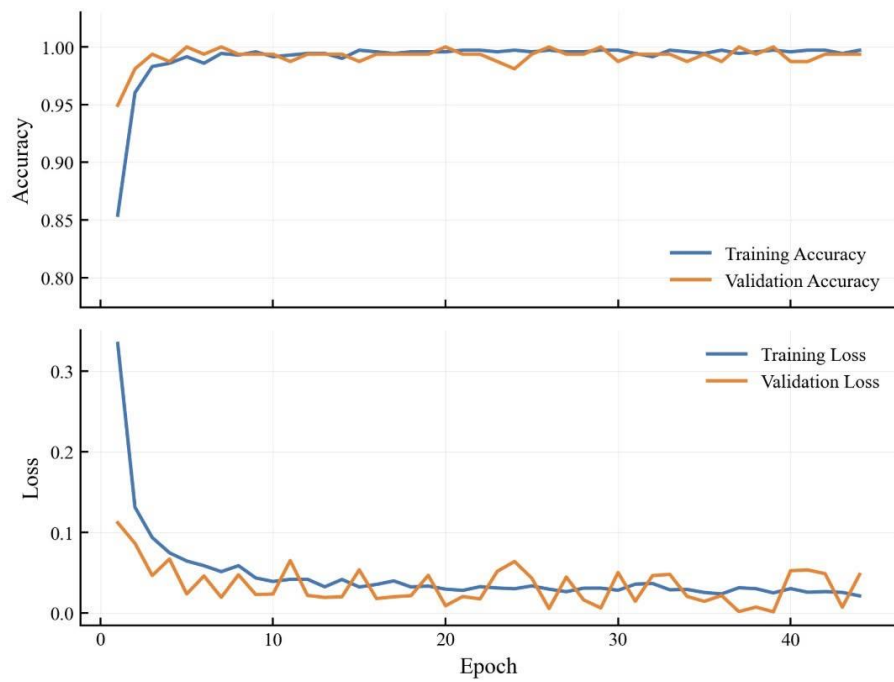


圖四十一、Camera costmap obstacle ON 畫面

整體測試結果顯示，Camera 可作為 LiDAR 導航系統的補充感測來源。透過 CLEAR / STOP 判斷與 costmap 輔助資訊，AMR 對前方近地面區域的感知能力更加完整，也提升了室內巡檢任務中的移動安全性。本階段仍屬於 Camera 輔助避障之初步驗證，受限於 RGB Camera 缺乏深度資訊與 Raspberry Pi 4 運算負載，目前較適合用於出發前檢查、低速移動狀態或定點巡檢時的安全判斷。

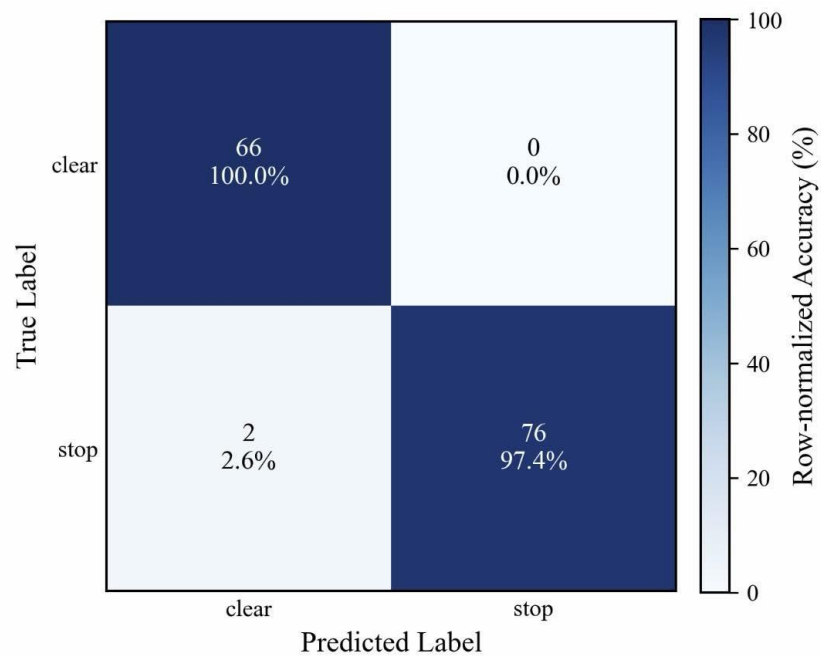
4.5 有無障礙物辨識模型訓練

本節針對有無障礙物辨識模型本身進行測試與分析，評估 CNN 模型對 clear 與 stop 兩類影像的分類能力。與 4.4 不同，本節專注於模型訓練過程、分類表現、實際樣本結果與誤判原因。此模型為 Camera 輔助避障流程中的核心判斷模組，其輸出結果會作為後續 CLEAR / STOP 狀態判斷之基礎。



圖四十二、有無障礙物辨識模型訓練與驗證曲線

驗證曲線如圖四十二所示。由 Accuracy 曲線可觀察到，訓練準確率在前期快速上升，表示 CNN 模型能於訓練初期逐漸掌握 clear 與 stop 兩類影像的主要差異。隨著 epoch 增加，訓練準確率與驗證準確率皆維持在高水準，且兩者差距不大，顯示模型不僅能學習訓練資料，也能對未參與訓練的驗證資料維持良好分類能力。由 Loss 曲線可觀察到，訓練 loss 在初期快速下降，代表模型預測結果與真實標籤之間的誤差逐漸減小；後期 loss 趨於穩定，表示模型已逐步收斂。驗證 loss 雖有小幅波動，但整體未出現明顯發散或嚴重過度擬合現象。



圖四十三有無障礙物辨識模型混淆矩陣

混淆矩陣如圖四十三所示。矩陣中橫軸為模型預測類別，縱軸為真實類別，對角線代表模型正確分類的樣本數，非對角線則代表誤判情形。由結果可知，clear 類共有 66 筆樣本被正確分類為 clear，未出現 clear 被誤判為 stop 的情形，因此 clear 類正確率為 100%。stop 類共有 76 筆樣本被正確分類為 stop，另有 2 筆 stop 樣本被誤判為 clear，stop 類正確率約為 97.4%。此結果顯示模型能穩定判斷一般可通行路面，同時也能有效辨識多數低矮障礙物情境。



圖四十四、與 stop 實際影像訓練資集

實際影像樣本如圖四十四所示。clear 類影像主要包含一般走廊、地板或無障礙路面，畫面中未出現會影響 AMR 前進的低矮障礙物；stop 類影像則包含電路板、紙

團、電池、線材或其他放置於地面上的小型物體。由樣本可看出，模型輸入主要聚焦於畫面下半部 ROI，並根據近地面區域是否存在障礙物進行分類。此結果說明模型並非只針對單一物體進行記憶，而是能根據不同障礙物外觀與地面狀態學習 clear / stop 影像特徵。

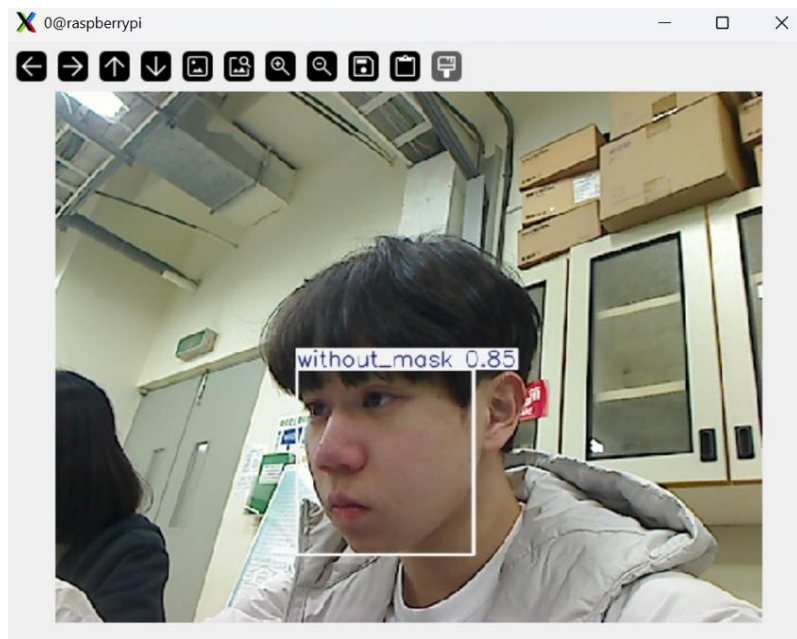
誤判情形主要可能來自三個原因。第一，光線變化會影響影像特徵，例如強烈陰影、反光或亮度不均，可能使障礙物輪廓變得不明顯。第二，地板材質與紋理可能造成干擾，例如地磚縫線、深色地毯紋路或地面污漬，可能與低矮障礙物的局部特徵相似。第三，若障礙物尺寸過小、顏色與地面接近，或位於 ROI 邊界附近，模型可能無法完整擷取其形狀與紋理特徵，造成 stop 被誤判為 clear。上述情形也是後續資料集擴充與模型改善的重要方向。

整體結果而言，識模型已具備良好的 clear / stop 二分類能力。訓練與驗證曲線顯示模型能穩定收斂，實際影像樣本亦顯示模型能區分正常可通行路面與低矮障礙物情境。後續若要提升模型可靠度，可增加不同光線條件、地板材質、障礙物種類、擺放角度與距離的資料，並調整 ROI 範圍與分類門檻，以降低誤判並提升實際巡檢場域中的穩定性。

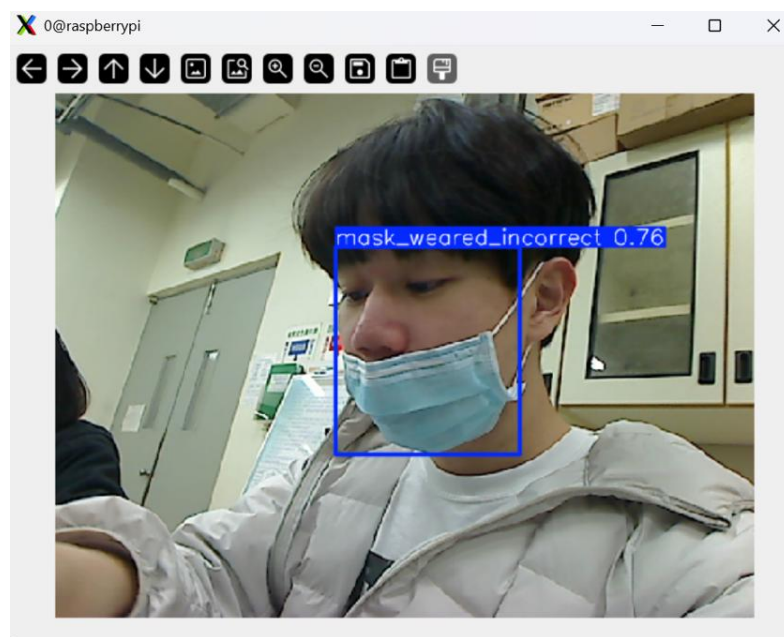
4.6 口罩辨識結果分析

口罩辨識流程採用動態 ROI 方式進行。當 Camera 擷取到巡檢畫面後，系統可依人臉位置擷取人臉附近區域，再將該影像送入口罩辨識模型進行判斷。與低矮障礙物固定使用下方 30% ROI 不同，口罩辨識的目標位置會受到人員高度、站立距離與拍攝角度影響，因此不適合使用固定區域。透過動態 ROI，可使系統依畫面中人臉位置調整辨識範圍，降低背景干擾並提高辨識有效性。

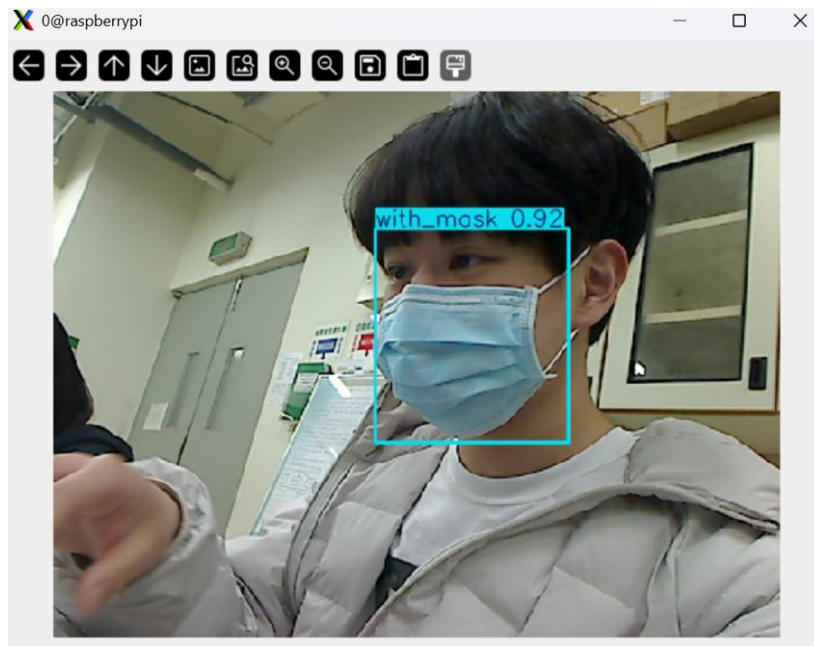
系統可於 Camera 畫面中辨識人員口罩配戴狀態，並將辨識結果顯示於畫面上。當判斷結果為已配戴口罩時，系統可將該狀態記錄為正常；若判斷結果為未配戴口罩或辨識結果符合異常條件，則可將此狀態標記為需注意事件。此結果顯示 Camera 不僅能用於導航輔助，也可支援巡檢任務中的人員狀態辨識。



圖四十五、未配戴口罩辨識畫面



圖四十六、未符合標準配戴口罩辨識畫面



圖四十七、口罩配戴標準辨識畫面

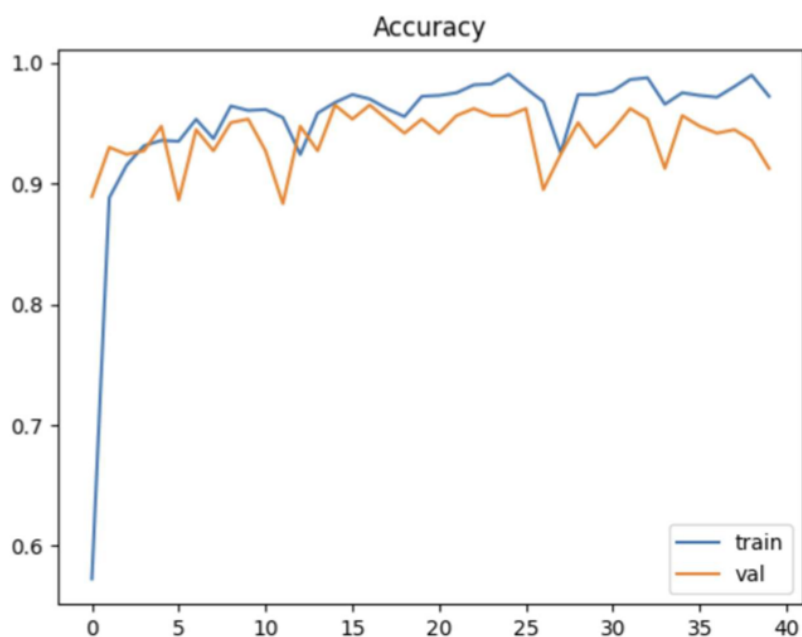
在巡檢應用中，口罩辨識結果可進一步作為任務判斷依據。若 AMR 於巡檢過程中偵測到未配戴口罩人員，系統可於終端機或遠端監控介面顯示異常狀態，並記錄發生位置與時間。若與任務切換流程結合，AMR 亦可暫停原定巡檢順序，前往指定回報點，或將事件資訊提供給管理端進行後續確認。此設計使口罩辨識不只是單純的影像分類結果，而能與巡檢任務流程產生連動，提升 AMR 在醫療或公共場域中的應用彈性。

需要注意的是，本階段口罩辨識仍屬於 Camera 語意感知功能之驗證，並未等同完整醫療管制或警報系統。若後續要進一步導入實際場域，仍需整合遠端通知、資料上傳、身分或區域管理，以及更完整的測試資料集。即便如此，本測試已能說明 Camera 在 Sensor Fusion 架構中可提供 LiDAR 無法取得的語意資訊，使 AMR 具備辨識人員狀態與輔助巡檢決策的能力。

4.7 顏色辨識結果分析

顏色辨識模型完成訓練後，以訓練準確率、驗證準確率與混淆矩陣作為主要評估依據。如圖四十八所示，訓練準確率在初期快速上升，表示模型於前幾個 epoch 已能掌握各顏色類別的主要特徵。隨著訓練進行，訓練準確率逐漸維持於高水準，驗證準確率亦大多保持在 0.9 以上，顯示模型對未參與訓練的驗證資料仍具有一定分類能

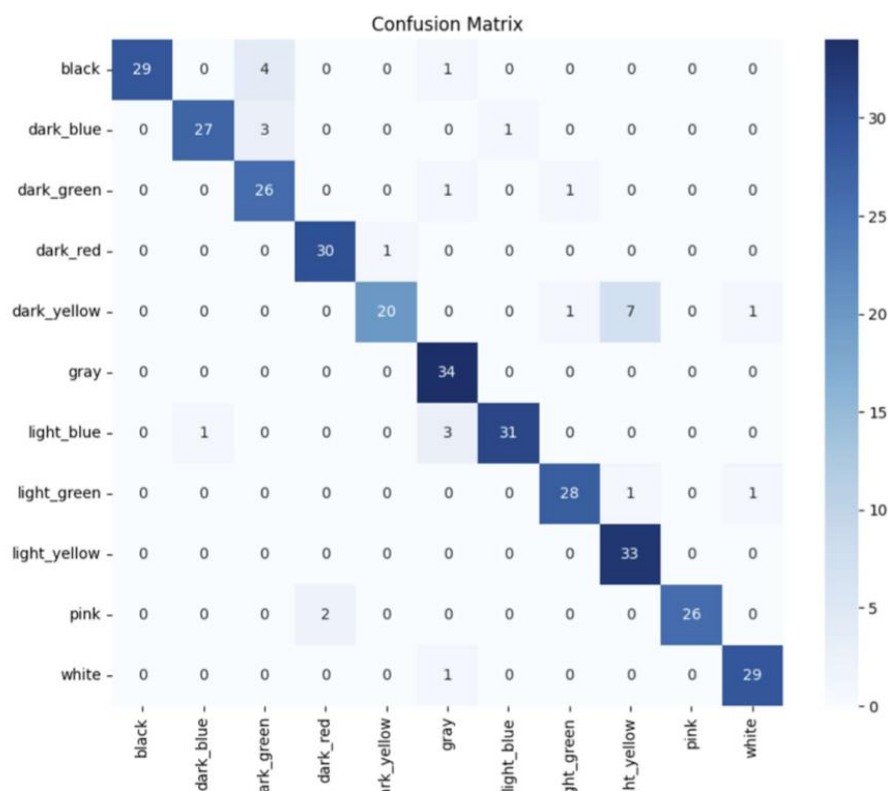
力。訓練曲線後段雖有小幅波動，但未出現驗證準確率長時間大幅下降的情況，因此未觀察到嚴重過度擬合現象。



圖四十八、11 色辨識模型 color11_classifier.h5 之訓練曲線

由訓練曲線可看出，訓練準確率普遍高於驗證準確率，屬於影像分類任務中常見的結果。兩者之間的差距並未持續擴大，表示模型雖對訓練資料具有較高擬合能力，但仍能維持基本泛化表現。驗證準確率在部分 epoch 出現下降，推測與資料量有限、同色系樣本差異較小、拍攝光源變化或驗證集樣本分布有關。由於顏色辨識容易受到光照、陰影與相機白平衡影響，驗證曲線的波動可視為實際應用時需要注意的現象。

混淆矩陣如圖四十九所示。多數顏色類別的預測結果集中於對角線，表示模型能正確區分大部分顏色樣本。其中，灰色、淺黃色、深紅色、白色與粉色等類別具有較明顯的分類效果，誤判數量相對較少。這類顏色在資料集中可能具有較穩定的色彩特徵，或與其他類別之間的差異較明顯，因此模型較容易建立判斷邊界。



圖四十九、11 色辨識模型 color11_classifier.h5 之混淆矩陣

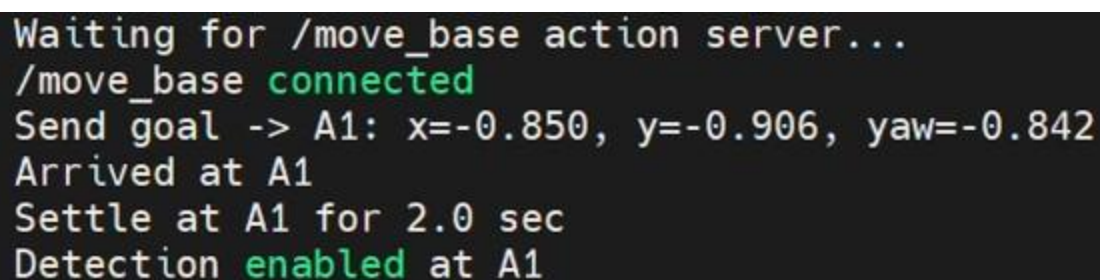
誤判主要出現在色相接近或亮度差異較小的類別之間。例如：深黃色與淺黃色、黑色與深綠色、淺藍色與灰色等類別，因顏色特徵在特定光線條件下可能相互接近，模型容易將亮度變化視為顏色差異，或將同色系深淺變化歸入相鄰類別。雖然能提供較完整的色彩資訊，但仍無法完全消除光源、反光與相機曝光造成的影響。從應用角度來看，顏色辨識模型已具備作為巡檢輔助判斷的基本能力。當目標顏色與背景差異明顯、光照條件穩定時，模型可提供較可靠的分類結果；若目標顏色屬於同色系深淺變化，或環境光源變動較大，則需加入更完整的樣本資料或輔助判斷機制。實際應用時透過固定拍攝距離、控制 ROI 範圍、增加不同光照條件下的訓練樣本，以及在判斷前連續擷取多張影像等方式，降低單次辨識誤判對任務流程造成的影響。

結果顯示，此可將 camera 影像轉換為可供任務判斷的顏色分類資訊。模型在多數類別上具備穩定辨識能力，但同色系深淺與光線變化仍是主要限制。後續若要提升在醫療巡檢情境中的可靠度，可增加資料集多樣性，特別是不同亮度、不同角度與不同背景下的樣本，使模型更能適應實際場域中的影像變化。

4.8 醫療巡檢應用流程測試結果

醫療巡檢任務測試主要用於確認第三章所規劃之點位配置與任務切換邏輯，是否能於實際系統中順利執行。測試內容包含起始位置前往第一個巡檢點、巡檢點影像辨識、異常狀態判定、任務目標切換，以及 B 點與 C 點之連續導航。此節著重於實際執行結果，因此以終端機輸出、Camera 辨識畫面與 RViz 導航路徑作為分析依據。

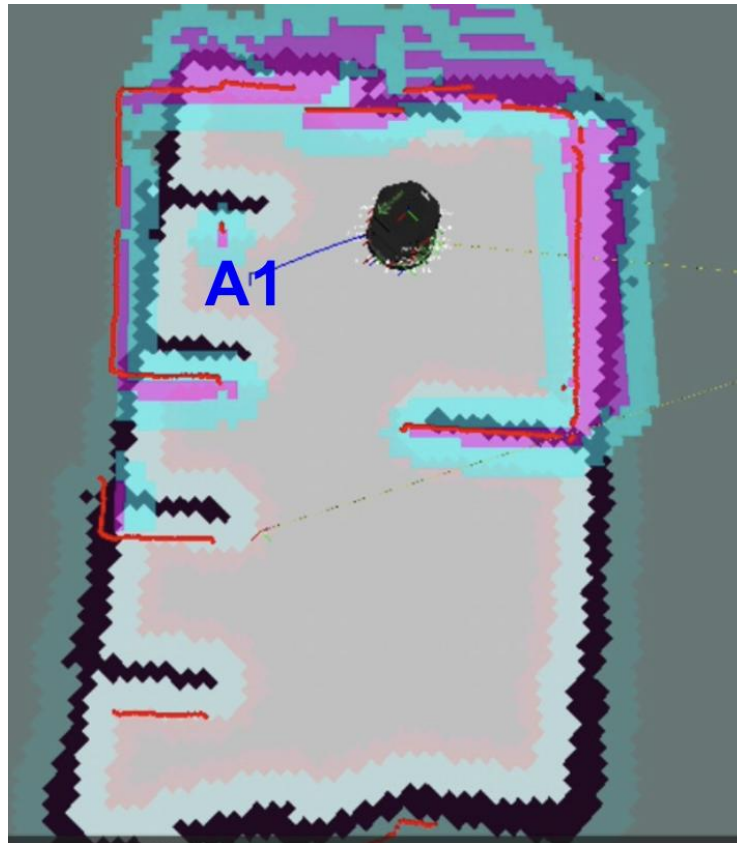
測試開始時，系統先將第一個巡檢點設定為 A1。由圖五十所示，終端機輸出可確認系統成功連接 move_base action server，並將 A1 座標送入導航模組。AMR 抵達 A1 後，系統停留指定時間並啟用影像偵測功能，表示巡檢任務已由單純目標點導航進入感知判斷階段。



```
Waiting for /move_base action server...  
/move_base connected  
Send goal -> A1: x=-0.850, y=-0.906, yaw=-0.842  
Arrived at A1  
Settle at A1 for 2.0 sec  
Detection enabled at A1
```

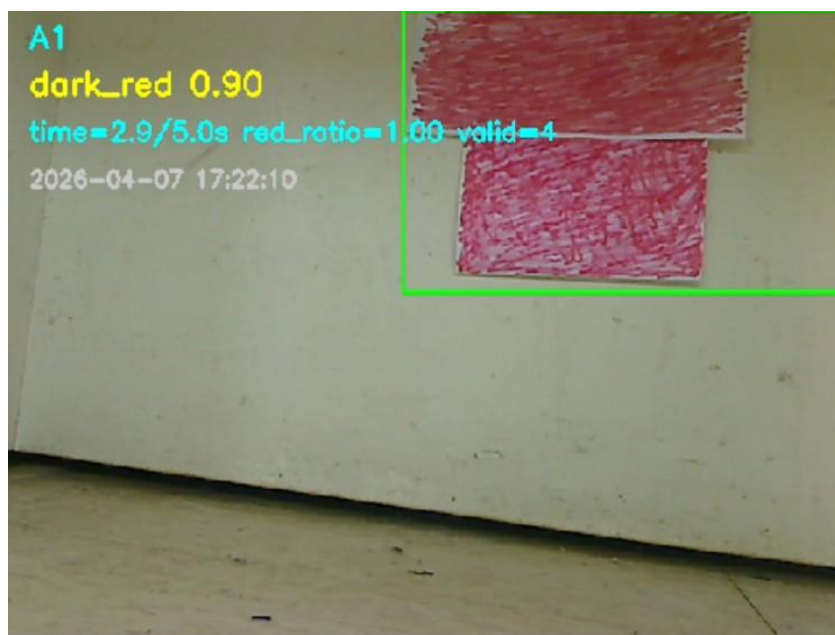
圖五十、AMR 前往 A1 巡檢點之終端機執行結果

圖五十一顯示 AMR 自起始位置前往 A1 巡檢點之 RViz 導航畫面。該畫面可用於觀察導航模組在地圖中產生之路徑，以及 AMR 於巡檢任務起始階段的移動狀態。



圖五十一、AMR 自起始位置導航至 A1 巡檢點之 RViz 路徑示意圖

AMR 抵達 A1 後，Camera 於指定 ROI 內進行顏色辨識。圖五十二所示，測試畫面顯示系統辨識出 dark_red，並輸出對應信心值。此結果表示影像感知模組可於巡檢點完成顏色判斷，並將辨識結果提供給任務管理邏輯使用。



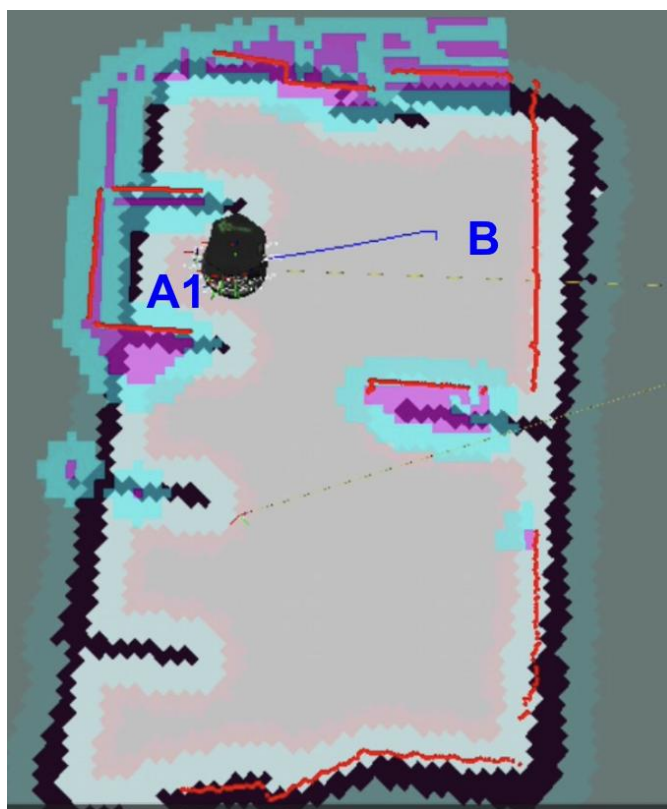
圖五十二、系統於指定 ROI 中辨識出 dark_red 異常顏色之相機圖

當 A1 被判定為異常後，終端機輸出顯示系統統計有效辨識幀數，判定 A1 發生 abnormal(red) 狀態，並接續送出前往 B 點的導航目標，如圖五十三所示。此結果顯示顏色辨識並非僅停留於畫面顯示，而是能實際觸發任務切換，使 AMR 從原巡檢流程轉向異常事件處理流程。

```
A1 result: red_frames=16, valid_frames=16, ratio=1.000  
A1: abnormal(red) detected -> B -> C -> back to A1  
Send goal -> B: x=-1.807, y=-0.268, yaw=0.734
```

圖五十三、終端機顯示 A1 巡檢點異常判定結果與後續任務切換訊息

圖五十四為 A1 異常觸發後重新規劃前往 B 點之 RViz 導航畫面。B 點在任務設定中對應檢驗中心，可用於模擬檢體配送、檢測結果回報或進一步確認流程。透過此結果可確認，任務目標更新後，導航模組能重新接收新目標並規劃對應路徑。



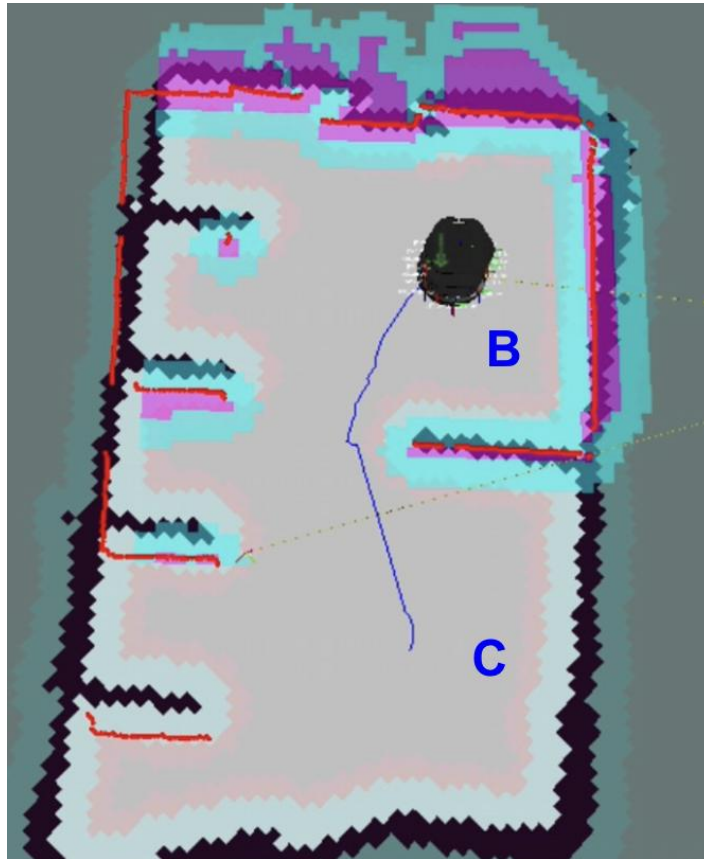
圖五十四、偵測異常後重新規劃前往 B 點之導航路徑

AMR 抵達 B 點後，系統設定短暫停留時間，再送出 C 點作為下一個導航目標。此流程可模擬異常檢體送達檢驗中心後，再前往配藥房或物資點執行後續處理。由終端機輸出可確認，系統能依照預設任務鏈自動更新下一階段目標，如圖五十五所示。

```
Stay at B for 2.0 sec  
Send goal -> C: x=-0.201, y=0.870, yaw=0.450
```

圖五十五、於 B 點停留後送出 C 點導航目標之終端機執行結果

圖五十六為 AMR 由 B 點前往 C 點之 RViz 導航畫面。B 點設定為檢驗中心，C 點設定為配藥房，因此此段路徑可視為異常事件後的後續處理流程。



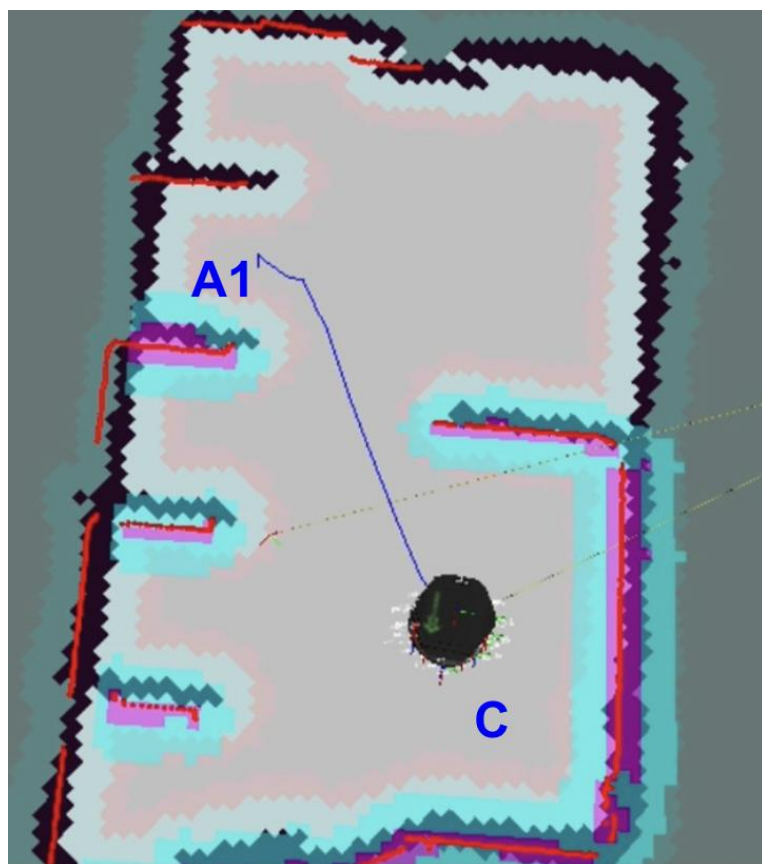
圖五十六、由 B 點前往 C 點之 RViz 導航路徑示意圖

終端機結果如圖五十七所示，AMR 抵達 C 點後，系統同樣設定短暫停留時間，並將下一目標設定為返回 A1。此結果表示系統能在完成檢驗中心與配藥房任務後，重新回到異常觸發點或原巡檢流程，形成較完整的任務閉環。

```
Stay at C for 2.0 sec  
Send goal -> A1(return): x=-0.850, y=-0.906, yaw=-0.842
```

圖五十七、於 C 點停留後送出返回 A1 目標之終端機執行結果

圖五十八為 AMR 由 C 點返回 A1 之 RViz 導航畫面。此路徑顯示異常任務完成後，系統可將目標切回原巡檢相關位置，使任務流程具備返回、復歸或接續巡檢的彈性。相較於單純到達指定點後結束任務，此結果更能呈現 AMR 依狀態執行多階段任務的能力。



圖五十八、由 C 點返回 A1 巡檢點之 RViz 導航路徑示意圖

由測試結果可知， $A1 \rightarrow B \rightarrow C \rightarrow A1$ 的任務鏈能順利串接影像辨識、異常判斷、目標點更新與路徑規劃。A 作為異常觸發點，B 點對應檢驗中心，C 點對應配藥房，最後返回 A1 代表系統具備任務復歸能力，此流程驗證了醫療巡檢應用不只是固定順序移動，而能依據感知結果改變後續任務方向。

第五章、結論與自評

5.1 研究收穫與結論

本專題以基於 Sensor Fusion 之 AMR 智慧導航系統為核心，整合 LiDAR、Camera、ROS 與 TurtleBot2 移動平台，完成室內自主移動、環境建圖、定位、路徑規劃、避障判斷與影像感知功能之建置。整體成果不僅呈現單一模組的實作能力，更重要的是將多項感測與導航功能串接為一套可執行的系統，使 AMR 能從環境感測、資料處理、任務判斷到移動控制形成完整流程。

在導航基礎方面，LiDAR 仍是系統中最主要的距離感測來源。透過 RPLiDAR A1 取得環境掃描資料後，系統能進行 SLAM 建圖，建立室內場域地圖，並進一步透過 AMCL 完成機器人在地圖中的定位。SLAM 地圖提供了後續導航所需的空間基準，使 AMR 能依據既有地圖進行路徑規劃，而不是只依賴即時感測結果做出短距離反應。從測試結果可知，建圖成果已能呈現主要牆面、通道與可通行區域，具備支援目標點導航與巡檢應用的基礎條件。

在地圖尚未完成前，為降低人工遙控需求，系統設計 random_walk 自主探索與避障演算法，使 AMR 能於未知環境中依據 LiDAR 前方與右側距離自行移動。此演算法並非取代地圖建立後的正式導航流程，而是服務於 SLAM 建圖階段，使機器人能在陌生場域中自主前進、沿牆與旋轉搜尋，協助取得更完整的環境掃描資料。透過此設計，建圖流程由人工控制逐步轉向自動化探索，也使 AMR 更符合全自動智慧移動平台的發展方向。

在 Sensor Fusion 架構方面，系統導入 Camera 作為 LiDAR 的視覺輔助感知來源。2D LiDAR 因掃描高度固定，對於低於掃描平面的電線、紙團、小型物體、門檻或地面雜物可能存在感測不足。Camera 則可透過影像取得車體前方近地面區域資訊，彌補 LiDAR 無法完整觀察低矮障礙物的限制。測試結果顯示，當障礙物落於下方 ROI 內時，Camera 模型可辨識出障礙狀態，並輸出 STOP 或 obstacle 判斷；當 ROI 內無障礙物時，則可維持 CLEAR 狀態，使原有導航流程不受額外干擾。此結果證明 Camera 可作為低矮障礙物感知補強來源，提升 AMR 對室內環境的判斷完整性。Camera 的功能並不侷限於低矮障礙物判斷，也能透過 ROI 切分支援不同任務需求。下方 ROI 可用於路面與

低矮障礙物偵測，中間 ROI 可用於顏色 tag、檢測試紙或狀態標籤辨識，上方 ROI 則可延伸至口罩辨識、人物辨識或物件辨識。這樣的設計使同一支 Camera 不只是單純影像擷取裝置，而是可根據任務目標轉換為多功能視覺感知來源。對於醫療巡檢、工廠巡邏或遠端監控等應用而言，Camera 所提供的語意資訊可補足 LiDAR 只有距離資料而缺乏目標外觀判斷的不足。

在顏色辨識部分，RGB+HSV 六通道 CNN 模型能對多數顏色類別進行有效分類，並可將辨識結果作為任務狀態判斷依據。選用多種深淺色系作為資料集，是因為尿液試紙、常見臨床檢測試紙、設備燈號、工廠狀態標籤與醫療標示常以顏色變化表達不同狀態。透過 Camera 擷取指定 ROI，再由顏色模型輸出分類結果，系統可進一步將異常顏色轉換為任務切換條件。此設計讓影像辨識不只停留在畫面分類，而能實際參與 AMR 任務控制。

醫療病房巡檢在本系統中作為應用驗證場景，用來展示導航、定位、影像感知與任務切換如何整合為一個連續流程。A0 至 A4 可作為病房巡檢點，B 點作為檢驗中心，C 點作為配藥房。當 AMR 於巡檢點偵測到異常顏色時，可將該點標記為異常，並依照任務邏輯前往 B 點與 C 點，模擬檢體配送、檢驗回報或藥物運送流程。這樣的設計使病房巡檢不只是固定順序移動，而是能依據感知結果改變後續任務方向，具備事件驅動任務控制的雛形。整體成果顯示，LiDAR、Camera 與 ROS 架構能在 AMR 平台上完成基本整合。LiDAR 提供穩定的地圖建立、定位與主要導航能力；Camera 補足低矮障礙物與語意辨識需求；ROS 則負責各節點之間的資料交換、座標轉換與控制命令傳遞。透過此架構，AMR 已具備由未知環境探索、地圖建立、定位導航、感知判斷到應用任務切換的完整雛形。

在實作過程中，也能明顯體會到系統整合比單一功能開發更具挑戰性。SLAM 建圖品質會影響定位穩定度，定位結果會影響路徑規劃，Costmap 參數會影響避障行為，Camera 推論速度則會影響輔助避障的即時性。任何一個模組不穩定，都可能連帶影響整體任務執行。因此，此次成果不僅建立了可運作的系統，也讓團隊理解 AMR 開發必須從感測、運算、控制與應用情境等多層面共同考量。

本專題完成的系統仍屬於雛形驗證階段，但已呈現出 Sensor Fusion 應用於 AMR 智慧導航的實際可行性。LiDAR 與 Camera 並非彼此取代，而是依各自特性分工：LiDAR 負責距離與空間結構，Camera 負責低矮區域與語意視覺資訊。兩者互補後，

AMR 對環境的理解能力更加完整，也為後續發展更高穩定度、更高即時性與更接近實際場域應用的智慧移動平台奠定基礎。

5.2 未來改善建議與發展展望

雖然系統已完成主要功能整合與初步測試，但若要進一步提升穩定性、即時性與應用完整度，仍有多個方向可持續改善。首先，運算平台效能是目前最明顯的限制之一。Raspberry Pi 4 需同時處理 ROS 導航節點、LiDAR 資料接收、TF Tree、AMCL、Costmap、move_base、Camera 影像擷取與 CNN 模型推論，當多個模組同時執行時，容易出現運算負載偏高、影像延遲或控制反應不夠即時的情形。未來若更換為具備 GPU 加速能力的嵌入式平台，例如：NVIDIA Jetson 系列，或將影像推論部分移至筆電端執行，可有效降低 Raspberry Pi 4 的負擔，並提升 Camera 即時判斷與導航整合的穩定性。

Camera 輔助低矮障礙物避障目前主要以靜態辨識與低速測試作為核心驗證。由於 RGB Camera 不具備深度量測能力，系統只能依固定視角、固定距離與固定大小假設建立輔助障礙區域。此方法可驗證 Camera 作為低矮障礙物感知補強來源的可行性，但仍無法精確描述障礙物的實際距離、形狀與佔用範圍。後續若導入 RGB-D Camera、Kinect 或雙目相機，便可取得深度資訊，使 Camera 感知結果更接近 LiDAR 或深度感測資料，進一步提高障礙物定位精度。

在感測融合層面，未來可由目前的功能層級互補進一步發展為更完整的資料融合。現階段 LiDAR 與 Camera 主要依功能分工進行整合，LiDAR 負責導航與距離資訊，Camera 負責低矮障礙物與語意辨識。後續可將 Camera 偵測結果更直接地轉換為 costmap layer，並依據障礙物位置、信心值與時間連續性動態更新代價地圖，使 planner 能更即時地根據 Camera 資訊修正路徑。若能建立穩定的 Camera-to-base_link 座標轉換與障礙物投影方式，Camera 輔助避障將可由靜態驗證提升為動態整合。

SLAM 與定位功能仍有提升空間。建圖時若移動速度過快、轉角掃描不足或里程計誤差累積，地圖邊界可能出現鋸齒、不平整或局部重疊，進而影響 AMCL 定位與路徑規劃結果。未來可從兩個方向改善：一是調整建圖過程中的移動策略，使機器人以更平穩速度探索環境，並增加轉角與狹窄區域的掃描完整度；二是導入更先進的

SLAM 方法或感測器組合，例如：結合 IMU、深度相機或輪速校正，以降低單一 LiDAR 與里程計造成的定位誤差。

自主探索演算法也可進一步優化，目前 random_walk 主要依據前方距離與右側距離在 FOLLOW_WALL 與 SPIN_SEARCH 模式之間切換，能支援建圖期間的基本自主移動。然而，這類規則式方法對複雜場域仍可能出現探索效率不足或重複掃描區域的情形。未來可加入 frontier-based exploration 等自主探索策略，使 AMR 根據地圖中未知區域自動選擇探索方向，減少無效移動，並提高建圖完整度。若能將 random_walk 作為初步安全移動機制，再結合地圖邊界分析與未知區域搜尋，建圖階段的自動化程度將更加完整。

影像辨識模型方面，資料集多樣性仍是影響模型穩定度的關鍵。顏色辨識容易受到光照、陰影、反光、相機白平衡與背景顏色影響；有無障礙物模型則可能受到地板材質、障礙物尺寸、拍攝角度與模糊程度影響。後續可增加不同光源、不同地面材質、不同距離與不同障礙物種類的資料，並加入資料增強方法，以提升模型面對真實環境變化時的泛化能力。若要提高物件辨識與口罩辨識效果，也可導入 YOLO、MobileNet SSD 或其他輕量化物件偵測模型，使 Camera 不僅能分類 ROI，也能定位畫面中的目標物。

ROI 切分方式未來也可由固定區域改為動態區域。現階段下方 ROI、中間 ROI 與上方 ROI 分別對應低矮障礙物、顏色 tag 與口罩或物件辨識，能使不同任務有明確分工。然而，實際場域中的目標物位置可能會因相機角度、機器人距離或環境配置而變動。若未來加入目標偵測模型，系統可根據目標位置動態調整 ROI，而不是固定取畫面某一區域。如此可降低背景干擾，也能使 Camera 更靈活地支援多種巡檢任務。

醫療巡檢應用目前主要以任務流程模擬為主，後續若要更接近實際場域，需補強任務管理與資料紀錄功能。例如：系統可建立巡檢紀錄表，記錄各巡檢點的抵達時間、辨識結果、異常狀態與後續處理流程；若偵測到異常，可將資料上傳至雲端或顯示於遠端監控介面，供人員即時查看。若進一步整合通知系統，便可在異常發生時自動發送提醒，讓 AMR 不只是移動平台，也能成為醫療或工廠環境中的資料收集與狀態回報節點。

在導航應用方面，未來可提升多點任務的彈性。現階段巡檢流程以預設點位順序與簡單條件切換為主，若要應對更複雜任務，可導入任務排程機制，使系統根據異常優先度、路徑距離、任務類型或電量狀態動態調整巡檢順序。例如，在多個病房同時發生異常時，AMR 可依距離或事件等級決定先前往哪一點；若電量不足，則可優先返回充電點；若某通道受阻，則可重新規劃任務順序。這些功能將使 AMR 更接近實際服務機器人的運作需求。

硬體結構也有改善空間，Camera 安裝角度會直接影響 ROI 判斷結果，LiDAR 安裝高度則會影響可偵測障礙物範圍。未來可設計更穩定的固定架，確保相機視角與 LiDAR 掃描平面在長時間運行中不易偏移。若加入可調角度或多相機配置，Camera 可同時觀察前方、低矮區域與較高位置目標，使語意感知範圍更完整。底盤部分則可進一步檢查輪子打滑、里程計誤差與轉向穩定性，以改善定位與導航表現。

軟體架構方面，未來可考慮由 ROS1 逐步轉向 ROS2 與 Nav2。ROS2 在節點通訊、即時性、安全性與多機協作方面具有更完整的設計，Nav2 也提供更模組化的導航框架與行為樹任務管理能力。若系統未來需要支援更複雜的任務流程、遠端監控、多感測器整合或多機器人協作，ROS2 會是值得發展的方向。不過，轉換平台也代表需要重新整理節點、參數、Topic、TF 與 navigation stack，因此可作為中長期改善方向逐步導入。

從應用展望來看，此系統不只適用於醫療病房巡檢，也可延伸至工廠巡檢、倉儲巡邏、實驗室監控與日常服務場域。在工廠環境中，顏色標籤、設備燈號或警示牌可作為事件觸發條件；在醫療場域中，檢測試紙、病房標籤或人員口罩狀態可成為巡檢判斷依據；在一般室內服務環境中，低矮障礙物偵測可降低機器人被線材、地毯或門檻卡住的風險。隨著感測器、運算平台與影像模型逐步提升，Sensor Fusion AMR 具有擴展至更多場域的潛力。

未來發展的核心方向，是讓 AMR 從「能移動」逐漸提升為「能理解環境並根據狀態行動」。LiDAR 提供空間結構，Camera 提供視覺語意，任務邏輯則將感測結果轉換為行動決策。若後續能強化即時運算、深度感測、動態 costmap、模型泛化與任務管理機制，系統將能更穩定地面對真實場域中的複雜變化，並朝向更完整的智慧巡檢與自主服務平台發展。

第六章、參考資料

- [1] 樹莓派官方網站 <https://www.raspberrypi.com/products/raspberry-pi-4-model-b/>
- [2] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Y. Ng, “ROS: an open-source Robot Operating System,” *Proceedings of the IEEE International Conference on Robotics and Automation Workshop on Open Source Software*, pp. 1–6, 2009.
- [3] M. Quigley, B. Gerkey, and W. D. Smart, *Programming Robots with ROS*, O’Reilly Media, 2015.
- [4] Y. Pyo, H. Cho, R. Jung, and T. Lim, *ROS Robot Programming*, ROBOTIS Co., Ltd., 2017.
- [5] A. Koubaa, Ed., *Robot Operating System (ROS): The Complete Reference*, Springer, 2016.
- [6] J. a. S. S. Zhang, “LOAM: Lidar Odometry and Mapping in Real-time,” 7 2014.
- [7] Pendse, Pratiksha. 2024. "An Overview on LiDAR for Autonomous Vehicles." Chemnitz University of Technology. <https://www.researchgate.net/publication/379900256>
- [8] Dummies. "Understanding LiDAR: Types and Applications." <https://www.dummies.com/article/technology/electronics/gadgets/understanding-lidar-types-and-applications-251234/>
- [9] Durrant-Whyte, Hugh, and Tim Bailey. "Simultaneous Localisation and Mapping (SLAM): Part I The Essential Algorithms." *IEEE Robotics & Automation Magazine*, 13(2), 99–110. <https://www.researchgate.net/publication/220320706>
- [10] Zhu, H. (2024, May 9). Indoor localization of mobile robots based on the fusion of an improved AMCL algorithm and a collision algorithm. *IEEE*, 17, 67188-67208. <https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=10528290>
- [11] Peavy, M., Kim, P., Oyediran, H., & Kim, K. (2023). Integration of Real-Time Semantic Building Map Updating with Adaptive Monte Carlo Localization (AMCL) for Robust Indoor Mobile Robot Localization. *Applied Sciences*, 13(2),

909.https://www.researchgate.net/figure/Modifications-to-the-AMCL-default-parameters-for-the-kidnapped-robot-problem_tbl1_366983180

[12] Li, G., Meng, J., Xie, Y., Zhang, X., Huang, Y., Jiang, L., & Liu, C. (2019). Reliable and fast localization in ambiguous environments using ambiguity grid map. *Sensors*, 19(15), 3331.<https://doi.org/10.3390/s19153331>

[13] James F. Kurose, Keith W. Ross, 'Computer Networking: A Top-Down Approach', 7th ed., Pearson Education, 2017.

[14] Yan, Y. (2023). Research on the A Star Algorithm for Finding Shortest Path. *Highlights in Science, Engineering and Technology: MCEE 2023*, 46, 154-161.

https://www.researchgate.net/publication/370573811_Research_on_the_A_Star_Algorithm_for_Finding_Shortest_Path

[15] Fox, D., Burgard, W., & Thrun, S. (1997). The dynamic window approach to collision avoidance. **IEEE Robotics & Automation Magazine**, 4(1), 23-33.
<https://doi.org/10.1109/100.580977>

[16] Open Source Robotics Foundation, “sensor_msgs/LaserScan Message,” *ROS Documentation*, sensor_msgs package, 2024.

[17] R. B. Rusu and S. Cousins, “3D is here: Point Cloud Library (PCL),” *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pp. 1–4, 2011.

[18] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Y. Ng, “ROS: an open-source Robot Operating System,” *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA) Workshop on Open Source Robotics*, pp. 1–6, 2009.

[19] D. V. Lu, D. Hershberger, and W. D. Smart, “Layered costmaps for context-sensitive navigation,” *2014 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 709–715, 2014.

[20] K. Zheng, “ROS Navigation Tuning Guide,” *arXiv preprint arXiv:1706.09068*, pp. 1–7, 2016.

[21] MathWorks, “Understanding Color Spaces and Color Space Conversion,” *MathWorks Documentation*. Available: MathWorks official documentation.

- [22] Z. Xian, R. Huang, D. Towey, and C. Yue, “Convolutional Neural Network Image Classification Based on Different Color Spaces,” *Tsinghua Science and Technology*, Vol. 30, No. 1, pp. 402–417, 2025.
- [23] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, pp. 436–444, 2015.
- [24] T. Foote, “tf: The Transform Library,” *2013 IEEE Conference on Technologies for Practical Robot Applications (TePRA)*, pp. 1–6, 2013.
- [25] Open Source Robotics Foundation, “About tf2,” *ROS 2 Documentation: Foxy documentation*, 2020.
- [26] Open Source Robotics Foundation, “REP 105 — Coordinate Frames for Mobile Platforms,” *ROS Enhancement Proposals*, 2010.

第七章、指導老師簽意見（簽名）